



iCET CERTIFIED DEVELOPER

DATABASE MANAGEMENT SYSTEMS

Registration No	PR26122170
Name	PERINPAM SENTHURAN
Batch	122 WEEKDAYS

TABLE OF CONTENTS

Data and Information.....	9
DATA.....	9
INFORMATION.....	9
Database and DBMS.....	10
DATABASE.....	10
DBMS – DATABASE MANAGEMENT SYSTEM.....	10
Relational database management system.....	11
non-relational database management system.....	11
CLIENT-dbms-db CONCEPT.....	12
Main features of MySQL.....	12
Data modals.....	13
evolution of data modals.....	15
BUSINESS RULES.....	15
entity relationship (er) model.....	16
degrees of relationships.....	18
MySQL.....	20
DATATYPES.....	21
1.01 TINYINT.....	21
1.02 SMALLINT.....	21
1.03 MEDIUMINT.....	22
1.04 INT.....	22
1.05 BIGINT.....	23
1.06 FLOAT.....	23
1.07 DOUBLE.....	24
1.08 DECIMAL.....	24
1.09 BOOLEAN.....	25
2.01 CHAR.....	25
2.02 VARCHAR.....	26
2.03 TEXT.....	26

2.04 ENUM	27
2.05 SET	27
2.06 BLOB	28
3.01 DATE	28
3.02 TIME	29
3.03 DATETIME	29
3.04 TIMESTAMP	30
3.05 YEAR	30
4.01 JSON	31
4.02 BLOB	31
MySQL Constraints.....	44
1. NOT NULL	44
2. UNIQUE	44
3. PRIMARY KEY	48
4. FOREIGN KEY	50
5. AUTO_INCREMENT	54
6. DEFAULT	60
7. CHECK	64
MySQL Operators.....	66
1. ARITHMETIC OPERATORS	66
1.01 + (Addition)	66
1.02 - (Subtraction)	66
1.03 * (Multiplication)	66
1.04 / (Division)	67
1.05 DIV (Integer Division)	67
1.06 % (Modulus)	67
1.07 MOD (Modulus)	68
2. Comparison Operators	69
2.01 = (Equal To)	69
2.02 != or <> (Not Equal To)	69
2.03 > (Greater Than)	69
2.04 < (Less Than)	69
2.05 >= (Greater Than or Equal To)	70

2.06	<= (Less Than or Equal To)	70
2.07	IS (Checks , Boolean or NULL)	70
2.08	IS NOT (Check , Not Boolean or NULL)	71
2.09	BETWEEN (Between a Range)	71
2.10	NOT BETWEEN (Not Between a Range)	71
2.11	IN (Match Any Value in List)	72
2.12	NOT IN (Not Match Any Value in List)	72
2.13	LIKE (Pattern Matching)	72
2.14	NOT LIKE (Not Pattern Matching)	73
3.	Logical Operators	74
3.01	AND / &&	74
3.02	OR /	74
3.03	NOT / !	74
4.	Subquery Operators	76
4.01	ALL	76
4.02	ANY / SOME	76
4.03	EXISTS	76
5.	Assignment Operators	78
5.01	= (Assignment)	78
5.02	:= (Assignment)	78
6.	SET Operators	79
6.01	UNION	79
6.02	UNION ALL	79
6.03	INTERSECT (MySQL 8.0.31+)	79
6.04	EXCEPT (MySQL 8.0.31+)	80
	MySQL Clauses	81
1.	QUERY CLAUSES	81
1.01	SELECT	81
1.02	DISTINCT	82
1.03	FROM	83
1.04	WHERE	84
1.05	GROUP BY	85
1.06	WITH ROLLUP	86

1.07	HAVING	87
1.08	ORDER BY	88
1.09	LIMIT	89
1.10	OFFSET	90
1.11	INTO	91
2.	JOIN CLAUSES	92
2.01	INNER JOIN	92
2.02	LEFT JOIN	93
2.03	RIGHT JOIN	94
2.04	CROSS JOIN	95
2.05	SELF JOIN	96
2.06	ON	97
2.07	USING	98
3.	SET OPERATION CLAUSES	99
3.01	UNION	99
3.02	UNION ALL	100
3.03	INTERSECT	101
3.04	EXCEPT	102
4.	CTE CLAUSES	103
4.01	WITH	103
4.02	WITH RECURSIVE	104
5.	WINDOW FUNCTION CLAUSES	106
5.01	OVER()	106
5.02	PARTITION BY	107
5.03	ROWS BETWEEN	108
5.04	RANGE BETWEEN	109
5.05	UNBOUNDED PRECEDING	110
5.06	CURRENT ROW	111
5.07	UNBOUNDED FOLLOWING	112
6.	DATA MANIPULATION (DML) CLAUSES	113
6.01	INSERT INTO	113
6.02	VALUES	114
6.03	SET	115

6.04 UPDATE	116
6.05 DELETE	117
6.06 REPLACE INTO	118
6.07 ON DUPLICATE KEY UPDATE	119
6.08 IGNORE	120
7. DATA DEFINITION (DDL) CLAUSES	121
7.01 CREATE TABLE	121
7.02 ALTER TABLE	122
7.03 DROP TABLE	123
7.04 TRUNCATE TABLE	123
7.05 RENAME TABLE	124
7.06 CREATE INDEX	125
7.07 DROP INDEX	126
7.08 CREATE VIEW	127
7.09 DROP VIEW	128
7.10 CREATE DATABASE	129
7.11 DROP DATABASE	129
8. CONDITIONAL CLAUSE	130
8.01 CASE WHEN	130
9. SUBQUERY CLAUSES	131
9.01 FROM (Subquery)	131
9.02 WHERE (Subquery)	132
MySQL Functions.....	133
1. STRING FUNCTIONS	133
1.01 CONCAT()	133
1.02 LENGTH()	133
1.03 CHAR_LENGTH()	133
1.04 UPPER()	134
1.05 LOWER()	134
1.06 TRIM()	134
1.07 LTRIM()	135
1.08 RTRIM()	135
1.09 SUBSTRING()	135

1.10	LEFT()	136
1.11	RIGHT()	136
1.12	REPLACE()	136
1.13	REVERSE()	137
1.14	REPEAT()	137
1.15	INSTR()	137
1.16	LPAD()	138
1.17	RPAD()	138
1.18	SPACE()	138
1.19	FORMAT()	139
2.	Numeric Functions	140
2.01	ABS()	140
2.02	CEIL() / CEILING()	140
2.03	FLOOR()	141
2.04	ROUND()	141
2.05	TRUNCATE()	141
2.06	SQRT()	142
2.07	POW() / POWER()	142
2.08	RAND()	142
2.09	MOD() Function	143
3.	Date and Time Functions	144
3.01	NOW()	144
3.02	CURDATE()	144
3.03	CURTIME()	144
3.04	SYSDATE()	144
3.05	DATE()	145
3.06	TIME()	145
3.07	YEAR() MONTH() DAY()	146
3.08	HOUR()	146
3.09	MINUTE()	147
3.10	SECOND()	147
3.11	DATEDIFF()	147
3.12	DATE_ADD()	148

3.13	DATE_SUB()	148
3.14	DATE_FORMAT()	148
3.15	STR_TO_DATE()	149
3.16	DAYNAME()	149
3.17	MONTHNAME()	150
4.	NULL Functions	151
4.01	IFNULL()	151
4.02	NULLIF()	151
4.03	COALESCE()	152
4.04	ISNULL()	152
5.	Control Flow Functions	153
5.01	IF()	153
5.02	CASE	153
6.	Aggregate Functions	154
6.01	COUNT()	154
6.02	SUM()	154
6.03	AVG()	154
6.04	MIN()	155
6.05	MAX()	155
6.06	GROUP_CONCAT()	155
7.	Type Conversion Functions	157
7.01	CAST()	157
7.02	CONVERT()	157
7.03	BINARY	157
9.	Window Functions (MySQL 8.0+)	159
9.01	ROW_NUMBER()	159
9.02	RANK()	159
9.03	DENSE_RANK()	160
9.04	LEAD()	160
9.05	LAG()	161
9.06	FIRST_VALUE()	162
9.07	LAST_VALUE()	162
9.08	NTILE()	162

9.09 PERCENT_RANK()	163
9.10 CUME_DIST()	164
10. Information Functions	165
10.01 DATABASE()	165
10.02 USER()	165
10.03 VERSION()	165
10.04 LAST_INSERT_ID()	165
10.05 ROW_COUNT()	166
MySQL Aliases	167
A1. Column Alias	167
A2. Table Alias	168
A3. Expression Alias	169
A4. Aggregate Alias	170
A5. Subquery Alias	171
A6. CTE Alias	172
PRACTICE QUESTION AND ANSWER	

Data and Information

DATA

Data is refers to raw facts and figures that are collected but not yet processed or given meaning

It can be numbers, texts, images, audio or video.

Data alone does not provide understanding. It just a raw material.

Eg:-

45, 67, 89, 90

Ravi, Meena, Kumar

30-degree Celsius, 34-degree Celsius, 50-degree Celsius

INFORMATION

Information is processed, organized or structured data that carries meaning and is useful to decision-making.

It is data that has been interpreted and presented in a way that makes sense.

Eg:-

The average exam score is 73.

Meena scored the highest in the class.

The temperature in Chilaw today is 30-degree Celsius.

Data (raw facts)	Information(processed meaning)
unorganized , unprocessed	organized, structured, meaningful
just collected facts	helps in decision making
marks :- 45, 67, 89, 90	average:- 73
value is low until processed	high value because it provides understanding

- Data is the input.
- Processing turns data into information.
- Information is the output that supports decision making.
- Flow:- data → processing →information →knowledge →decision

Database and DBMS

DATABASE

Database is a collection of information organized in such a way that it can be accessed easily.

Why databases?

Before databases organizations stored their data using traditional file system. So they had to face so many problems.

Data redundancy	same data stored in multiple files
Data inconsistency	different version of same data Eg:- Nuwan Dissanayaka, Nuwan, Dissanayaka
Data isolation	data scattered in different departments
No proper security	no access control for users
No backup and recovery	high risk of data loss
Difficulties in data sharing	hard to share the data between departments

DBMS – DATABASE MANAGEMENT SYSTEM

DBMS is a software that manages the databases and provide an interface to store, retrieve and manipulate data efficiently.

- Store data efficiently
- Reduce data redundancy
- Ensure data security
- Provide backup and recovery
- Supports multi-user access

Types of DBMSs,

- Relational (RDBMS)
- Non-Relational (NoSQL)

RELATIONAL DATABASE MANAGEMENT SYSTEM

- Uses structured schema
- Uses SQL (structured query language)
- Tables are related by primary and foreign keys
- Eg:-
 - MySQL
 - Oracle DB
 - Microsoft SQL Server
- Key features,
 - structural schema
 - strong data consistency
 - relationship between tables

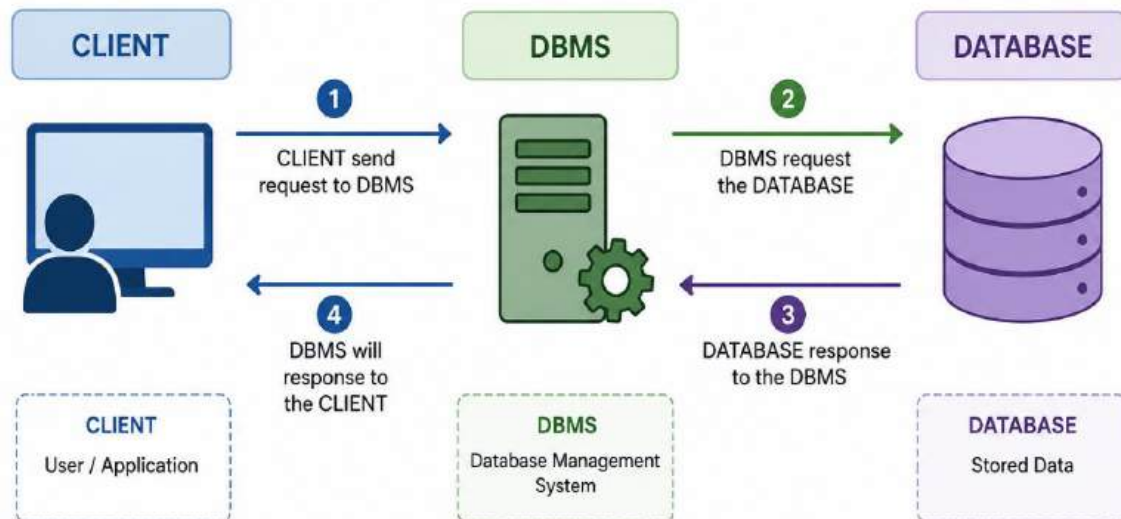
NON-RELATIONAL DATABASE MANAGEMENT SYSTEM

- Document based
- Key-Value
- Graphs
- Column based
- Eg:-
 - MongoDB
 - Cassandra
- Key features,
 - flexible schema
 - suitable for large scale data
 - high performance for big data applications

Features	RDBMS	NoSQL
structure	tables	document, key-value
schema	fixed	flexible
query language	SQL	no fixed language
best for	banking, ERP	social media, big data

CLIENT-DBMS-DB CONCEPT

CLIENT send request to DBMS and DBMS request the DATABASE
and the DATABASE response to the DBMS and DBMS will response to the CLIENT



MAIN FEATURES OF MYSQL

- **Data storage and management**
 - Stores data in tables
 - Support relationship
- **Security**
 - Username and password authentication
 - User roles and privileges
- **Multi-user access**
 - Many users can access simultaneously
- **Backup and recovery**
 - Supports data backup and recovery
- **SQL support**
 - User SQL(structured query language)
 - Supports SELECT, INSERT, UPDATE, DELETE
- **Data integrity**
 - Constraints (NOT NULL, UNIQUE, PRIMARY KEY , ...)

DATA MODALS

A data modal is a conceptual representation of a data is structured, stored and related inside the database.

Also it is a blueprint that describes the structure of data , relationship between data and rules for storing data.

MAIN COMPONENTS OF A DATA MODAL

1.Entities

an entity represents a "thing" or "object" in the real world that is distinguishable from all other objects.

Eg:- student, course, teacher

- **Strong entity**

This is an entity that can exist independently of other entities, it has its own identifier ; primary key

Eg:- student in a school database

- **Weak entity**

This entity cannot be uniquely identified by its attributes alone and relies on a relationship with a strong entity.

Eg:- a dependent (child) of an employee. If the employee leaves the company the dependent record usually doesn't need to exist.

2.Attributes

Attributes are the properties or characteristics that describe an entity.

- **Key attribute**

Attributes that uniquely identify each record in a table

Eg:- student_id, teacher_id

- **Composite attribute**

An attribute that can be divided into smaller sub-parts.

Eg:- name can be divided as first_name, last_name, middle_name

- **Multivalued attribute**

An attribute that can have more than one value.

Eg:- to phone number there will be home, work and personal.

- **Derived attribute**

An attribute whose value is calculated from other attributes. It isn't store permanently, but it figured out.

Eg:- age is derived from date of birth

3.Relationships

This defines how entities interact with one another.

Eg:- one student register for multiple courses , and one course can be registered by many students(many to many relationship)

- **One to One**

One instance of entity A is associate with exactly one instance of entity B.

Eg:- one person can have only one passport, and one passport can belongs to one person.

- **One to many**

One instance of entity A can associated with many instances of entity B.

Eg:- one customer can have many orders, but one order is belongs to one customer

- **Many to one**

Many instances of entity A is can be associated with many instances of entity B.

Eg:- one student can registered to many courses, and one course can be registered by many students.

EVOLUTION OF DATA MODALS

- **Early age of data modals**
 - Hierarchical and network modals
- **Modern age of data modals (after 1970)**
 - Relational , ER modals
- **New age of data modals**
 - Object-Oriented, object/relational, XML, Big data and NoSQL.

BUSINESS RULES

In DBMS , business rules are statements that define how data should be stored, used and controlled according to the organizations' policies or logic.

Business rules are the constraints or conditions that describe how a business operates and how the database must handle data.

They ensure that the data in the database is accurate , valid and consistent.

Eg:- *imagine a university database,*

- A student must have an id
- A student can enroll many courses
- Each course must have a lecturer
- A student can't register for more than 5 courses per semester

How business rules are implemented in DBMS,

- Primary keys
- Foreign keys
- Other constraints(NOT NULL, UNIQUE, CHECK)
- Triggers
- Application logic

ENTITY RELATIONSHIP (ER) MODEL

The ER modal is a specific data modal used mainly for conceptual database design.

This is used to identify entities, their attributes and relationships between them in a database.

ER model is important for,

- Understand the structure of data
- Plan a database before implementation
- Reduce data redundancy
- Create clear database relationship

An ER modal has 2 parts,

1. Cardinality constraint

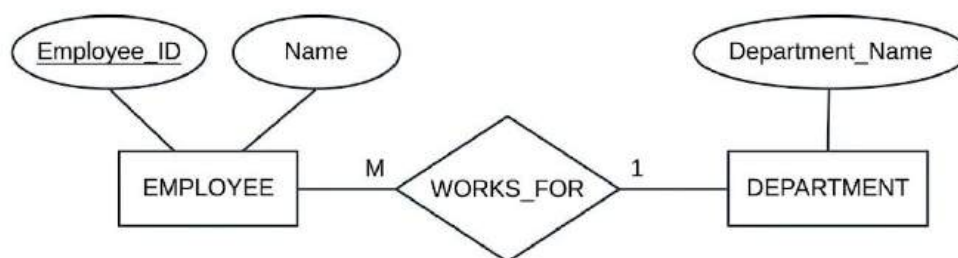
- One to One
- One to Many
- Many to Many

2. Participation constraint

- Total participation (mandatory)
- Partial participation (optional)

There are many ER modal types available. Such as,

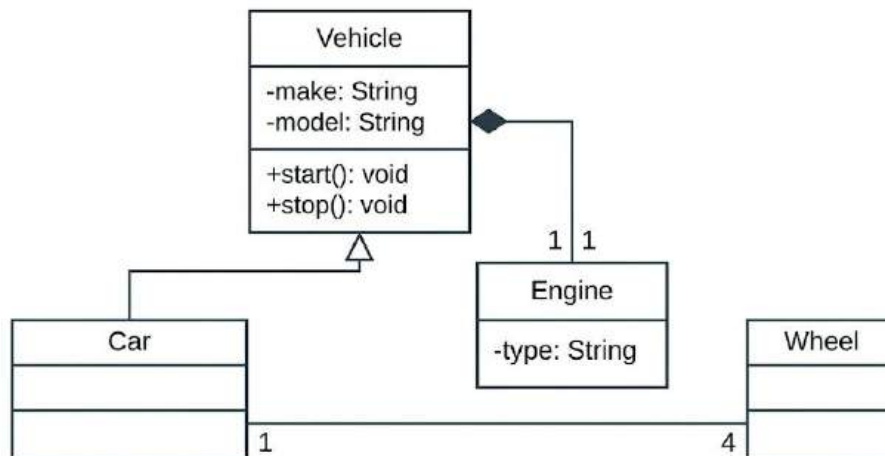
○ Chen notation



○ Crow foot notation



- UML class diagram notation



draw.io ERD Reference: Crow's Foot Notation & Symbols

Entity-Relationship Diagrams (ERDs) utilize specific shapes to represent data structures and relationship constraints. This guide focuses on the standard **draw.io stencils** for **entities**, **attributes**, and **Crow's Foot notation** used to define how records relate across tables.

CORE ERD COMPONENTS

Entities: Standard and Weak

Standard Entity	Entities use rectangles (standard) to represent data objects.
Weak Entity	or double-lined rectangles (weak).

Relationships: Diamonds and Associative Links

	Standard relationships use diamonds.
	...associative relationships use a diamond inside a rectangle.

Attributes: Oval-Based Properties

Key Attribute	Multivalued Attribute	Derived Attribute	Composite Attribute
Properties are ovals: underlined for keys	...double-lined for multivalued	...dashed for derived	...or branching for composite.

CROW'S FOOT CARDINALITY

Understanding the Symbol Pairs: Each line end uses two symbols: the inner (closest to entity) and the outer (maximum cardinality).

	Meaning: One and Only One Notation detail: Two vertical bars; mandatory single relationship.
	Meaning: Zero or One Notation detail: Circle (inner) and vertical bar (outer); optional single.
	Meaning: One or Many Notation detail: Vertical bar (inner) and crow's foot (outer); mandatory many.
	Meaning: Zero or Many Notation detail: Circle (inner) and crow's foot (outer); optional many.

DEGREES OF RELATIONSHIPS

There are 4 degrees.

1. Unary relationship

An entity associate with itself

2. Binary relationship

The association between 2 entities. This is the most common degree.

3. Ternary relationship

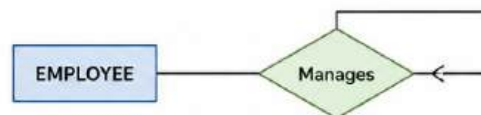
A relationship involves 3 entities simultaneously

4. N-ary relationship

When there are n entities participating in a relationship, it is called n-ary relationship.

1. UNARY

(One entity set participates in the relationship)



An EMPLOYEE manages zero or more EMPLOYEEs.

2. BINARY

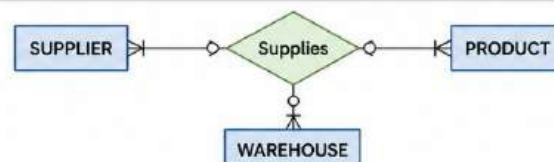
(Two entity sets participate in the relationship)



A CUSTOMER places zero or more ORDERs. An ORDER is placed by one CUSTOMER.

3. TERNARY

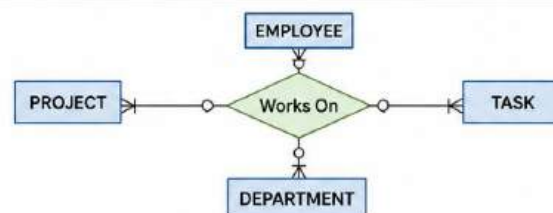
(Three entity sets participate in the relationship)



SUPPLIER supplies PRODUCT to WAREHOUSE.

4. N-ARY

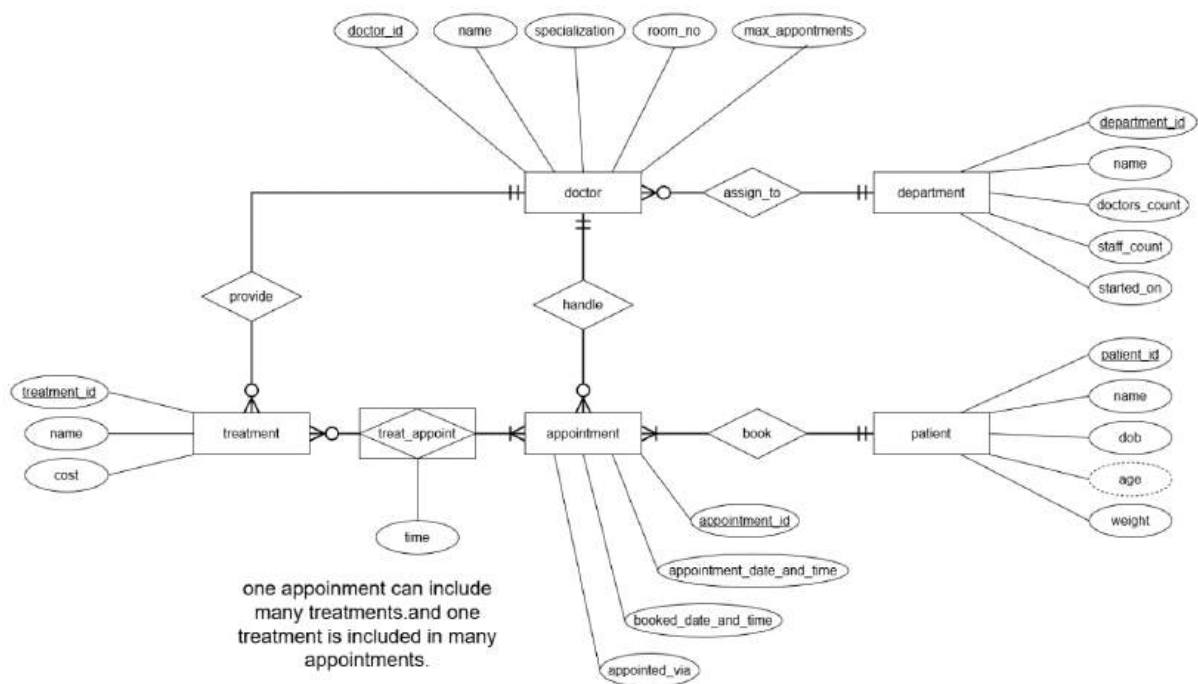
(Four or more entity sets participate in the relationship)



EMPLOYEE works on TASK of a PROJECT in a DEPARTMENT.

Example question,

A well-established private hospital network in Sri Lanka is planning to implement a database system to efficiently manage its operations. The hospital handles a large number of patients daily and needs to maintain accurate records of patient details, doctors, departments, appointments, and treatments. Each doctor is assigned to a specific department such as Cardiology, Neurology, or Orthopedics. Patients can book multiple appointments with different doctors depending on their medical needs. Every appointment is handled by one doctor and belongs to one patient, and during an appointment, the doctor may provide one or more treatments. Each treatment has a defined cost, and the hospital wants to keep track of all treatments given to patients through appointments. The system should allow the hospital management to store, update, and retrieve data efficiently for decision-making purposes.



MySQL

What is SQL?

SQL refers to Structured Query Language. It is a language which is used in relational DBMSs to communicate with the database. It helps us to perform CRUD operations from the database.

C - Create

R - READ

U - Update

D - Delete

MySQL is also a DBMS, which is familiar with the language SQL.

Types of MySQL commands,

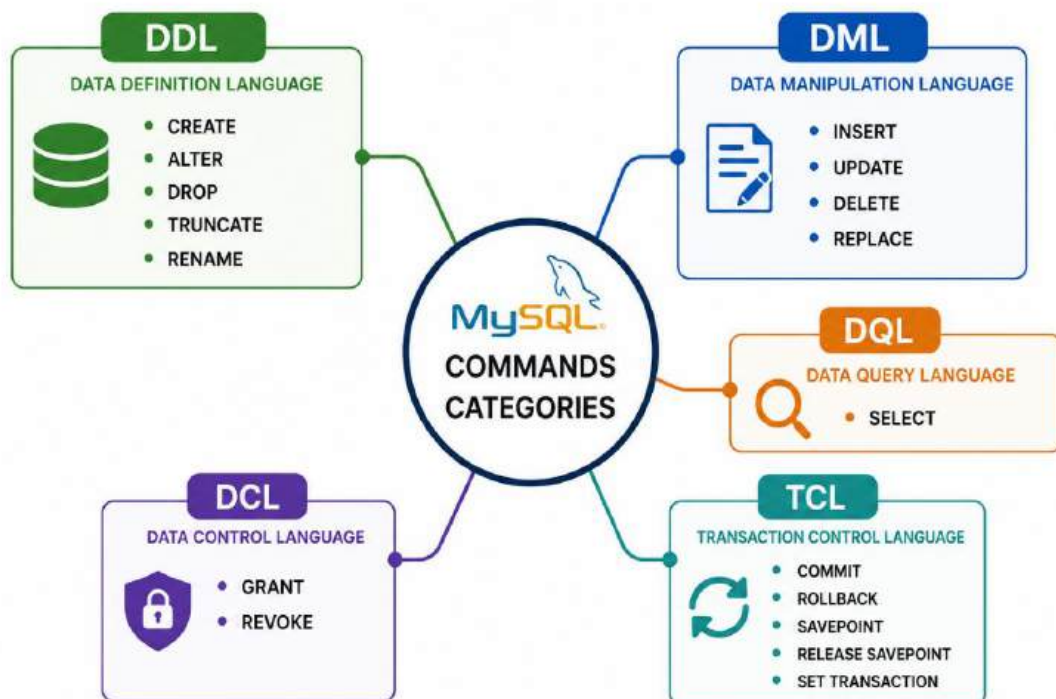
DDL - Data Definition Language

DML - Data Manipulation Language

DQL - Data Query Language

DCL - Data Control Language

TCL - Transaction Control Language



DATA TYPES

DATATYPES

1.01 TINYINT

Category: Numeric

What it does: Stores a very small whole number. Takes only 1 byte of storage.

Syntax:

`column_name TINYINT;`

Example:

`is_active TINYINT;`

Key Points:

- Range: -128 to 127
- BOOLEAN is just an alias for TINYINT(1) – 0 = false, 1 = true
- Use for flags and small status codes

1.02 SMALLINT

Category: Numeric

What it does: Stores a small whole number using 2 bytes. Use when TINYINT is too small but INT is wasteful.

Syntax:

`column_name SMALLINT;`

Example:

`age SMALLINT;`

Key Points:

- Range: -32,768 to 32,767
- Useful for port numbers, year values, or small counters

1.03 MEDIUMINT

Category: Numeric

What it does: Stores a medium-sized whole number using 3 bytes. Sits between SMALLINT and INT.

Syntax:

```
column_name MEDIUMINT;
```

Example:

```
page_views MEDIUMINT;
```

Key Points:

- Range: -8,388,608 to 8,388,607
- Rarely used – INT is usually preferred for simplicity

1.04 INT

Category: Numeric

What it does: Stores a standard whole number using 4 bytes. The default and most common integer type.

Syntax:

```
column_name INT;
```

Example:

```
student_id INT AUTO_INCREMENT PRIMARY KEY;
```

Key Points:

- Range: -2,147,483,648 to 2,147,483,647
- Best choice for primary keys and foreign keys
- INT(11) – the number 11 is display width only, NOT a value limit (deprecated in MySQL 8.0)

1.05 BIGINT

Category: Numeric

What it does: Stores a very large whole number using 8 bytes. Use when INT range is not enough.

Syntax:

`column_name BIGINT;`

Example:

`transaction_id BIGINT;`

Key Points:

- Range: $\pm 9,223,372,036,854,775,807$
- Uses twice the storage of INT – only use when you truly need the range
- Common for distributed IDs and millisecond timestamps

1.06 FLOAT

Category: Numeric

What it does: Stores an approximate decimal number using 4 bytes. About 7 significant digits of precision.

Syntax:

`column_name FLOAT;`

Example:

`temperature FLOAT;`

Key Points:

- Approximate – small rounding errors can occur
- Never use FLOAT for money or financial values – use DECIMAL instead
- Good for sensor readings, scientific data, measurements

1.07 DOUBLE

Category: Numeric

What it does: Stores a higher-precision approximate decimal using 8 bytes. About 15 significant digits.

Syntax:

`column_name DOUBLE;`

Example:

`latitude DOUBLE;`

Key Points:

- More precise than FLOAT but still approximate
- Used for GPS coordinates, statistics, scientific calculations
- Still not suitable for exact financial values – use DECIMAL

1.08 DECIMAL

Category: Numeric

What it does: Stores an exact fixed-point number with no rounding errors. Always use this for money.

Syntax:

`column_name DECIMAL(p, s);`

Example:

`price DECIMAL(8, 2);`
`//stores up to 999999.99//`

Key Points:

- p = total number of digits allowed – e.g. p=8 allows up to 8 digits in total
- s = digits after the decimal point – e.g. s=2 means two decimal places
- DECIMAL(8,2) can store values from -999999.99 to 999999.99
- DECIMAL(10,2) is the standard for prices – holds up to 99,999,999.99

1.09 BOOLEAN

Category: Numeric

What it does: Stores a true or false value. In MySQL it is just an alias for TINYINT(1).

Syntax:

`column_name BOOLEAN;`

Example:

`is_active BOOLEAN;`

`INSERT INTO users (is_active) VALUES (TRUE); -- stored as 1`

Key Points:

- TRUE is stored as 1, FALSE is stored as 0
- Not a real boolean – it is TINYINT(1) internally
- Any non-zero value is treated as TRUE in conditions

2.01 CHAR

Category: String

What it does: Stores a fixed-length string. Always uses exactly n characters, padding shorter values with spaces.

Syntax:

`column_name CHAR(n);`

Example:

`country_code CHAR(2);`

`//stores 'LK', 'US', 'IN'//`

Key Points:

- n = exact number of characters to store (0 to 255)
- Shorter values are padded with spaces to fill n characters
- Best for fixed-length data: country codes, currency codes, hash values

2.02 VARCHAR

Category: String

What it does: Stores a variable-length string. Uses only the space needed, up to a maximum of n characters.

Syntax:

```
column_name VARCHAR(n);
```

Example:

```
email VARCHAR(100);
```

Key Points:

- n = maximum number of characters allowed (up to 65,535)
- 'hi' in VARCHAR(100) uses 2 chars of storage, not 100
- Most commonly used string type – names, emails, titles, URLs
- Can be fully indexed unlike TEXT columns

2.03 TEXT

Category: String

What it does: Stores large blocks of text. Stored separately from the main row for efficiency.

Syntax:

```
column_name TEXT;
```

```
column_name MEDIUMTEXT;
```

```
column_name LONGTEXT;
```

Example:

```
description TEXT;
```

Key Points:

- TEXT = up to 64 KB | MEDIUMTEXT = up to 16 MB | LONGTEXT = up to 4 GB
- Cannot have a DEFAULT value
- Cannot be fully indexed – only prefix indexes are supported

2.04 ENUM

Category: String

What it does: Stores exactly one value chosen from a predefined list. Internally stored as a small integer.

Syntax:

```
column_name ENUM('val1', 'val2', 'val3');
```

Example:

```
status ENUM('active', 'inactive', 'pending');
```

Key Points:

- Only one value from the list can be stored per row
- Inserting a value not in the list causes an error
- Changing the list requires ALTER TABLE – can be slow on large tables

2.05 SET

Category: String

What it does: Like ENUM but can store zero or more values from the predefined list at the same time.

Syntax:

```
column_name SET('val1', 'val2', 'val3');
```

Example:

```
permissions SET('read', 'write', 'delete');  
//can store: 'read' or 'read,write' or 'read,write,delete'//
```

Key Points:

- Multiple values stored as a comma-separated string in one column
- Maximum 64 values allowed in the list
- Stored internally as a bitmask integer – very compact

2.06 BLOB

Category: String

What it does: Stores raw binary data such as images, files, or encrypted content. No character set applies.

Syntax:

`column_name BLOB;`

`column_name MEDIUMBLOB;`

`column_name LONGBLOB;`

Example:

`profile_pic BLOB;`

Key Points:

- BLOB = up to 64 KB | MEDIUMBLOB = up to 16 MB | LONGBLOB = up to 4 GB
- Stores binary data exactly as-is – no encoding or charset conversion
- Best practice: store large files on disk and keep only the file path in the DB

3.01 DATE

Category: Date and Time

What it does: Stores a calendar date with no time component. Uses 3 bytes.

Syntax:

`column_name DATE;`

Example:

`dob DATE;`

`//stores '2000-04-15'//`

Key Points:

- Format: YYYY-MM-DD
- Range: 1000-01-01 to 9999-12-31
- Use CURDATE() to insert today's date

3.02 TIME

Category: Date and Time

What it does: Stores a time of day or a duration. No date component.

Syntax:

```
column_name TIME;
```

Example:

```
open_time TIME;  
//stores '09:00:00'//
```

Key Points:

- Format: HH:MM:SS
- Range: -838:59:59 to 838:59:59 – supports durations, not just clock times
- Use CURTIME() to insert the current time

3.03 DATETIME

Category: Date and Time

What it does: Stores both date and time. Does not convert timezones – stores exactly what you insert.

Syntax:

```
column_name DATETIME;
```

Example:

```
created_at DATETIME DEFAULT NOW();
```

Key Points:

- Format: YYYY-MM-DD HH:MM:SS
- Range: 1000-01-01 00:00:00 to 9999-12-31 23:59:59
- No timezone conversion – stores exactly what you give it

3.04 TIMESTAMP

Category: Date and Time

What it does: Stores date and time in UTC. Auto-converts to the server timezone when reading. Can auto-update on row changes.

Syntax:

`column_name` **TIMESTAMP;**

Example:

`updated_at` **TIMESTAMP DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP;**

Key Points:

- Range: 1970-01-01 to 2038-01-19 only (Year 2038 problem)
- Automatically converts to/from UTC based on server timezone
- Use DATETIME instead if you need dates beyond year 2038

3.05 YEAR

Category: Date and Time

What it does: Stores a 4-digit year only. The most compact date type – just 1 byte.

Syntax:

`column_name` **YEAR;**

Example:

`grad_year` **YEAR;**
`//stores 2024//`

Key Points:

- Format: YYYY only – no month or day
- Range: 1901 to 2155
- YEAR(2) was removed in MySQL 8.0 – always use 4-digit YEAR

4.01 JSON

Category: Other

What it does: Stores a valid JSON document. MySQL validates the JSON on insert and stores it in binary format for fast access.

Syntax:

```
column_name JSON;
```

Example:

```
settings JSON;
```

```
//stores: {"theme": "dark", "lang": "en"} //
```

```
//read a field: SELECT settings->>'$.theme' FROM users; //
```

Key Points:

- Invalid JSON is rejected on insert – data is always valid
- Use ->> operator to access individual fields inside the JSON
- Cannot be directly indexed – create a generated column to index a specific field

4.02 BLOB

Category: Other

What it does: Stores raw binary data such as images, files, or encrypted content. No character set applies.

Syntax:

```
column_name BLOB;
```

```
column_name MEDIUMBLOB;
```

```
column_name LONGBLOB;
```

Example:

```
profile_pic BLOB;
```

Key Points:

- BLOB = up to 64 KB | MEDIUMBLOB = up to 16 MB | LONGBLOB = up to 4 GB
- Stores binary data exactly as-is – no encoding or charset conversion
- Best practice: store large files on disk or cloud and keep only the file path in the DB

ID	Statement / Function	Description
C01	NOT NULL	Ensures a column cannot store NULL; every row must supply a value for this column.
C02	UNIQUE	Guarantees all values in a column or column set are distinct across all rows.
C03	PRIMARY KEY	Uniquely identifies each row; combines NOT NULL and UNIQUE on one or more columns.
C04	FOREIGN KEY	Links a column to another table's primary key, enforcing referential integrity between tables.
C05	AUTO_INCREMENT	Automatically assigns the next integer value to a column each time a new row is inserted.
C06	DEFAULT	Sets a fallback value for a column when no value is explicitly provided during insertion.
C07	CHECK	Validates that column values satisfy a specified boolean expression before saving.
001	+ (Addition)	Adds two numeric values together and returns the sum.
002	- (Subtraction)	Subtracts the right operand from the left and returns the difference.
003	* (Multiplication)	Multiplies two numeric values and returns the product.
004	/ (Division)	Divides the left operand by the right and returns the quotient as a decimal.
005	DIV (Integer Division)	Performs integer division, discarding any remainder and returning a whole number.
006	% (Modulus)	Returns the remainder after dividing the left operand by the right operand.
007	MOD (Modulus)	Returns the remainder same as %; used as a keyword alternative to the % operator.
008	= (Equal To)	Compares two values and returns TRUE if they are exactly equal.

009	<code>!= / <></code> (Not Equal To)	Returns TRUE when two values are not equal to each other.
010	<code>></code> (Greater Than)	Returns TRUE when the left value is strictly larger than the right value.
011	<code><</code> (Less Than)	Returns TRUE when the left value is strictly smaller than the right value.
012	<code>>=</code> (Greater Than or Equal To)	Returns TRUE when the left value is larger than or equal to the right.
013	<code><=</code> (Less Than or Equal To)	Returns TRUE when the left value is smaller than or equal to the right.
014	<code>IS</code>	Tests whether a value equals a boolean or is NULL; safer than <code>=</code> for NULL checks.
015	<code>IS NOT</code>	Returns TRUE when a value is not NULL or not the specified boolean state.
016	<code>BETWEEN</code>	Returns TRUE when a value falls within an inclusive lower and upper boundary range.
017	<code>NOT BETWEEN</code>	Returns TRUE when a value falls outside the specified lower and upper range bounds.
018	<code>IN</code>	Returns TRUE when a value matches any element in a provided list of values.
019	<code>NOT IN</code>	Returns TRUE when a value does not match any element in the provided list.
020	<code>LIKE</code>	Performs wildcard pattern matching using <code>%</code> for any characters and <code>_</code> for one character.
021	<code>NOT LIKE</code>	Returns TRUE when a value does not match the specified wildcard pattern string.
022	<code>AND / &&</code>	Returns TRUE only when all combined conditions are simultaneously true.
023	<code>OR / </code>	Returns TRUE when at least one of the combined conditions evaluates to true.
024	<code>NOT / !</code>	Negates a boolean expression, flipping TRUE to FALSE and vice versa.
025	<code>ALL</code>	Returns TRUE if a subquery comparison holds true for every value returned.

026	ANY / SOME	Returns TRUE if a subquery comparison holds true for at least one value.
027	EXISTS	Returns TRUE if a subquery produces one or more result rows.
028	= (Assignment)	Assigns a value to a variable in SET statements or column assignments.
029	:= (Assignment)	Assigns a value to a user-defined variable anywhere inside a SELECT statement.
030	UNION (Operator)	Combines result sets of two queries into one, removing duplicate rows automatically.
031	UNION ALL (Operator)	Combines result sets of two queries into one, keeping all duplicate rows intact.
032	INTERSECT (Operator)	Returns only rows that appear in both query result sets (MySQL 8.0.31+).
033	EXCEPT (Operator)	Returns rows from the first query that are absent in the second query (MySQL 8.0.31+).
CL01	SELECT	Specifies which columns or expressions to retrieve from one or more tables.
CL02	DISTINCT	Removes duplicate rows from query results, returning only unique value combinations.
CL03	FROM	Specifies the table or subquery from which to retrieve data for the query.
CL04	WHERE	Filters rows before grouping, returning only rows that satisfy the given condition.
CL05	GROUP BY	Groups rows sharing common values into summary rows for aggregate function use.
CL06	WITH ROLLUP	Adds subtotal and grand-total rows to a GROUP BY result for hierarchical summaries.
CL07	HAVING	Filters grouped rows after GROUP BY, typically used with aggregate function conditions.
CL08	ORDER BY	Sorts the final result set by one or more columns, ascending or descending.

CL09	LIMIT	Restricts the number of rows returned to a specified maximum count.
CL10	OFFSET	Skips a specified number of rows before starting to return query results.
CL11	INTO	Redirects SELECT output into variables, a file, or another table.
CL12	INNER JOIN	Returns only rows where a matching record exists in both joined tables.
CL13	LEFT JOIN	Returns all rows from the left table plus matching rows from the right table.
CL14	RIGHT JOIN	Returns all rows from the right table plus matching rows from the left table.
CL15	CROSS JOIN	Produces a Cartesian product pairing every row of one table with every other.
CL16	SELF JOIN	Joins a table to itself to compare rows within the same table.
CL17	ON	Specifies the join condition defining how two tables are related to each other.
CL18	USING	Shorthand join condition when both tables share a column of the same name.
CL19	UNION (Clause)	Combines two SELECT result sets removing duplicates as a set operation clause.
CL20	UNION ALL (Clause)	Combines two SELECT result sets keeping all duplicates as a set operation clause.
CL21	INTERSECT (Clause)	Returns only rows common to both SELECT results as a set operation clause.
CL22	EXCEPT (Clause)	Returns rows in the first SELECT absent from the second as a set operation clause.
CL23	WITH (CTE)	Defines a named temporary result set reusable within the same query statement.
CL24	WITH RECURSIVE	Creates a recursive CTE that repeatedly references itself to traverse hierarchical data.
CL25	OVER()	Defines the window frame for a window function to operate on a row set.

CL26	PARTITION BY	Divides rows into groups within a window function without collapsing them like GROUP BY.
CL27	ROWS BETWEEN	Specifies the physical row range relative to the current row in a window frame.
CL28	RANGE BETWEEN	Specifies a logical value range around the current row in a window function frame.
CL29	UNBOUNDED PRECEDING	Sets the window frame start at the very first row of the partition.
CL30	CURRENT ROW	Sets a window frame boundary at the position of the currently processed row.
CL31	UNBOUNDED FOLLOWING	Sets the window frame end at the very last row of the partition.
CL32	INSERT INTO	Adds one or more new rows of data into a specified database table.
CL33	VALUES	Supplies the literal data values to be inserted in an INSERT statement.
CL34	SET (DML)	Assigns new column values in an UPDATE statement or variable assignment.
CL35	UPDATE	Modifies existing rows in a table based on an optional WHERE condition.
CL36	DELETE	Removes rows from a table that satisfy the specified WHERE condition.
CL37	REPLACE INTO	Inserts a row or deletes then re-inserts it if a duplicate key exists.
CL38	ON DUPLICATE KEY UPDATE	Updates specified columns when an INSERT causes a duplicate unique or primary key.
CL39	IGNORE	Silently skips errors like duplicate keys instead of aborting the statement.
CL40	CREATE TABLE	Defines and creates a new table with specified columns, types, and constraints.
CL41	ALTER TABLE	Modifies an existing table's structure by adding, dropping, or changing columns.
CL42	DROP TABLE	Permanently deletes a table and all its data from the database.

CL43	TRUNCATE TABLE	Removes all rows from a table quickly without logging individual row deletions.
CL44	RENAME TABLE	Changes the name of an existing table to a new specified name.
CL45	CREATE INDEX	Creates an index on table columns to speed up data retrieval queries.
CL46	DROP INDEX	Removes an existing index from a table to free storage or restructure indexing.
CL47	CREATE VIEW	Creates a saved virtual table based on a stored SELECT query definition.
CL48	DROP VIEW	Permanently removes an existing view definition from the database schema.
CL49	CREATE DATABASE	Creates a new empty database container for storing tables and other objects.
CL50	DROP DATABASE	Permanently deletes an entire database along with all tables and data within.
CL51	CASE WHEN	Evaluates conditions in order and returns a result for the first true condition.
CL52	FROM (Subquery)	Uses a nested SELECT as a derived table in the FROM clause.
CL53	WHERE (Subquery)	Uses a nested SELECT inside WHERE to filter rows based on computed results.
F01	CONCAT()	Joins two or more string values together into one combined string.
F02	LENGTH()	Returns the number of bytes in a string, differing from character count for multibyte strings.
F03	CHAR_LENGTH()	Returns the number of characters in a string, regardless of byte size per character.
F04	UPPER()	Converts all characters in a string to uppercase letters.
F05	LOWER()	Converts all characters in a string to lowercase letters.
F06	TRIM()	Removes leading and trailing whitespace or specified characters from a string.

F07	LTRIM()	Removes whitespace from the left (beginning) side of a string only.
F08	RTRIM()	Removes whitespace from the right (end) side of a string only.
F09	SUBSTRING()	Extracts a portion of a string starting at a given position for a given length.
F10	LEFT()	Returns a specified number of characters from the beginning of a string.
F11	RIGHT()	Returns a specified number of characters from the end of a string.
F12	REPLACE()	Replaces all occurrences of a substring within a string with a new substring.
F13	REVERSE()	Returns the characters of a string in reverse order from end to beginning.
F14	REPEAT()	Returns a string repeated a specified number of times concatenated together.
F15	INSTR()	Returns the position of the first occurrence of a substring within a string.
F16	LPAD()	Left-pads a string with a specified character to reach a target total length.
F17	RPAD()	Right-pads a string with a specified character to reach a target total length.
F18	SPACE()	Returns a string consisting of a specified number of space characters.
F19	FORMAT()	Formats a number with comma-separated thousands and a specified number of decimal places.
F20	ABS()	Returns the absolute (non-negative) value of a numeric expression.
F21	CEIL() / CEILING()	Rounds a number up to the nearest integer greater than or equal to it.
F22	FLOOR()	Rounds a number down to the nearest integer less than or equal to it.
F23	ROUND()	Rounds a number to a specified number of decimal places using standard rounding rules.

F24	TRUNCATE()	Cuts a number to a specified number of decimal places without rounding.
F25	SQRT()	Returns the square root of a non-negative numeric value.
F26	POW() / POWER()	Raises a base number to the power of a specified exponent.
F27	RAND()	Returns a random floating-point value between 0 (inclusive) and 1 (exclusive).
F28	MOD() Function	Returns the remainder of dividing one number by another, same as the % operator.
F29	NOW()	Returns the current date and time at the moment the statement began executing.
F30	CURDATE()	Returns the current date only, without any time component, as YYYY-MM-DD.
F31	CURTIME()	Returns the current time only, without any date component, as HH:MM:SS.
F32	SYSDATE()	Returns the exact date and time the function itself is called, not statement start.
F33	DATE()	Extracts the date portion from a datetime or timestamp expression.
F34	TIME()	Extracts the time portion from a datetime or timestamp expression.
F35	YEAR() / MONTH() / DAY()	Extracts the year, month, or day integer part from a date or datetime value.
F36	HOUR()	Extracts the hour component from a time or datetime value as an integer.
F37	MINUTE()	Extracts the minute component from a time or datetime value as an integer.
F38	SECOND()	Extracts the second component from a time or datetime value as an integer.
F39	DATEDIFF()	Returns the number of days between two date values (first minus second).
F40	DATE_ADD()	Adds a specified time interval to a date and returns the resulting date.

F41	DATE_SUB()	Subtracts a specified time interval from a date and returns the resulting date.
F42	DATE_FORMAT()	Formats a date or datetime value into a custom string using format specifiers.
F43	STR_TO_DATE()	Converts a formatted string into a MySQL date value using a specified format.
F44	DAYNAME()	Returns the full name of the weekday for a given date (e.g., Monday).
F45	MONTHNAME()	Returns the full name of the month for a given date (e.g., January).
F46	IFNULL()	Returns the first argument if it is not NULL, otherwise returns the second argument.
F47	NULLIF()	Returns NULL if both arguments are equal; otherwise returns the first argument.
F48	COALESCE()	Returns the first non-NULL value from a list of provided expressions.
F49	ISNULL()	Returns 1 if the argument is NULL, otherwise returns 0.
F50	IF()	Returns one value if a condition is true, another value if it is false.
F51	CASE (Function)	Evaluates conditions sequentially and returns the corresponding result for the first match.
F52	COUNT()	Counts the number of non-NULL values or total rows in a result set.
F53	SUM()	Calculates the total sum of numeric values in a column or expression.
F54	AVG()	Calculates the average of non-NULL numeric values in a column or expression.
F55	MIN()	Returns the smallest value in a column or expression across all rows.
F56	MAX()	Returns the largest value in a column or expression across all rows.
F57	GROUP_CONCAT()	Concatenates non-NULL values from a group into a single comma-separated string.

F58	CAST()	Converts a value from one data type to another explicitly specified type.
F59	CONVERT()	Converts a value to a different data type or character set encoding.
F60	BINARY	Converts a string to binary for case-sensitive byte-level comparison.
F61	ROW_NUMBER()	Assigns a unique sequential integer to each row within a window partition.
F62	RANK()	Assigns a rank to each row; tied rows get the same rank, leaving gaps after.
F63	DENSE_RANK()	Assigns a rank with no gaps; tied rows share rank but next rank increments by one.
F64	LEAD()	Returns a value from a row a specified number of rows ahead in the window.
F65	LAG()	Returns a value from a row a specified number of rows behind in the window.
F66	FIRST_VALUE()	Returns the value of an expression from the first row of the window frame.
F67	LAST_VALUE()	Returns the value of an expression from the last row of the window frame.
F68	NTILE()	Divides rows into a specified number of ranked buckets as evenly as possible.
F69	PERCENT_RANK()	Returns a row's relative rank as a fraction between 0 and 1 within the partition.
F70	CUME_DIST()	Returns the cumulative distribution: fraction of rows with values less or equal to current.
F71	DATABASE()	Returns the name of the currently selected database in the connection.
F72	USER()	Returns the current MySQL username and hostname for the active connection.
F73	VERSION()	Returns the version string of the MySQL server currently running.
F74	LAST_INSERT_ID()	Returns the auto-generated ID produced by the most recent INSERT statement.

F75	ROW_COUNT()	Returns the number of rows affected by the last data-modifying statement executed.
A01	Column Alias (AS)	Renames a column in query output using AS or an inline label for readability.
A02	Table Alias	Assigns a short temporary name to a table for easier reference within a query.
A03	Expression Alias	Gives a readable name to a computed expression column in query results.
A04	Aggregate Alias	Names an aggregate function result column for clarity in output or HAVING clauses.
A05	Subquery Alias	Assigns a required name to a derived table subquery in the FROM clause.
A06	CTE Alias	Names a Common Table Expression defined in a WITH clause for reuse in the query.

CONSTRAINTS

MySQL Constraints

1. NOT NULL

Ensures a column must have a value. Or cannot be null.

Situation 01: defining when creating the table

Eg:- `name VARCHAR(100) NOT NULL`

Situation 02: defining after creating the table

Eg:- `name VARCHAR(100) NOT NULL`

2. UNIQUE

This column can be null. But cannot be duplicated.

Situation 01: defining when creating the table in column line

Eg:- `email VARCHAR(100) UNIQUE`

Situation 02: defining after creating the table in column line

Eg:- `ALTER TABLE customer
MODIFY email VARCHAR(100) UNIQUE;`

Situation 03: defining in CREATE TABLE statement at the end

```
Eg:- CREATE TABLE customer(  
      customer_id VARCHAR(20) PRIMARY KEY,  
      name VARCHAR(50) NOT NULL,  
      nic VARCHAR(20),  
      email VARCHAR(100),  
      amount DECIMAL(8,2),  
      -----  
);
```

The below situations describe the syntaxes which need to inserted in the blanks.

Sub situation 01: only defining

Eg:- `UNIQUE (email)`
or
Eg:- `UNIQUE KEY (email)`
or
Eg:- `UNIQUE INDEX (email)`

Sub situation 02: defining and naming

Eg:- `CONSTRAINT uq_email UNIQUE (email)`
or
Eg:- `UNIQUE KEY uq_email (email)`
or
Eg:- `UNIQUE INDEX uq_email (email)`

Situation 04: defining after creating the table at the end

```
Eg:- CREATE TABLE customer(  
      customer_id VARCHAR(20) PRIMARY KEY,  
      name VARCHAR(50) NOT NULL,  
      nic VARCHAR(20),  
      email VARCHAR(100),  
      amount DECIMAL(8,2),  
);  
  
ALTER TABLE customer  
-----;
```

The below situations describe the syntaxes which need to inserted in the blanks.

Sub situation 01: only defining

Eg:- `ADD UNIQUE (email)`
or
Eg:- `ADD UNIQUE KEY (email)`
or
Eg:- `ADD UNIQUE INDEX (email)`

Sub situation 02: defining and naming

Eg:- `ADD CONSTRAINT uq_email UNIQUE (email)`

or

Eg:- `ADD UNIQUE KEY uq_email (email)`

or

Eg:- `ADD UNIQUE INDEX uq_email (email)`

Situation 05: defining composite unique constraints when creating the table at the end

```
Eg:- CREATE TABLE orders (  
      user_id INT,  
      product_id INT,  
      -----  
    );
```

The below situations describe the syntaxes which need to inserted in the blanks.

Sub situation 01: only defining

Eg:- `UNIQUE (user_id, product_id)`

or

Eg:- `UNIQUE KEY (user_id, product_id)`

or

Eg:- `UNIQUE INDEX (user_id, product_id)`

Sub situation 02: defining and naming

Eg:- `CONSTRAINT uq_user_product UNIQUE (user_id, product_id)`

or

Eg:- `UNIQUE KEY uq_user_product (user_id, product_id)`

or

Eg:- `UNIQUE INDEX uq_user_product (user_id, product_id)`

Situation 06: defining composite unique constraints after creating the table

```
Eg:- CREATE TABLE orders (  
      user_id INT,  
      product_id INT,  
      );  
  
      ALTER TABLE orders  
      -----;
```

The below situations describe the syntaxes which need to be inserted in the blanks.

Sub situation 01: only defining

```
Eg:- ADD UNIQUE (user_id, product_id)  
      or  
Eg:- ADD UNIQUE KEY (user_id, product_id)  
      or  
Eg:- ADD UNIQUE INDEX (user_id, product_id)
```

Sub situation 02: defining and naming

```
Eg:- ADD UNIQUE KEY uq_user_product (user_id, product_id)  
      or  
Eg:- ADD UNIQUE INDEX uq_user_product (user_id, product_id)  
      or  
Eg:- ADD CONSTRAINT uq_user_product UNIQUE (user_id, product_id)
```

Situation 07: dropping the UNIQUE constraint

```
Eg:- ALTER TABLE orders  
      DROP INDEX uq_user_product;
```

3. PRIMARY KEY

The key which uniquely identifies each record. This column can't be null. And cannot be duplicated. Differ from UNIQUE by the NOT NULL feature. Simply cannot be null , cannot be duplicated.

Situation 01: used when creating the table

Eg:- `customer_id VARCHAR(20) PRIMARY KEY`

Situation 02: used when creating the table.at the end of column names

Eg:- `CREATE TABLE customer(
 customer_id VARCHAR(20),
 name VARCHAR(50) NOT NULL,
 amount DECIMAL(8,2),
 PRIMARY KEY customer_id
);`

Situation 03: used after creating the table

Eg:- `ALTER TABLE customer
 ADD PRIMARY KEY (customer_id);`

Situation 04: used in defining composite primary key when creating the table

Eg:- `CREATE TABLE student_subject(
 student_id VARCHAR(20),
 subject_id VARCHAR(20),
 marks INT NOT NULL,
 PRIMARY KEY (student_id , subject_id)
);`

Situation 05: used in defining composite primary key after creating the table

Eg:- `ALTER TABLE student_subject
 ADD PRIMARY KEY (student_id , subject_id);`

Situation 06: dropping a primary of a table

Eg: - `ALTER TABLE student_subject
DROP PRIMARY KEY;`

4. FOREIGN KEY

A Foreign Key is a column in one table that refers to the Primary Key of another table. It creates a link/relationship between two tables and ensures data consistency.

Situation 01: defining when creating the table

```
Eg:- CREATE TABLE orders(  
      order_id VARCHAR(20) PRIMARY KEY,  
      customer_id VARCHAR(20),  
      email VARCHAR(100) UNIQUE,  
      -----  
      );
```

The below situations describe the syntaxes which need to be inserted in the blanks.

Sub situation 01: defining only

```
Eg:- FOREIGN KEY (customer_id)  
      REFERENCES customer(customer_id)
```

Sub situation 02: defining and naming

```
Eg:- CONSTRAINT uk_customer  
      FOREIGN KEY (customer_id)  
      REFERENCES customer(customer_id)
```

Situation 02: defining after creating the table

```
Eg:- CREATE TABLE orders (  
      user_id INT,  
      product_id INT,  
      );  
  
      ALTER TABLE orders  
      -----;
```

The below situations describe the syntaxes which need to be inserted in the blanks.

Sub situation 01: defining only

Eg:- `ADD FOREIGN KEY (customer_id)
REFERENCES customer(customer_id)`

Sub situation 02: defining and naming

Eg:- `ADD CONSTRAINT uk_customer
FOREIGN KEY (customer_id)
REFERENCES customer(customer_id)`

Sub situation 03: dropping the foreign key constraint

Eg:- `ALTER TABLE orders
DROP CONSTRAINT uk_customer;`

Eg:- `ALTER TABLE orders
DROP FOREIGN KEY uk_customer;`

Eg:- `ALTER TABLE orders
DROP INDEX uk_customer;`

- WE CANNOT USE THE SAME FOREIGN KEY NAME EVEN AFTER DROPPING THE CONSTRAINT USING "DROP CONSTRAINT" OR "DROP FOREIGN KEY".
- TO DO THIS WE MUST DELETE THE FOREIGN KEY CONSTRAINT USING "DROP INDEX" STATEMENT.

Situation 03: “ON UPDATE” statements under the foreign key constraint

Sub situation 01: “ON UPDATE CASCADE”

When the parent primary key is updated, the child foreign key is automatically updated too.

Eg:- `ALTER TABLE orders
ADD CONSTRAINT uk_customer
FOREIGN KEY (customer_id)
REFERENCES customer(customer_id)
ON UPDATE CASCADE;`

Sub situation 02: "ON UPDATE SET NULL"

When the parent primary key is updated, the child foreign key becomes NULL.

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON UPDATE SET NULL;
```

Sub situation 03: "ON UPDATE RESTRICT" or "ON UPDATE NO ACTION"

Blocks/prevents updating the parent primary key if child rows exist.

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON UPDATE RESTRICT;
```

OR

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON UPDATE NO ACTION;
```

Situation 04: "ON DELETE" statements under the foreign key constraint

Sub situation 01: "ON DELETE CASCADE"

When the parent row is deleted, the child rows are automatically deleted too.

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON DELETE CASCADE;
```

Sub situation 02: “ON DELETE SET NULL”

When the parent row is deleted, the child foreign key column becomes NULL – child row still exists.

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON DELETE SET NULL;
```

Sub situation 03: “ON DELETE RESTRICT” or “ON DELETE NOT ACTION”

Blocks/prevents deleting the parent row if child rows exist.

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON DELETE RESTRICT;
```

OR

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id)
      ON DELETE NO ACTION;
```

- ❖ We can also use one ON UPDATE statement and one ON DELETE statement together

```
Eg:- ALTER TABLE orders
      ADD CONSTRAINT uk_customer
      FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
      ON DELETE CASCADE
      ON UPDATE SET NULL;
```


5. AUTO_INCREMENT

AUTO_INCREMENT automatically generates a unique number for a column every time a new row is inserted – you do not need to provide the value manually.

Datatypes which allow AUTO_INCREMENT datatypes are:

INT, TINYINT, BIGINT, SMALLINT

```
Eg:- CREATE TABLE customer(  
      customer_id INT AUTO_INCREMENT PRIMARY KEY,  
      name VARCHAR(50),  
      dob DATE,  
    );
```

The column should be constrained as PRIMARY KEY or UNIQUE. Must include a PRIMARY KEY or UNIQUE when defining a column to AUTO_INCREMENT.

```
Eg:- CREATE TABLE customer(  
      customer_id INT AUTO_INCREMENT PRIMARY KEY,  
      name VARCHAR(50),  
      dob DATE,  
    );
```

```
Eg:- CREATE TABLE customer(  
      customer_id INT AUTO_INCREMENT UNIQUE,  
      name VARCHAR(50),  
      dob DATE,  
    );
```

Inserting to the table without mentioning the id is a better way to the column defined as AUTO_INCREMENT. The value will be automatically defined to those columns.

```
Eg:- CREATE TABLE users(  
      id INT AUTO_INCREMENT PRIMARY KEY,  
      name VARCHAR(50),  
    );
```

```
INSERT INTO users (name)
VALUES
('rasalatchumi'),
('thurkka'),
('senthuran');
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
3	senthuran

Even if we try to insert a customized INT primary key value, MySQL ignore it.

```
Eg:- INSERT INTO users (id, name) VALUES (NULL, 'Bob');
      or
      INSERT INTO users (id, name) VALUES (0, 'Charlie');
      or
      INSERT INTO users (id, name) VALUES (50, 'David');
```

These inserts are accepted. But the commands for insert values for **id** are ignored.

Even if we delete the already id auto incremented row, MySQL will not give the deleted id number to another row. The table will be show as that value missed in the middle. That value will not reused by MySQL. That value permanently ignored in the relevant table.

```
Eg:- SELECT * FROM users;
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
3	senthuran

```
DELETE FROM users
WHERE id = 3;
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka

```
INSERT INTO users (name)
VALUES
('bineth');
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
4	bineth

If we need to start numbering from a value , we can do it like below,

```
Eg:- CREATE TABLE users(
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(50),
    ) AUTO_INCREMENT = 100 ;
```

```
INSERT INTO users (name)
VALUES
('rasalatchumi'),
('thurkka'),
('senthuran');
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
100	rasalatchumi
101	thurkka
102	senthuran

We can also define the starting value after creating the table

Sub situation 01: Defining starting value before inserting

```
Eg:- CREATE TABLE users(
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(50),
    );
```

```
ALTER TABLE users
AUTO_INCREMENT = 100;
```

```
INSERT INTO users (name)
VALUES
('rasalatchumi'),
('thurkka'),
('senthuran');
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
100	rasalatchumi
101	thurkka
102	senthuran

Sub situation 02: Defining starting value after inserting some values

Eg:-

```
CREATE TABLE users(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
);
```

```
INSERT INTO users (name)
VALUES
('rasalatchumi'),
('thurkka'),
('senthuran');
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
3	senthuran

```
ALTER TABLE users
AUTO_INCREMENT = 100;
```

```
INSERT INTO users (name)
VALUES
('bineth'),
('sameera'),
('isira');
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
3	senthuran
100	bineth
101	sameera
102	isira

This will not support if we command to start from a value that is low from current maximum value. If we do like this that command will be ignored and the next auto incremented value will assigned.

```
Eg:- CREATE TABLE users(  
      id INT AUTO_INCREMENT PRIMARY KEY,  
      name VARCHAR(50),  
    );
```

```
INSERT INTO users (name)  
VALUES  
('rasalatchumi'),  
('thurkka'),  
('senthuran');
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
3	senthuran

```
ALTER TABLE users  
AUTO_INCREMENT = 2;
```

```
INSERT INTO users (name)  
VALUES  
('bineth'),  
('sameera'),  
('isira');
```

```
SELECT * FROM users;
```

<i>id</i>	<i>name</i>
1	rasalatchumi
2	thurkka
3	senthuran
4	bineth
5	sameera
6	isira

- adding auto increment after creating the table

Eg:-

```
ALTER TABLE users  
MODIFY user_id INT AUTO_INCREMENT;
```

- Removing auto increment

Eg:-

```
ALTER TABLE users  
MODIFY user_id INT;
```

6. DEFAULT

DEFAULT constraint automatically assigns a value to a column when no value is provided during INSERT.

how each datatypes can include default values

INT	DEFAULT 1
VARCHAR	DEFAULT "no available"
DECIMAL	DEFAULT "2.5"
BOOLEAN	DEFAULT TRUE or DEFAULT FALSE
DATETIME or TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
DATE	DEFAULT (CURRENT_DATE)

```
Eg:- CREATE TABLE orders (  
      order_id INT AUTO_INCREMENT,  
      customer_name VARCHAR(20) NOT NULL,  
      status VARCHAR(20) DEFAULT "pending",  
      quantity INT DEFAULT 1,  
      is_paid BOOLEAN DEFAULT ,  
      reg_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
      attempt_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
      date DEFAULT (CURRENT_DATE),  
      PRIMARY KEY (order_id)  
);
```

Situation 01: Adding default constraint when creating the table

```
Eg:- CREATE TABLE orders (  
      order_id INT AUTO_INCREMENT,  
      customer_name VARCHAR(20) NOT NULL,  
      status VARCHAR(20) DEFAULT "pending",  
      PRIMARY KEY (order_id)  
);
```

Situation 02: Adding default constraint after creating the table

```
Eg:- CREATE TABLE orders (  
      order_id INT AUTO_INCREMENT PRIMARY KEY,  
      customer_name VARCHAR(20) NOT NULL,  
      status VARCHAR(20)  
);
```

Sub situation 01: Using alter

Eg:- `ALTER TABLE orders
ALTER status SET DEFAULT "pending";`

Sub situation 02: Using modify

Eg:- `ALTER TABLE orders
MODIFY status VARCHAR(20) DEFAULT "pending";`

Situation 03: Inserting data

Sub situation 01: If we need to insert data to the default defined column

Eg:- `INSERT INTO orders (customer_name, status)
VALUES ("senthuran","finished");`

`SELECT * FROM orders;`

<i>order_id</i>	<i>customer_name</i>	<i>status</i>
1	senthuran	finished

Sub situation 02: If we don't need to insert to the default defined column

○ Method 1

Mentioning as default in the relevant column when inserting

Eg:- `INSERT INTO orders (customer_name, status)
VALUES ("thurkka", DEFAULT);`

`SELECT * FROM orders;`

<i>order_id</i>	<i>customer_name</i>	<i>status</i>
1	senthuran	finished
2	thurkka	pending

- Method 2

Doesn't mentioning the column while inserting.

Eg:- `INSERT INTO orders (customer_name)
VALUES ("senthuran");`

`SELECT * FROM orders;`

<i>order_id</i>	<i>customer_name</i>	<i>status</i>
1	senthuran	finished
2	thurkka	pending
3	rasalatchumi	peding

Situation 04: Adding default after creating the table

Eg:- `CREATE TABLE orders (
 order_id INT AUTO_INCREMENT,
 customer_name VARCHAR(20) NOT NULL,
 status VARCHAR(20),
 PRIMARY KEY (order_id)
);`

`ALTER TABLE orders
ALTER status SET DEFAULT "pending";`

Situation 05: Dropping default constraint

Eg:- `ALTER TABLE orders
ALTER status DROP DEFAULT "pending";`

Situation 06: "ON UPDATE" statement with DEFAULT

This allows to auto update the date and time of updation when we updating a data row.

Sub situation 01: when creating the table

Eg:- `CREATE TABLE orders (
 status VARCHAR(20) DEFAULT 'pending',
 quantity INT DEFAULT 1,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
 updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
 ON UPDATE CURRENT_TIMESTAMP
);`

Sub situation 02: after creating the table

```
Eg:- ALTER TABLE orders
      MODIFY updated_at DATETIME
      DEFAULT CURRENT_TIMESTAMP
      ON UPDATE CURRENT_TIMESTAMP;
```

Sub situation 03: removing "ON UPDATE" if already defined

```
Eg:-CREATE TABLE orders (
      status VARCHAR(20) DEFAULT 'pending',
      quantity INT DEFAULT 1,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
      ON UPDATE CURRENT_TIMESTAMP
);
```

```
ALTER TABLE orders
MODIFY updated_at DATETIME
      DEFAULT CURRENT_TIMESTAMP;
```

7. CHECK

CHECK constraint validates data based on a condition before inserting or updating a row. If the condition is false, MySQL rejects the row. The conditions are indicated by operators.

Situation 01: Defining with column line

Eg:- `dob DATE,
appointed_date DATE CHECK(appointed_date > dob),`

Situation 02: Defining at the end of the CREATE TABLE query

Sub situation 01: Only defining

Eg:- `CREATE TABLE users (
 age INT,
 CHECK (age >= 18)
);`

Sub situation 02: Defining and naming

Eg:- `CREATE TABLE users (
 age INT,
 CONSTRAINT chk_age CHECK (age >= 18)
);`

Situation 03: Defining after creating the table

Sub situation 01: Only defining

Eg:- `ALTER TABLE users
ADD CHECK (age >= 18);`

Sub situation 02: Defining and naming

Eg:- `ALTER TABLE users
ADD CONSTRAINT chk_age CHECK (age >= 18);`

Situation 04: Dropping check constraint

Eg:- `ALTER TABLE users
DROP CHECK chk_age;`

CHECK will pass the row if the column is inserted as null. So, we can also use CHECK constraint with NOT NULL.

NOTE 01: Disable and enable constraints.

Disable: `SET constraint_name = 0;`

Enable: `SET constraint_name = 1;`

Constraint allowing this are:

- FOREIGN KEY
- PRIMARY KEY
- UNIQUE

Why this is used:

Bulk data import	Speed up millions of inserts
Drop tables	Avoid FK reference errors
Truncate tables	Clear parent tables without errors
Restore backup	Restore tables in any order
Reordering data	Update referenced keys safely
Testing/Development	Insert test data without order concerns

Eg:- `SET foreign_key_checks = 0;`

`DROP TABLE orders;`

`DROP TABLE users;`

`SET foreign_key_checks = 1;`

Never leave constraints disabled – it risks inserting bad or inconsistent data into your database.

OPERATORS

MySQL Operators

1. ARITHMETIC OPERATORS

1.01 + (ADDITION)

What it does: Takes two numbers and adds them together.

Process: Left value + Right value = Result

```
SELECT 10 + 5;
//result: 15//

SELECT salary + 1000 AS new_salary FROM employees;
//adds 1000 to every salary row//
```

1.02 - (SUBTRACTION)

What it does: Takes two numbers and subtracts right from left.

Process: Left value - Right value = Result

```
SELECT 10 - 5;
//result: 5//

SELECT salary - 500 AS reduced_salary FROM employees;
//subtracts 500 from every salary//
```

1.03 * (MULTIPLICATION)

What it does: Multiplies two numbers together.

Process: Left value × Right value = Result

```
SELECT 10 * 5;
//result: 50//

SELECT price * quantity AS total FROM orders;
//multiplies price by quantity//
```

1.04 / (DIVISION)

What it does: Divides left number by right number .result includes decimals.

Process: Left value ÷ Right value = Result with decimals

```
SELECT 10 / 3;
//result: 3.3333//

SELECT salary / 12 AS monthly_salary FROM employees;
//divides annual salary by 12//
```

1.05 DIV (INTEGER DIVISION)

What it does: Divides left number by right number – result has NO decimals, cuts off remainder.

Process: Left value ÷ Right value = Result without decimals

```
SELECT 10 DIV 3;
//result: 3 (not 3.3333)//

SELECT 7 DIV 2;
//result: 3 (not 3.5)//
```

1.06 % (MODULUS)

What it does: Divides two numbers and returns only the remainder.

Process: Left value ÷ Right value = Remainder only

```
SELECT 10 % 3;
//result: 1 (10 divided by 3 leaves remainder 1)//

SELECT 10 % 2;
//result: 0 (even number, no remainder)//

SELECT 11 % 2;
//result: 1 (odd number)//

SELECT order_id % 2 FROM orders;
//0 = even order, 1 = odd order//
```

1.07 MOD (MODULUS)

What it does: Exactly same as % operator – just a word version.

Process: Left value ÷ Right value = Remainder only

```
SELECT 10 MOD 3;  
//result: 1 (same as 10 % 3)//
```

```
SELECT MOD(10, 3);  
//result: 1 (function style)//
```


2. COMPARISON OPERATORS

2.01 = (EQUAL TO)

What it does: Checks if two values are exactly equal.

Process: Compares left and right → returns TRUE if equal, FALSE if not.

```
SELECT * FROM users WHERE age = 18;
//returns only rows where age is exactly 18//

SELECT * FROM users WHERE status = 'active';
//returns only active users//
```

2.02 != OR <> (NOT EQUAL TO)

What it does: Checks if two values are NOT equal.

Process: Compares left and right → returns TRUE if NOT equal, FALSE if equal.

```
SELECT * FROM users WHERE age != 18;
//returns all rows except age 18//

SELECT * FROM users WHERE status <> 'banned';
//returns all non-banned users//
```

2.03 > (GREATER THAN)

What it does: Checks if left value is greater than right value.

Process: Compares left and right → returns TRUE if left is bigger,

```
SELECT * FROM users WHERE age > 18;
//returns age 19, 20, 21... NOT 18//

SELECT * FROM orders WHERE price > 100;
//returns prices above 100//
```

2.04 < (LESS THAN)

What it does: Checks if left value is less than right value.

Process: Compares left and right → returns TRUE if left is smaller, does NOT include the value itself.

```
SELECT * FROM users WHERE age < 18;
//returns age 17, 16, 15... NOT 18//

SELECT * FROM orders WHERE price < 100;
//returns prices below 100//
```

2.05 >= (GREATER THAN OR EQUAL TO)

What it does: Checks if left value is greater than OR equal to right value.

Process: Compares left and right → returns TRUE if left is bigger OR exactly equal. INCLUDES the value itself.

```
SELECT * FROM users WHERE age >= 18;
//returns age 18, 19, 20... INCLUDES 18//

SELECT * FROM orders WHERE price >= 100;
//returns prices 100 and above//
```

2.06 <= (LESS THAN OR EQUAL TO)

What it does: Checks if left value is less than OR equal to right value.

Process: Compares left and right → returns TRUE if left is smaller OR exactly equal. INCLUDES the value itself.

```
SELECT * FROM users WHERE age <= 18;
//returns age 18, 17, 16... INCLUDES 18//

SELECT * FROM orders WHERE price <= 100;
//returns prices 100 and below//
```

2.07 IS (CHECKS , BOOLEAN OR NULL)

What it does: Checks if a value IS TRUE, FALSE, or NULL.

Process: Evaluates the boolean or NULL state of a value → returns TRUE or FALSE.

```
SELECT * FROM users WHERE active IS TRUE;
//returns active users//

SELECT * FROM users WHERE active IS FALSE;
//returns inactive users//
```

```
SELECT * FROM users WHERE email IS NULL;  
//returns users with no email//
```

2.08 IS NOT (CHECK , NOT BOOLEAN OR NULL)

What it does: Checks if a value is NOT TRUE, NOT FALSE, or NOT NULL.

Process: Evaluates the boolean or NULL state → returns TRUE if condition is NOT met.

```
SELECT * FROM users WHERE active IS NOT TRUE;  
//returns non-active users//  
  
SELECT * FROM users WHERE active IS NOT FALSE;  
//returns active users//  
  
SELECT * FROM users WHERE email IS NOT NULL;  
//returns users who have email//
```

2.09 BETWEEN (BETWEEN A RANGE)

What it does: Checks if a value falls within a range – INCLUDES both end values.

Process: Checks if value $\geq$ lower AND value $\leq$ upper → returns TRUE if within range.

```
SELECT * FROM users WHERE age BETWEEN 18 AND 30;  
//returns age 18, 19, 20 ... 30 (includes both 18 and 30)//  
  
SELECT * FROM orders WHERE price BETWEEN 100 AND 500;  
//returns prices from 100 to 500//
```

2.10 NOT BETWEEN (NOT BETWEEN A RANGE)

What it does: Checks if a value falls OUTSIDE a range.

Process: Checks if value $<$ lower OR value $>$ upper → returns TRUE if outside range.

```
SELECT * FROM users WHERE age NOT BETWEEN 18 AND 30;  
//returns age below 18 or above 30//  
  
SELECT * FROM orders WHERE price NOT BETWEEN 100 AND 500;  
//returns prices below 100 or above 500//
```

2.11 IN (MATCH ANY VALUE IN LIST)

What it does: Checks if a value matches ANY value in a given list.

Process: Compares value against each item in list → returns TRUE if any match found.

```
SELECT * FROM users WHERE status IN ('active','pending','approved');  
//returns users whose status is active OR pending OR approved//
```

```
SELECT * FROM orders WHERE country IN ('USA', 'UK', 'Canada');
```

2.12 NOT IN (NOT MATCH ANY VALUE IN LIST)

What it does: Checks if a value does NOT match ANY value in a given list.

Process: Compares value against each item in list → returns TRUE if NO match found.

```
SELECT * FROM users WHERE status NOT IN ('banned','inactive');  
//returns users who are NOT banned and NOT inactive//
```

2.13 LIKE (PATTERN MATCHING)

What it does: Checks if a string matches a pattern using wildcards.

Process: Matches string against pattern character by character → % means any characters, _ means exactly one character.

```
SELECT * FROM users WHERE name LIKE 'A%';  
//starts with A//
```

```
SELECT * FROM users WHERE name LIKE '%son';  
//ends with son//
```

```
SELECT * FROM users WHERE name LIKE '%oh%';  
//contains oh anywhere//
```

```
SELECT * FROM users WHERE name LIKE 'A_';  
//A + exactly one character//
```

```
SELECT * FROM users WHERE email LIKE '%@gmail.com';  
//gmail users//
```

2.14 NOT LIKE (NOT PATTERN MATCHING)

What it does: Checks if a string does NOT match a pattern.

Process: Same as LIKE but returns TRUE when pattern does NOT match.

```
SELECT * FROM users WHERE name NOT LIKE 'A%';
```

```
//names NOT starting with A//
```

```
SELECT * FROM users WHERE email NOT LIKE '%@gmail.com';
```

```
//non gmail users//
```

3. LOGICAL OPERATORS

3.01 AND / &&

What it does: Combines two conditions – BOTH must be true to return a row.

Process: Evaluates left condition → evaluates right condition → returns TRUE only if BOTH are true.

```
SELECT * FROM users WHERE age >= 18 AND status = 'active';

//returns ONLY users who are BOTH 18+ AND active//

//if age is 18 but status is banned → NOT returned//

//if age is 15 but status is active → NOT returned//
```

3.02 OR / ||

What it does: Combines two conditions – AT LEAST ONE must be true to return a row.

Process: Evaluates left condition → if true returns row, if false evaluates right condition → returns TRUE if either is true.

```
SELECT * FROM users WHERE age < 18 OR age > 60;

//returns users younger than 18 OR older than 60//

//if age is 15 → returned (first condition true)//

//if age is 65 → returned (second condition true)//

//if age is 30 → NOT returned (both false)//
```

3.03 NOT / !

What it does: Reverses a condition – makes TRUE become FALSE and FALSE become TRUE.

Process: Evaluates condition → flips the result → returns TRUE if original was FALSE.

```
SELECT * FROM users WHERE NOT status = 'banned';

//returns all users EXCEPT banned ones//

//same as: WHERE status != 'banned'//
```

```
SELECT * FROM users WHERE NOT age < 18;
```

```
//returns users 18 and above//
```

```
//same as: WHERE age >= 18//
```

4. SUBQUERY OPERATORS

4.01 ALL

What it does: Compares a value against ALL values returned by a subquery – ALL must satisfy the condition.

Process: Runs subquery → compares value against every result → returns TRUE only if condition is true for ALL values.

```
SELECT * FROM employees
WHERE salary > ALL (SELECT salary FROM managers);

//subquery returns: 3000, 5000, 7000//

//employee must have salary > 7000 (greater than ALL = greater
than highest)//
```

4.02 ANY / SOME

What it does: Compares a value against ANY value in subquery – at least ONE must satisfy the condition.

Process: Runs subquery → compares value against each result → returns if condition is true for AT LEAST ONE value.

```
SELECT * FROM employees
WHERE salary > ANY (SELECT salary FROM managers);

//subquery returns: 3000, 5000, 7000//

//employee must have salary > 3000 (greater than ANY = greater
than lowest)//
```

4.03 EXISTS

What it does: Checks if a subquery returns ANY rows at all – does not care about values, only whether rows exist.

Process: Runs subquery → if at least one row returned → TRUE, if no rows returned → FALSE.

```
SELECT * FROM users
WHERE EXISTS (
    SELECT * FROM orders
    WHERE orders.user_id = users.id
);
```



```
//for each user, checks if they have any orders//  
//returns user ONLY if at least one order found//
```

```
SELECT * FROM users  
WHERE NOT EXISTS (  
    SELECT * FROM orders  
    WHERE orders.user_id = users.id  
);
```

```
//returns users who have NO orders at all//
```

5. ASSIGNMENT OPERATORS

5.01 = (ASSIGNMENT)

What it does: Assigns a value to a variable – only works inside SET statement.

Process: Stores given value into the variable for later use.

```
SET @total = 0;

SET @name = 'John';

SET @price = 99.99;


//Use the variable later//

SELECT * FROM orders WHERE price > @price;
```

5.02 := (ASSIGNMENT)

What it does: Assigns a value to a variable – works ANYWHERE including SELECT.

Process: Stores given value into the variable at the point it is evaluated.

```
SET @total := 0;


//Use in SELECT to create row numbers//

SELECT @row_num := @row_num + 1 AS row_number, name
FROM users, (SELECT @row_num := 0) r;

//assigns and increments row number for each row//


//Store max salary then use it//

SELECT @max := MAX(salary) FROM employees;

SELECT * FROM employees WHERE salary = @max;
```

6. SET OPERATORS

6.01 UNION

What it does: Combines results of two SELECT queries into one result – REMOVES duplicate rows.

Process: Runs first query → runs second query → merges results → removes duplicates → returns final result.

```
SELECT name FROM users
UNION
SELECT name FROM admins;

//if 'John' exists in both tables → appears ONLY ONCE in
result//

//automatically removes duplicates//
```

6.02 UNION ALL

What it does: Combines results of two SELECT queries – KEEPS all duplicate rows.

Process: Runs first query → runs second query → merges results → keeps everything including duplicates → returns final result.

```
SELECT name FROM users
UNION ALL
SELECT name FROM admins;

//if 'John' exists in both tables → appears TWICE in result//

//faster than UNION because no duplicate check needed//
```

6.03 INTERSECT (MYSQL 8.0.31+)

What it does: Returns ONLY rows that exist in BOTH queries.

Process: Runs first query → runs second query → finds matching rows in both → returns only common rows.

```
SELECT name FROM users
INTERSECT
SELECT name FROM admins;
```

```
//returns ONLY names that exist in BOTH users AND admins  
tables//
```

```
//if John is in users but not admins → NOT returned//
```

```
//if John is in both → returned//
```

6.04 EXCEPT (MYSQL 8.0.31+)

What it does: Returns rows from first query that do NOT exist in second query.

Process: Runs first query → runs second query → removes rows from first that appear in second → returns remaining rows.

```
SELECT name FROM users  
EXCEPT  
SELECT name FROM admins;
```

```
//returns users who are NOT in admins table//
```

```
//if John is in users AND admins → NOT returned//
```

```
//if John is ONLY in users → returned//
```

CLAUSES

MySQL Clauses

1. QUERY CLAUSES

1.01 SELECT

Category: Query

What it does: Specifies which columns or expressions to retrieve from a table. Every query begins with SELECT.

```
SELECT CONCAT("John", " ", "Doe");
```

```
//result: John Doe//
```

```
SELECT CONCAT(first_name, " ", last_name)
```

```
AS full_name FROM users;
```

```
//joins first and last name//
```

Syntax:

```
SELECT column1, column2, ...
```

```
FROM table_name;
```

Example:

```
SELECT name, department, salary
```

```
FROM employees;
```

Output:

name	department	salary
Alice	HR	55000
Bob	Finance	72000
Carol	IT	68000

Key Points:

- Use * to select all columns → SELECT *
- You can rename columns using AS → SELECT name AS employee_name
- Runs after FROM in MySQL's execution order

1.02 DISTINCT

Category: Query

What it does: Removes duplicate rows from the result. Only unique rows are returned.

Syntax:

```
SELECT DISTINCT column1  
FROM table_name;
```

Example:

```
SELECT DISTINCT department  
FROM employees;
```

Output:

department
HR
Finance
IT

Key Points:

- Without DISTINCT, duplicate values would appear multiple times
- Can be used with multiple columns → SELECT DISTINCT dept, city (combination must be unique)
- Slightly slower than regular SELECT due to duplicate removal

1.03 FROM

Category: Query

What it does: Specifies the table (or tables) from which to retrieve data.

Syntax:

```
SELECT columns  
FROM table_name;
```

Example:

```
SELECT name, salary  
FROM employees;
```

Output:

name	salary
Alice	55000
Bob	72000

Key Points:

- First clause evaluated in MySQL's execution order
- Can reference multiple tables when used with JOIN
- Can use a subquery as a table → FROM (SELECT ...) AS sub

1.04 WHERE

Category: Query

What it does: Filters rows before any grouping occurs. Only rows that match the condition are included in the result.

Syntax:

```
SELECT columns
FROM table_name
WHERE condition;
```

Example:

```
SELECT name, salary
FROM employees
WHERE salary > 60000;
```

Output:

name	salary
Bob	72000
Carol	68000

Key Points:

- Runs before GROUP BY and SELECT in execution order
- Cannot use SELECT column aliases here (alias not yet defined)
- Use AND, OR, NOT to combine multiple conditions

1.05 GROUP BY

Category: Query

What it does: Groups rows that share the same value in a column.
Used with aggregate functions like COUNT, SUM, AVG, MAX, MIN.

Syntax:

```
SELECT column, AGGREGATE(column2)
FROM table_name
GROUP BY column;
```

Example:

```
SELECT department, COUNT(*) AS total, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output:

department	total	avg_salary
HR	4	52000
Finance	3	71000
IT	5	67000

Key Points:

- Every column in SELECT must either be in GROUP BY or be inside an aggregate function
- Runs after WHERE and before HAVING
- Can group by multiple columns → GROUP BY dept, city

1.06 WITH ROLLUP

Category: Query

What it does: An extension of GROUP BY that adds extra summary rows showing subtotals and a grand total at the end of the result.

Syntax:

```
SELECT column, AGGREGATE(column2)
FROM table_name
GROUP BY column WITH ROLLUP;
```

Example:

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department WITH ROLLUP;
```

Output:

department	total_salary
Finance	213000
HR	208000
IT	335000
NULL	756000

Key Points:

- The NULL row at the bottom represents the grand total of all groups
- Use IFNULL(department, 'Grand Total') to replace NULL with a label
- Only ROLLUP is supported in MySQL – CUBE and GROUPING SETS are not

1.07 HAVING

Category: Query

What it does: Filters groups after GROUP BY. Use HAVING when you want to filter based on aggregate values like COUNT or SUM.

Syntax:

```
SELECT column, AGGREGATE(column2)
FROM table_name
GROUP BY column
HAVING condition;
```

Example:

```
SELECT department, COUNT(*) AS total
FROM employees
GROUP BY department
HAVING total > 3;
```

Output:

department	total
HR	4
IT	5

Key Points:

- HAVING runs after GROUP BY; WHERE runs before GROUP BY
- Use WHERE to filter rows, use HAVING to filter groups
- MySQL allows column aliases from SELECT to be used in HAVING

1.08 ORDER BY

Category: Query

What it does: Sorts the result set by one or more columns. Default is ascending (ASC). Use DESC for descending order.

Syntax:

```
SELECT columns
FROM table_name
ORDER BY column1 ASC|DESC, column2 ASC|DESC;
```

Example:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Output:

name	salary
Bob	72000
Carol	68000
Alice	55000

Key Points:

- Runs last in execution order (just before LIMIT)
- Can sort by multiple columns → ORDER BY dept ASC, salary DESC
- Can use SELECT aliases here → ORDER BY annual_salary DESC
- First consider column1 and then consider column2

1.09 LIMIT

Category: Query

What it does: Restricts how many rows are returned in the result. Very useful for pagination and previewing data.

Syntax:

```
SELECT columns  
FROM table_name  
LIMIT number;
```

Example:

```
SELECT name, salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 3;
```

Output:

name	salary
Bob	72000
Carol	68000
Dave	65000

Key Points:

- Always pair with ORDER BY to get predictable results
- Runs last in execution order
- Can be combined with OFFSET for pagination → LIMIT 10 OFFSET 20

1.10 OFFSET

Category: Query

What it does: Skips a specified number of rows before starting to return results. Paired with LIMIT for pagination.

Syntax:

```
SELECT columns
FROM table_name
LIMIT number OFFSET skip_count;
```

Example:

```
//Page 3: skip first 20 rows, show next 10//
```

```
SELECT name, salary
FROM employees
ORDER BY name
LIMIT 10 OFFSET 20;
```

Output:

name	salary
Karen	61000
Laura	54000
...	...

Key Points:

- OFFSET 0 and no OFFSET are identical
- Shorthand: LIMIT 20, 10 means OFFSET 20, LIMIT 10
- For large datasets, high OFFSET values can be slow

1.11 INTO

Category: Query

What it does: Exports query results directly into a file on the server. Two variants: OUTFILE (text) and DUMPFILE (binary).

Syntax:

```
SELECT columns
FROM table_name
INTO OUTFILE "file_path";
```

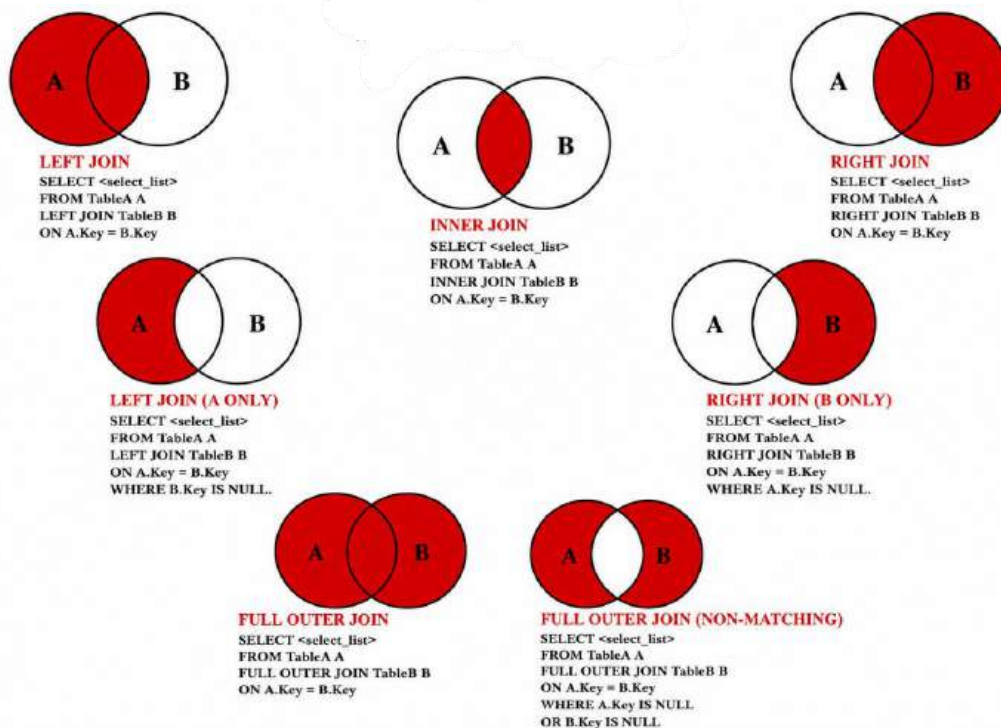
Example:

```
SELECT name, salary
FROM employees
INTO OUTFILE "/tmp/employees.csv"
FIELDS TERMINATED BY ","
LINES TERMINATED BY "\n";
```

Key Points:

- File is written on the MySQL server machine, not the client
- MySQL user needs FILE privilege
- Use LOAD DATA INFILE to import back

2. JOIN CLAUSES



2.01 INNER JOIN

Category: JOIN

What it does: Returns only rows where there is a matching value in both tables. Rows with no match are excluded from the result.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2 ON table1.column = table2.column;
```

Example:

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id;
```

Output:

name	department_name
Alice	HR
Bob	Finance
Carol	IT

Key Points:

- Most common JOIN type
- Employees without a matching department are excluded
- JOIN alone means INNER JOIN in MySQL

2.02 LEFT JOIN

Category: JOIN

What it does: Returns all rows from the left table, and matching rows from the right table. If no match exists, the right side shows NULL.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2 ON table1.column = table2.column;
```

Example:

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;
```

Output:

name	department_name
Alice	HR
Bob	Finance
Eve	NULL

Key Points:

- Eve has no department, so department_name is NULL
- Also written as LEFT OUTER JOIN
- Use WHERE d.id IS NULL to find unmatched left rows only

2.03 RIGHT JOIN

Category: JOIN

What it does: Returns all rows from the right table, and matching rows from the left table. If no match exists, the left side shows NULL.

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

Example:

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.id;
```

Output:

name	department_name
Alice	HR
Bob	Finance
NULL	Legal

Key Points:

- Legal department has no employees, so name is NULL
- Also written as RIGHT OUTER JOIN
- Most RIGHT JOINS can be rewritten as LEFT JOINS by swapping tables

2.04 CROSS JOIN

Category: JOIN

What it does: Returns every possible combination of rows from both tables (Cartesian product). No ON condition is needed.

Syntax:

```
SELECT columns  
FROM table1  
CROSS JOIN table2;
```

Example:

```
SELECT e.name, p.project_name  
FROM employees e  
CROSS JOIN projects p;
```

Output:

name	project_name
Alice	Alpha
Alice	Beta
Bob	Alpha
Bob	Beta

Key Points:

- If table1 has 10 rows and table2 has 5 rows, result has 50 rows
- No ON condition – use carefully on large tables
- Useful for generating combinations or test data

2.05 SELF JOIN

Category: JOIN

What it does: Joins a table with itself. Used to compare rows within the same table, such as finding an employee's manager who is also in the same employees table.

Syntax:

```
SELECT a.column, b.column
FROM table a
INNER JOIN table b ON a.column = b.column;
```

Example:

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
INNER JOIN employees m ON e.manager_id = m.id;
```

Output:

employee	manager
Bob	Alice
Carol	Alice
Dave	Bob

Key Points:

- Requires table aliases to distinguish the two copies of the same table
- Not a separate keyword – it's an INNER/LEFT JOIN applied to the same table
- Commonly used for hierarchical data like org charts

2.06 ON

Category: JOIN

What it does: Defines the condition used to match rows between two joined tables. Specifies which columns to compare.

Syntax:

```
SELECT columns
FROM table1
JOIN table2 ON table1.column = table2.column;
```

Example:

```
SELECT o.order_id, c.customer_name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id;
```

Output:

order_id	customer_name
101	Alice
102	Bob

Key Points:

- Can use any comparison operator: =, >, <, !=
- Can include multiple conditions → ON a.id = b.id AND a.status = 'active'
- Evaluated before WHERE in execution order

2.07 USING

Category: JOIN

What it does: A shorthand for ON when both tables have a column with the exact same name. Cleaner syntax than ON for common column names.

Syntax:

```
SELECT columns
FROM table1
JOIN table2 USING (column_name);
```

Example:

```
SELECT name, department_name
FROM employees
INNER JOIN departments USING (dept_id);
```

Output:

name	department_name
Alice	HR
Bob	Finance

Key Points:

- The shared column appears only once in the result (unlike ON where it appears twice)
- Both tables must have the exact same column name
- Cannot use table prefix with USING → USING (dept_id) not USING (e.dept_id)

3. SET OPERATION CLAUSES

3.01 UNION

Category: Set Operations

What it does: Combines the results of two or more SELECT queries into one result set. Duplicate rows are automatically removed.

Syntax:

```
SELECT columns FROM table1
UNION
SELECT columns FROM table2;
```

Example:

```
SELECT name FROM employees
UNION
SELECT name FROM contractors;
```

Output:

name
Alice
Bob
Carol

Key Points:

- Both queries must have the same number of columns and compatible data types
- Slower than UNION ALL because it removes duplicates
- Column names come from the first SELECT statement

3.02 UNION ALL

Category: Set Operations

What it does: Combines results from two SELECT queries keeping all rows including duplicates. Faster than UNION.

Syntax:

```
SELECT columns FROM table1
UNION ALL
SELECT columns FROM table2;
```

Example:

```
SELECT name FROM employees
UNION ALL
SELECT name FROM contractors;
```

Output:

name
Alice
Bob
Alice
Carol

Key Points:

- Alice appears twice because she exists in both tables
- Faster than UNION – no duplicate removal step
- Use when you know there are no duplicates or want to keep them

3.03 INTERSECT

Category: Set Operations

What it does: Returns only the rows that appear in both SELECT query results. (*Available in MySQL 8.0.31+*)

Syntax:

```
SELECT columns FROM table1
INTERSECT
SELECT columns FROM table2;
```

Example:

```
SELECT name FROM employees
INTERSECT
SELECT name FROM contractors;
```

Output:

name
Alice

Key Points:

- Only rows common to both queries are returned
- Available from MySQL 8.0.31 onwards
- Equivalent to an INNER JOIN on all columns

3.04 EXCEPT

Category: Set Operations

What it does: Returns rows from the first query that do not appear in the second query result. (*Available in MySQL 8.0.31+*)

Syntax:

```
SELECT columns FROM table1
EXCEPT
SELECT columns FROM table2;
```

Example:

```
SELECT name FROM employees
EXCEPT
SELECT name FROM contractors;
```

Output:

name
Bob
Carol

Key Points:

- Returns rows in the first query that are NOT in the second
- Available from MySQL 8.0.31 onwards
- Equivalent to a LEFT JOIN where the right side IS NULL

4. CTE CLAUSES

4.01 WITH

Category: CTE

What it does: Creates a named temporary result set (Common Table Expression) that you can reference in the main query. Makes complex queries easier to read and maintain.

Syntax:

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT * FROM cte_name;
```

Example:

```
WITH high_earners AS (  
    SELECT name, salary  
    FROM employees  
    WHERE salary > 70000  
)  
SELECT name, salary  
FROM high_earners  
ORDER BY salary DESC;
```

Output:

name	salary
Bob	72000
Frank	75000

Key Points:

- Available in MySQL 8.0+
- CTE only exists for the duration of the query
- You can define multiple CTEs → WITH cte1 AS (...), cte2 AS (...)
- Makes nested subqueries much more readable

4.02 WITH RECURSIVE

Category: CTE

What it does: A CTE that references itself. Used for hierarchical or tree-structured data like org charts, file systems, or category trees.

Syntax:

```
WITH RECURSIVE cte_name AS (  
    //Base case (starting point)//  
  
    SELECT ...  
    UNION ALL  
  
    //Recursive case (references itself)//  
  
    SELECT ... FROM cte_name JOIN ...  
)  
SELECT * FROM cte_name;
```

Example:

```
WITH RECURSIVE emp_tree AS (  
    //Base: top-level manager (no manager above)//  
  
    SELECT id, name, manager_id, 1 AS level  
    FROM employees  
    WHERE manager_id IS NULL  
    UNION ALL  
  
    //Recursive: find each employee's subordinates//  
  
    SELECT e.id, e.name, e.manager_id, t.level + 1  
    FROM employees e  
    INNER JOIN emp_tree t ON e.manager_id = t.id  
)  
SELECT name, level  
FROM emp_tree;
```

Output:

name	level
Alice	1
Bob	2
Carol	2
Dave	3

Key Points:

- Available in MySQL 8.0+
- Must have a base case and a recursive case connected by UNION ALL
- Add a depth limit to avoid infinite loops → WHERE level < 10

5. WINDOW FUNCTION CLAUSES

5.01 OVER()

Category: Window Functions

What it does: Defines a window (a set of rows) for a window function to operate on. Without OVER(), aggregate functions collapse all rows into one. With OVER(), each row keeps its identity while the function looks at a window of rows.

Syntax:

```
SELECT column,  
       FUNCTION() OVER() AS alias  
FROM table_name;
```

Example:

```
SELECT name, salary,  
       SUM(salary) OVER() AS company_total,  
       ROUND(salary / SUM(salary) OVER() * 100, 2) AS pct  
FROM employees;
```

Output:

name	salary	company_total	pct
Alice	55000	250000	22.00
Bob	72000	250000	28.80
Carol	68000	250000	27.20

Key Points:

- Available in MySQL 8.0+
- OVER() with nothing inside applies the function to all rows
- Can be combined with PARTITION BY and ORDER BY inside OVER()

5.02 PARTITION BY

Category: Window Functions

What it does: Used inside OVER() to split rows into groups (partitions). The window function runs independently for each partition, like a GROUP BY that doesn't collapse rows.

Syntax:

```
SELECT column,  
    FUNCTION() OVER(PARTITION BY column) AS alias  
FROM table_name;
```

Example:

```
SELECT name, department, salary,  
    AVG(salary) OVER(PARTITION BY department) AS dept_avg  
FROM employees;
```

Output:

name	department	salary	dept_avg
Alice	HR	55000	52000
Carol	HR	49000	52000
Bob	Finance	72000	71000

Key Points:

- Available in MySQL 8.0+
- Each row keeps its own data while seeing the partition-level aggregate
- Similar to GROUP BY but does not reduce rows to one per group

5.03 ROWS BETWEEN

Category: Window Functions

What it does: Defines a frame of rows relative to the current row within a window. Controls which rows are included in the window function calculation based on row position.

Syntax:

```
FUNCTION() OVER(  
    ORDER BY column  
    ROWS BETWEEN start AND end  
)
```

Example:

```
SELECT name, salary,  
    SUM(salary) OVER(  
        ORDER BY salary  
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW  
    ) AS running_2row_sum  
FROM employees;
```

Output:

name	salary	running_2row_sum
Alice	49000	49000
Carol	55000	104000
Bob	72000	127000

Key Points:

- Available in MySQL 8.0+
- Frame options: UNBOUNDED PRECEDING, N PRECEDING, CURRENT ROW, N FOLLOWING, UNBOUNDED FOLLOWING
- Used for sliding window calculations like moving averages

5.04 RANGE BETWEEN

Category: Window Functions

What it does: Similar to ROWS BETWEEN but defines the frame based on value range rather than row position. Includes all rows within a value distance from the current row.

Syntax:

```
FUNCTION() OVER(  
    ORDER BY column  
    RANGE BETWEEN start AND end  
)
```

Example:

```
SELECT name, salary,  
    SUM(salary) OVER(  
        ORDER BY salary  
        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS running_total  
FROM employees;
```

Output:

name	salary	running_total
Alice	49000	49000
Carol	55000	104000
Bob	72000	176000

Key Points:

- Available in MySQL 8.0+
- RANGE treats rows with the same value as one group; ROWS treats each row individually
- Default frame when ORDER BY is used is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

5.05 UNBOUNDED PRECEDING

Category: Window Functions

What it does: Refers to the very first row of the partition. Used in ROWS/RANGE BETWEEN to start the frame from the beginning of the partition.

Syntax:

```
FUNCTION() OVER(  
    ORDER BY column  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
)
```

Example:

```
SELECT name, salary,  
    SUM(salary) OVER(  
        ORDER BY salary  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS cumulative_salary  
FROM employees;
```

Output:

name	salary	cumulative_salary
Alice	49000	49000
Carol	55000	104000
Bob	72000	176000

Key Points:

- Available in MySQL 8.0+
- Most common use: running totals / cumulative sums
- Always used in combination with ROWS BETWEEN or RANGE BETWEEN

5.06 CURRENT ROW

Category: Window Functions

What it does: Refers to the current row being processed in the window frame. Used in ROWS/RANGE BETWEEN as either the start or end of the frame.

Syntax:

```
FUNCTION() OVER(  
    ORDER BY column  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
)
```

Example:

```
SELECT name, salary,  
    MAX(salary) OVER(  
        ORDER BY salary  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS max_so_far  
FROM employees;
```

Output:

name	salary	max_so_far
Alice	49000	49000
Carol	55000	55000
Bob	72000	72000

Key Points:

- Available in MySQL 8.0+
- Default end of frame when ORDER BY is specified
- Can also be used as the start → ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

5.07 UNBOUNDED FOLLOWING

Category: Window Functions

What it does: Refers to the very last row of the partition. Used in ROWS/RANGE BETWEEN to extend the frame to the end of the partition.

Syntax:

```
FUNCTION() OVER(  
    ORDER BY column  
    ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING  
)
```

Example:

```
SELECT name, salary,  
    SUM(salary) OVER(  
        ORDER BY salary  
        ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING  
    ) AS salary_from_here_to_end  
FROM employees;
```

Output:

name	salary	salary_from_here_to_end
Alice	49000	176000
Carol	55000	127000
Bob	72000	72000

Key Points:

- Available in MySQL 8.0+
- Useful for reverse cumulative sums
- Always used inside ROWS BETWEEN or RANGE BETWEEN

6. DATA MANIPULATION (DML) CLAUSES

6.01 INSERT INTO

Category: DML

What it does: Adds one or more new rows into a table.

Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO employees (name, department, salary)
VALUES ("Alice", "HR", 55000);
```

Key Points:

- Column list is optional if you provide values for all columns in order
- Can insert multiple rows → VALUES (row1), (row2), (row3)
- If a required column is missing, MySQL throws an error unless it has a default value

6.02 VALUES

Category: DML

What it does: Provides the actual data to be inserted into the table. Used together with INSERT INTO.

Syntax:

```
INSERT INTO table_name (columns)
VALUES (val1, val2), (val3, val4);
```

Example:

```
INSERT INTO employees (name, department, salary
VALUES
    ("Bob",    "Finance", 72000),
    ("Carol",  "IT",      68000),
    ("Dave",   "IT",      65000);
```

Key Points:

- Multiple rows in one INSERT is faster than separate INSERT statements
- String values must be in quotes; numbers do not need quotes
- Values must match the column order and data types

6.03 SET

Category: DML

What it does: Used in UPDATE to assign new values to one or more columns. Only the specified columns are changed; others remain untouched.

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Example:

```
UPDATE employees
SET salary = 70000,
    department = "Finance"
WHERE name = "Alice";
```

Key Points:

- Always use with WHERE – without it, ALL rows are updated
- Can update multiple columns in one statement by separating with commas
- Can reference the current value → SET salary = salary * 1.10

6.04 UPDATE

Category: DML

What it does: Modifies existing rows in a table. Always combined with SET to specify what to change and WHERE to specify which rows.

Syntax:

```
UPDATE table_name  
SET column = value  
WHERE condition;
```

Example:

```
UPDATE employees  
SET salary = salary * 1.10  
WHERE department = "IT";
```

Key Points:

- Without WHERE, all rows are updated – be very careful
- Can update multiple tables in one statement using JOIN
- Use SELECT first to verify which rows match your WHERE condition before updating

6.05 DELETE

Category: DML

What it does: Removes rows from a table that match the WHERE condition. Without WHERE, all rows are deleted.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

Example:

```
DELETE FROM employees  
WHERE department = "Temp"  
AND salary < 30000;
```

Key Points:

- Without WHERE, all rows are deleted (but the table structure remains)
- Use TRUNCATE TABLE if you want to delete all rows faster
- Deleted rows can be recovered with ROLLBACK if inside a transaction

6.06 REPLACE INTO

Category: DML

What it does: MySQL-specific clause. Inserts a new row, but if a row with the same PRIMARY KEY or UNIQUE KEY exists, it deletes the old row and inserts the new one.

Syntax:

```
REPLACE INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example:

```
REPLACE INTO employees (id, name, salary)
VALUES (5, "Alice", 60000);
```

Key Points:

- If id=5 already exists, it is deleted and re-inserted with new values
- Different from UPDATE – REPLACE deletes then inserts; auto-increment ID changes
- If no duplicate key exists, it behaves exactly like INSERT

6.07 ON DUPLICATE KEY UPDATE

Category: DML

What it does: MySQL-specific clause. When an INSERT would cause a duplicate PRIMARY KEY or UNIQUE KEY, it updates the existing row instead of throwing an error.

Syntax:

```
INSERT INTO table_name (columns)
VALUES (values)
ON DUPLICATE KEY UPDATE column = value;
```

Example:

```
INSERT INTO employees (id, name, salary)
VALUES (5, "Alice", 60000)
ON DUPLICATE KEY UPDATE salary = 60000;
```

Key Points:

- Unlike REPLACE INTO, the original row is updated (not deleted and re-inserted)
- Auto-increment ID stays the same
- Use VALUES(column) to reference the attempted insert value →
ON DUPLICATE KEY UPDATE salary = VALUES(salary)

6.08 IGNORE

Category: DML

What it does: Tells MySQL to skip rows that would cause errors (like duplicate keys or type mismatches) and continue processing the rest, instead of stopping with an error.

Syntax:

```
INSERT IGNORE INTO table_name (columns)
VALUES (values);
```

Example:

```
INSERT IGNORE INTO employees (id, name, salary)
VALUES (5, "Alice", 60000);
```

//If id=5 already exists, this row is silently skipped//

Key Points:

- Works with INSERT, UPDATE, DELETE
- Errors are converted to warnings – the query continues
- Use carefully – it can hide real data problems

7. DATA DEFINITION (DDL) CLAUSES

7.01 CREATE TABLE

Category: DDL

What it does: Creates a new table in the database with defined columns, data types, and constraints.

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints  
);
```

Example:

```
CREATE TABLE employees (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100) NOT NULL,  
    department    VARCHAR(50),  
    salary        DECIMAL(10, 2),  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Key Points:

- Use IF NOT EXISTS to avoid errors if table already exists
- Define PRIMARY KEY to uniquely identify each row
- Choose appropriate data types for each column

7.02 ALTER TABLE

Category: DDL

What it does: Modifies an existing table structure – add, remove, or change columns, indexes, and constraints.

Syntax:

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Example:

```
//Add a new column//
```

```
ALTER TABLE employees ADD email VARCHAR(100);
```

```
//Change a column's data type//
```

```
ALTER TABLE employees MODIFY salary DECIMAL(12, 2);
```

```
//Drop a column//
```

```
ALTER TABLE employees DROP COLUMN email;
```

Key Points:

- Can ADD, MODIFY, DROP, RENAME columns
- Can ADD or DROP indexes and constraints
- On large tables, ALTER TABLE can be slow as it may rebuild the entire table

7.03 DROP TABLE

Category: DDL

What it does: Permanently deletes a table and all of its data from the database. This cannot be undone.

Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE IF EXISTS old_employees;
```

Key Points:

- Use IF EXISTS to avoid errors if the table does not exist
- Completely removes the table structure and all data – no recovery without backup
- Cannot DROP a table that is referenced by a FOREIGN KEY constraint

7.04 TRUNCATE TABLE

Category: DDL

What it does: Deletes all rows from a table **instantly** by dropping and recreating the table. Much faster than DELETE for clearing all rows.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE temp_logs;
```

Key Points:

- Cannot be used with WHERE – it always removes all rows
- Resets AUTO_INCREMENT counter back to 1
- Cannot be rolled back in most MySQL storage engines

7.05 RENAME TABLE

Category: DDL

What it does: Renames one or more tables. Can also be used to move a table from one database to another.

Syntax:

```
RENAME TABLE old_name TO new_name;
```

Example:

```
RENAME TABLE employees TO staff;
```

```
//Rename multiple tables at once//
```

```
RENAME TABLE employees TO staff,  
                departments TO divisions;
```

Key Points:

- Atomic operation – all renames succeed or all fail
- Views, stored procedures referencing the old name will break
- Can move between databases → `RENAME TABLE db1.table1 TO db2.table1`

7.06 CREATE INDEX

Category: DDL

What it does: Creates an index on a table column to speed up queries that search or filter on that column.

Syntax:

```
CREATE INDEX index_name  
ON table_name (column);
```

Example:

```
CREATE INDEX idx_department  
ON employees (department);
```

```
//Unique index//
```

```
CREATE UNIQUE INDEX idx_email  
ON employees (email);
```

Key Points:

- Speeds up SELECT queries but slightly slows down INSERT/UPDATE/DELETE
- PRIMARY KEY and UNIQUE constraints automatically create indexes
- Use on columns frequently used in WHERE, JOIN, and ORDER BY

7.07 DROP INDEX

Category: DDL

What it does: Removes an existing index from a table.

Syntax:

```
DROP INDEX index_name ON table_name;
```

Example:

```
DROP INDEX idx_department ON employees;
```

Key Points:

- Dropping an index does not affect the data in the table
- Cannot drop the PRIMARY KEY index this way – use ALTER TABLE
- Use SHOW INDEX FROM table_name to see existing indexes

7.08 CREATE VIEW

Category: DDL

What it does: Creates a virtual table based on a SELECT query. The view does not store data – it runs the query every time it is accessed.

Syntax:

```
CREATE VIEW view_name AS
SELECT columns
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW high_earners AS
SELECT name, department, salary
FROM employees
WHERE salary > 70000;
```

```
//Use it like a table//
```

```
SELECT * FROM high_earners;
```

Key Points:

- Simplifies complex queries – query the view instead of writing the full SQL each time
- Useful for security – grant access to a view instead of the full table
- Changes to the base table are automatically reflected in the view

7.09 DROP VIEW

Category: DDL

What it does: Permanently removes a view from the database.

Syntax:

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW IF EXISTS high_earners;
```

Key Points:

- Only removes the view definition – the underlying table data is unaffected
- Use IF EXISTS to avoid errors if the view does not exist
- Any queries or procedures using the view will break after it is dropped

7.10 CREATE DATABASE

Category: DDL

What it does: Creates a new database (schema) in MySQL.

Syntax:

```
CREATE DATABASE database_name;
```

Example:

```
CREATE DATABASE IF NOT EXISTS company_db  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

Key Points:

- Use IF NOT EXISTS to prevent errors if database already exists
- Always specify CHARACTER SET utf8mb4 for full Unicode support (including emojis)
- After creation, use database_name to start working in it

7.11 DROP DATABASE

Category: DDL

What it does: Permanently deletes an entire database, including all tables, views, and data inside it.

Syntax:

```
DROP DATABASE database_name;
```

Example:

```
DROP DATABASE IF EXISTS old_company_db;
```

Key Points:

- Extremely destructive – all data is permanently gone without a backup
- Use IF EXISTS to avoid errors if database does not exist
- Cannot be rolled back

8. CONDITIONAL CLAUSE

8.01 CASE WHEN

Category: Conditional

What it does: Adds conditional logic inside a query. Works like an if-else – evaluates conditions in order and returns the first matching result.

Syntax:

CASE

```
WHEN condition1 THEN result1
WHEN condition2 THEN result2
ELSE default_result
```

END;

Example:

```
SELECT name, salary,
CASE
    WHEN salary > 80000 THEN 'Senior'
    WHEN salary > 55000 THEN 'Mid'
    ELSE "Junior"
END AS level
FROM employees;
```

Output:

name	salary	level
Alice	55000	Junior
Bob	72000	Mid
Frank	85000	Senior

Key Points:

- Conditions are checked in order – first match wins
- ELSE is optional; if omitted and no condition matches, returns NULL
- Can be used in SELECT, WHERE, ORDER BY, GROUP BY, and HAVING

9. SUBQUERY CLAUSES

9.01 FROM (SUBQUERY)

Category: Subquery

What it does: Uses a SELECT query as a temporary table inside FROM. The result of the inner query is treated like a table.

Syntax:

```
SELECT columns
FROM (SELECT ... FROM ...) AS alias;
```

Example:

```
SELECT dept, avg_salary
FROM (
    SELECT department AS dept,
           AVG(salary) AS avg_salary

    FROM employees
    GROUP BY department
) AS dept_summary
WHERE avg_salary > 60000;
```

Output:

dept	avg_salary
Finance	71000
IT	67000

Key Points:

- The subquery alias (e.g. dept_summary) is mandatory in MySQL
- Runs the inner query first, then the outer query operates on its result
- Also called a "derived table"

9.02 WHERE (SUBQUERY)

Category: Subquery

What it does: Uses a SELECT query inside a WHERE condition to filter rows based on the result of another query.

Syntax:

```
SELECT columns
FROM table_name
WHERE column IN (SELECT column FROM other_table);
```

Example:

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Output:

name	salary
Bob	72000
Frank	85000

Key Points:

- The subquery runs first, then its result is used by the outer WHERE
- Can use IN, NOT IN, =, >, <, EXISTS, ANY, ALL with subqueries
- A subquery returning one value can use = ; for multiple values use IN

FUNCTIONS

MySQL Functions

1. STRING FUNCTIONS

1.01 CONCAT()

What it does: Joins two or more strings into one string.

Process: Takes all given strings → attaches them one after another → returns single combined string.

```
SELECT CONCAT("John", " ", "Doe");  
//result: John Doe//  
  
SELECT CONCAT(first_name, " ", last_name)  
AS full_name FROM users;  
//joins first and last name//
```

1.02 LENGTH()

What it does: Returns the length of a string in BYTES.

Process: Counts number of bytes in string → returns count as number.

```
SELECT LENGTH("hello");  
//result: 5//  
  
SELECT LENGTH("hello world");  
//result: 11//  
  
//Real use: find weak passwords//  
SELECT * FROM users AS weak_password_ids  
WHERE LENGTH(password) < 8;
```

1.03 CHAR_LENGTH()

What it does: Returns the length of a string in CHARACTERS – differs from LENGTH for special characters.

Process: Counts number of characters → returns count as number.

```
SELECT CHAR_LENGTH("hello");  
//result: 5//  
  
SELECT CHAR_LENGTH("café");  
//result: 4 (4 characters)//
```

```
SELECT LENGTH("café");  
//result: 5 (5 bytes – special char uses 2 bytes)//
```

1.04 UPPER()

What it does: Converts all letters in a string to UPPERCASE.

Process: Goes through each character → converts lowercase letters to uppercase → returns new string.

```
SELECT UPPER("hello");  
//result: HELLO//  
  
SELECT UPPER("John Doe");  
//result: JOHN DOE//  
  
SELECT UPPER(name) FROM users;  
//converts all names to uppercase//
```

1.05 LOWER()

What it does: Converts all letters in a string to LOWERCASE.

Process: Goes through each character → converts uppercase letters to lowercase → returns new string.

```
SELECT LOWER("HELLO");  
//result: hello//  
  
SELECT LOWER("John Doe");  
//result: john doe//  
  
SELECT LOWER(email) FROM users;  
//converts all emails to lowercase//
```

1.06 TRIM()

What it does: Removes spaces from BOTH sides of a string.

Process: Checks left side → removes spaces → checks right side → removes spaces → returns cleaned string.

```
SELECT TRIM(" hello ");  
//result: 'hello'//  
  
SELECT TRIM(" John Doe ");  
//result: 'John Doe'//
```

```
UPDATE users SET name = TRIM(name);  
//clean up all names in table//
```

1.07 LTRIM()

What it does: Removes spaces from LEFT side only.

Process: Checks left side → removes spaces → returns string with clean left side.

```
SELECT LTRIM("  hello ");  
//result: 'hello '  
  
//left spaces removed, right spaces kept//
```

1.08 RTRIM()

What it does: Removes spaces from RIGHT side only.

Process: Checks right side → removes spaces → returns string with clean right side.

```
SELECT RTRIM("  hello ");  
//result: '  hello'  
  
//right spaces removed, left spaces kept//
```

1.09 SUBSTRING()

What it does: Extracts a part of a string from a given position.

Process: Goes to given start position → extracts given number of characters → returns extracted part.

```
SELECT SUBSTRING("hello world", 1, 5);  
//result: hello//  
//starts at position 1, takes 5 characters//  
  
SELECT SUBSTRING("hello world", 7);  
//result: world//  
//starts at position 7, takes rest of string//  
  
SELECT SUBSTRING(name, 1, 3) FROM users;  
//first 3 characters of name//
```

1.10 LEFT()

What it does: Extracts N characters from the LEFT side of a string.

Process: Starts from left → takes given number of characters → returns extracted part.

```
SELECT LEFT("hello world", 5);  
//result: hello //  
  
SELECT LEFT(name, 1) FROM users;  
//first letter of each name//
```

1.11 RIGHT()

What it does: Extracts N characters from the RIGHT side of a string.

Process: Starts from right → takes given number of characters → returns extracted part.

```
SELECT RIGHT("hello world", 5);  
//result: world//  
  
SELECT RIGHT(phone, 4) FROM users;  
//last 4 digits of phone//
```

1.12 REPLACE()

What it does: Replaces all occurrences of a substring with another substring.

Process: Searches string for given substring → replaces every occurrence → returns new string.

```
SELECT REPLACE("hello world", "world", "MySQL");  
//result: hello MySQL//  
  
SELECT REPLACE(phone, "-", "") FROM users;  
//removes all dashes from phone//  
  
UPDATE users SET email = REPLACE(email, "@old.com", "@new.com");
```

1.13 REVERSE()

What it does: Reverses the characters of a string.

Process: Takes last character → places first → continues until all reversed → returns reversed string.

```
SELECT REVERSE("hello");  
//result: olleh//  
  
SELECT REVERSE("MySQL");  
//result: LQSyM//
```

1.14 REPEAT()

What it does: Repeats a string a given number of times.

Process: Takes string → copies it N times → joins copies together → returns result.

```
SELECT REPEAT("ab", 3);  
//result: ababab//  
  
SELECT REPEAT("*", 5);  
//result: *****//  
  
SELECT REPEAT("-", 10);  
//result: -----//  
  
//Real use: create separator lines//  
  
SELECT REPEAT("=", 20);  
//result: =====//
```

1.15 INSTR()

What it does: Finds the position where a substring first appears in a string.

Process: Searches string from left → finds first match → returns position number (starts at 1), returns 0 if not found.

```
SELECT INSTR("hello world", "world");  
//result: 7 (world starts at position 7)//  
  
SELECT INSTR("hello", "xyz");  
//result: 0 (not found)//
```

1.16 LPAD()

What it does: Pads a string on the LEFT side to reach a given length.

Process: Checks current length → adds given pad character to left side → repeats until desired length reached.

```
SELECT LPAD("5", 4, "0");
//result: 0005 (padded with zeros on left)//

SELECT LPAD("hi", 5, "-");
//result: ---hi//

SELECT LPAD(id, 6, '0') FROM orders;
//formats IDs as 000001, 000002//
```

1.17 RPAD()

What it does: Pads a string on the RIGHT side to reach a given length.

Process: Checks current length → adds given pad character to right side → repeats until desired length reached.

```
SELECT RPAD("5", 4, "0");
//result: 5000//

SELECT RPAD("hi", 5, "-");
//result: hi---//
```

1.18 SPACE()

What it does: Returns a string of N space characters.

Process: Creates a string with given number of spaces → returns it.

```
SELECT SPACE(5);
//result: '     ' (5 spaces)//

SELECT CONCAT("Hello", SPACE(3), "World");
//result: Hello   World//

//Real use: formatting output//

SELECT CONCAT(first_name, SPACE(1), last_name) AS full_name
FROM users;
```


1.19 FORMAT()

What it does: Formats a number with commas as thousands separators and rounds to given decimal places.

Process: Takes number → adds comma separators every 3 digits → rounds to given decimals → returns formatted string.

```
SELECT FORMAT(1234567.891, 2);
```

```
//result: 1,234,567.89//
```

```
SELECT FORMAT(1000000, 0);
```

```
//result: 1,000,000//
```

```
SELECT FORMAT(9.5, 2);
```

```
//result: 9.50//
```

```
//Real use: display prices nicely//
```

```
SELECT FORMAT(salary, 2) AS formatted_salary FROM employees;
```

```
//result: 5,000.00 instead of 5000//
```

2. NUMERIC FUNCTIONS

2.01 ABS()

What it does: Returns the absolute (positive) value of a number – removes negative sign.

Process: Checks if number is negative → removes negative sign → returns positive value.

```
SELECT ABS(-5);  
//result: 5//  
  
SELECT ABS(5);  
//result: 5//  
  
SELECT ABS(-99.9);  
//result: 99.9//  
  
SELECT ABS(salary - 5000) FROM employees;  
//distance from 5000 salary//
```

2.02 CEIL() / CEILING()

What it does: Rounds a decimal number UP to nearest whole number – always goes up.

Process: Checks decimal part → if any decimal exists → rounds up to next whole number.

```
SELECT CEIL(4.1);  
//result: 5//  
  
SELECT CEIL(4.9);  
//result: 5//  
  
SELECT CEIL(4.0);  
//result: 4//  
  
SELECT CEIL(-4.1);  
//result: -4 (goes up toward zero)//
```

2.03 FLOOR()

What it does: Rounds a decimal number DOWN to nearest whole number – always goes down.

Process: Checks decimal part → removes decimal → returns lower whole number.

```
SELECT FLOOR(4.9);  
//result: 4//  
  
SELECT FLOOR(4.1);  
//result: 4//  
  
SELECT FLOOR(-4.1);  
//result: -5 (goes down away from zero)//
```

2.04 ROUND()

What it does: Rounds a number to given decimal places – standard rounding rules apply.

Process: Checks digit after desired decimal place → if 5 or above rounds up → if below 5 rounds down.

```
SELECT ROUND(4.5);  
//result: 5//  
  
SELECT ROUND(4.4);  
//result: 4//  
  
SELECT ROUND(4.567, 2);  
//result: 4.57 (rounds to 2 decimal places)//  
  
SELECT ROUND(price, 2) FROM orders;  
//rounds price to 2 decimals//
```

2.05 TRUNCATE()

What it does: Cuts off decimal places without rounding – just removes extra decimals.

Process: Goes to desired decimal position → cuts off everything after it → returns shortened number.

```
SELECT TRUNCATE(4.999, 2);  
//result: 4.99 (just cuts, no rounding)//
```

```
SELECT TRUNCATE(4.999, 0);  
//result: 4//  
  
SELECT ROUND(4.999, 2);  
//result: 5.00 (rounds up)//  
  
//difference: TRUNCATE cuts, ROUND rounds//
```

2.06 SQRT()

What it does: Returns the square root of a number.

Process: Calculates which number multiplied by itself gives the input → returns that number.

```
SELECT SQRT(16);  
//result: 4 (4 * 4 = 16)//  
  
SELECT SQRT(25);  
//result: 5 (5 * 5 = 25)//  
  
SELECT SQRT(2);  
//result: 1.4142//
```

2.07 POW() / POWER()

What it does: Raises a number to a given power (exponent).

Process: Multiplies number by itself the given number of times → returns result.

```
SELECT POW(2, 3);  
//result: 8 (2 * 2 * 2)//  
  
SELECT POW(5, 2);  
//result: 25 (5 * 5)//  
  
SELECT POWER(10, 3);  
//result: 1000//
```

2.08 RAND()

What it does: Returns a random decimal number between 0 and 1.

Process: Generates random number → returns value between 0.0 and 1.0 → different every time.

```
SELECT RAND();  
//result: 0.7832 (random each time)//  
  
SELECT RAND() * 100;  
//random number between 0 and 100//  
  
SELECT * FROM users ORDER BY RAND() LIMIT 5;  
//get 5 random users//
```

2.09 MOD() FUNCTION

What it does: Returns the remainder of dividing two numbers – function version of % operator.

Process: Divides first number by second → returns only the remainder.

```
SELECT MOD(10, 3);  
//result: 1//  
  
SELECT MOD(10, 2);  
//result: 0 (even number)//  
  
SELECT MOD(11, 2);  
//result: 1 (odd number)//  
  
//Real use: find even rows//  
SELECT * FROM orders WHERE MOD(order_id, 2) = 0;
```

3. DATE AND TIME FUNCTIONS

3.01 NOW()

What it does: Returns the CURRENT date and time at the moment query runs.

Process: Reads system clock → returns date and time together.

```
SELECT NOW();  
//result: 2024-01-15 10:30:45//  
  
INSERT INTO orders (created_at) VALUES (NOW());  
//save current time//
```

3.02 CURDATE()

What it does: Returns only the CURRENT DATE – no time.

Process: Reads system clock → returns date part only.

```
SELECT CURDATE();  
//result: 2024-01-15//  
  
SELECT * FROM orders WHERE DATE(created_at) = CURDATE();  
//today's orders//
```

3.03 CURTIME()

What it does: Returns only the CURRENT TIME – no date.

Process: Reads system clock → returns time part only.

```
SELECT CURTIME();  
//result: 10:30:45//
```

3.04 SYSDATE()

What it does: Returns current date and time – similar to NOW() but with one key difference.

Process: Reads system clock at the EXACT moment function is called → returns date and time.

```
SELECT SYSDATE();  
//result: 2024-01-15 10:30:45//
```

```
//Difference between NOW() and SYSDATE()//

SELECT NOW(), SLEEP(2), NOW();
//NOW() returns SAME time both times//

SELECT SYSDATE(), SLEEP(2), SYSDATE();
//SYSDATE() returns DIFFERENT time (2 sec later)//

//NOW() is fixed at query start time//

//SYSDATE() updates every time it is called//
```

3.05 DATE()

What it does: Extracts only the DATE part from a datetime value – removes time part.

Process: Takes datetime value → keeps only date portion → removes time portion → returns date only.

```
SELECT DATE('2024-01-15 10:30:45');
//result: 2024-01-15//

SELECT DATE(NOW());
//result: today's date only//

//Real use: compare dates ignoring time//

SELECT * FROM orders WHERE DATE(created_at) = '2024-01-15';
//finds all orders on that date regardless of time//
```

3.06 TIME()

What it does: Extracts only the TIME part from a datetime value – removes date part.

Process: Takes datetime value → keeps only time portion → removes date portion → returns time only.

```
SELECT TIME('2024-01-15 10:30:45');
//result: 10:30:45//

SELECT TIME(NOW());
//result: current time only//
```

```
//Real use: find orders placed in morning//  
  
SELECT * FROM orders WHERE TIME(created_at)  
BETWEEN '08:00:00' AND '12:00:00';
```

3.07 YEAR() MONTH() DAY()

What it does: Extracts specific part from a date.

Process: Reads date value → extracts requested part → returns as number.

```
SELECT YEAR('2024-01-15');  
//result: 2024//  
  
SELECT MONTH('2024-01-15');  
//result: 1//  
  
SELECT DAY('2024-01-15');  
//result: 15//  
  
//Real use: group by year//  
  
SELECT YEAR(created_at), COUNT(*) FROM orders  
GROUP BY YEAR(created_at);
```

3.08 HOUR()

What it does: Extracts the HOUR part from a time or datetime value.

Process: Takes time or datetime → reads hour value → returns it as number (0-23).

```
SELECT HOUR('10:30:45');  
//result: 10//  
  
SELECT HOUR(NOW());  
//result: current hour//  
  
//Real use: find orders by hour//  
  
SELECT HOUR(created_at) AS hour, COUNT(*) AS total  
FROM orders  
GROUP BY HOUR(created_at);  
  
//shows how many orders per hour of day//
```


3.09 MINUTE()

What it does: Extracts the MINUTE part from a time or datetime value.

Process: Takes time or datetime → reads minute value → returns it as number (0-59).

```
SELECT MINUTE('10:30:45');  
//result: 30//  
  
SELECT MINUTE(NOW());  
//result: current minute//  
  
//Real use//  
SELECT * FROM orders WHERE MINUTE(created_at) = 0;  
//orders placed at exactly the hour mark//
```

3.10 SECOND()

What it does: Extracts the SECOND part from a time or datetime value.

Process: Takes time or datetime → reads second value → returns it as number (0-59).

```
SELECT SECOND('10:30:45');  
//result: 45//  
  
SELECT SECOND(NOW());  
//result: current second//
```

3.11 DATEDIFF()

What it does: Returns the number of DAYS between two dates.

Process: Subtracts second date from first date → returns number of days difference.

```
SELECT DATEDIFF('2024-01-15', '2024-01-01');  
//result: 14 (14 days apart)//  
  
SELECT DATEDIFF(NOW(), created_at) AS days_ago FROM orders;  
//how old is order//
```

3.12 DATE_ADD()

What it does: Adds a time interval to a date – returns new date.

Process: Takes given date → adds given interval → returns resulting date.

```
SELECT DATE_ADD('2024-01-01', INTERVAL 7 DAY);  
//result: 2024-01-08//  
  
SELECT DATE_ADD('2024-01-01', INTERVAL 1 MONTH);  
//result: 2024-02-01//  
  
SELECT DATE_ADD('2024-01-01', INTERVAL 1 YEAR);  
//result: 2025-01-01//
```

3.13 DATE_SUB()

What it does: Subtracts a time interval from a date – returns new date.

Process: Takes given date → subtracts given interval → returns resulting date.

```
SELECT DATE_SUB('2024-01-15', INTERVAL 7 DAY);  
//result: 2024-01-08//  
  
SELECT DATE_SUB(NOW(), INTERVAL 1 MONTH);  
//one month ago//
```

3.14 DATE_FORMAT()

What it does: Formats a date into a custom display format.

Process: Reads date → replaces format codes with actual date parts → returns formatted string.

```
SELECT DATE_FORMAT('2024-01-15', '%d/%m/%Y');  
//result: 15/01/2024//  
  
SELECT DATE_FORMAT(NOW(), '%M %d, %Y');  
//result: January 15, 2024//  
  
SELECT DATE_FORMAT(created_at, '%d-%m-%Y') FROM orders;
```

3.15 STR_TO_DATE()

What it does: Converts a string into a proper DATE value using a given format.

Process: Takes string → reads it according to given format → converts to proper date value → returns date.

```
SELECT STR_TO_DATE('15/01/2024', '%d/%m/%Y');
//result: 2024-01-15//

SELECT STR_TO_DATE('January 15 2024', '%M %d %Y');
//result: 2024-01-15//

SELECT STR_TO_DATE('15-Jan-24', '%d-%b-%y');
//result: 2024-01-15//

//Real use: convert user entered dates//

INSERT INTO orders (order_date)
VALUES (STR_TO_DATE('15/01/2024', '%d/%m/%Y'));
```

3.16 DAYNAME()

What it does: Returns the name of the day of the week for a given date.

Process: Takes date → calculates which day of week it falls on → returns day name as string.

```
SELECT DAYNAME('2024-01-15');
//result: Monday//

SELECT DAYNAME(NOW());
//result: current day name//

SELECT DAYNAME('2024-12-25');
//result: Wednesday//

//Real use: find all Monday orders//

SELECT * FROM orders WHERE DAYNAME(created_at) = 'Monday';

//Group sales by day of week//

SELECT DAYNAME(created_at) AS day, COUNT(*) AS total
FROM orders
GROUP BY DAYNAME(created_at);
```

3.17 MONTHNAME()

What it does: Returns the name of the month for a given date.

Process: Takes date → reads month number → returns full month name as string.

```
SELECT MONTHNAME('2024-01-15');  
//result: January//  
  
SELECT MONTHNAME('2024-12-25');  
//result: December//  
  
SELECT MONTHNAME(NOW());  
//result: current month name//  
  
//Real use: group sales by month name//  
  
SELECT MONTHNAME(created_at) AS month, SUM(price) AS total  
FROM orders  
GROUP BY MONTHNAME(created_at);
```

4. NULL FUNCTIONS

4.01 IFNULL()

What it does: Returns a replacement value if the given value is NULL – otherwise returns the value itself.

Process: Checks if value is NULL → if NULL returns replacement → if not NULL returns original value.

```
SELECT IFNULL(email, 'no email') FROM users;

//if email is NULL → shows 'no email'//

//if email has value → shows the email//

SELECT name, IFNULL(phone, 'not provided') FROM users;
```

4.02 NULLIF()

What it does: Returns NULL if two values are EQUAL – otherwise returns first value.

Process: Compares two values → if equal returns NULL → if not equal returns first value.

```
SELECT NULLIF(5, 5);
//result: NULL (both equal)//

SELECT NULLIF(5, 3);
//result: 5 (not equal, returns first)//

//Real use: avoid division by zero//

SELECT name, total / NULLIF(quantity, 0) FROM orders;

//if quantity is 0 → NULLIF returns NULL → division returns
NULL (no error)//

//if quantity > 0 → NULLIF returns quantity → normal
division//
```

4.03 COALESCE()

What it does: Returns the FIRST non-NULL value from a list of values in a row.

Process: Checks first value → if NULL moves to next → continues until non-NULL found → returns that value.

```
SELECT COALESCE(NULL, NULL, 'hello');  
//result: hello (first non-NULL)//  
  
SELECT COALESCE(NULL, 'second', 'third');  
//result: second//  
  
//Real use: fallback contact info//  
  
SELECT name, COALESCE(email, phone, address, 'no contact')  
FROM users;  
  
//shows email if exists, if not shows phone, if not shows  
address, if not shows 'no contact'//
```

4.04 ISNULL()

What it does: Checks if a value is NULL – returns 1 for TRUE, 0 for FALSE.

Process: Checks if value is NULL → returns 1 if NULL → returns 0 if not NULL.

```
SELECT ISNULL(NULL);  
// result: 1 (is NULL)//  
  
SELECT ISNULL('hello');  
//result: 0 (not NULL)//  
  
SELECT ISNULL(email) FROM users;  
//1 for users with no email, 0 for users with email//
```

5. CONTROL FLOW FUNCTIONS

5.01 IF()

What it does: Returns one value if condition is TRUE and another value if FALSE – like an if-else.

Process: Evaluates condition → if TRUE returns second argument → if FALSE returns third argument.

```
SELECT IF(age >= 18, 'adult', 'minor') FROM users;
```

```
//age is 20 → returns 'adult'//
```

```
//age is 15 → returns 'minor'//
```

```
SELECT name, IF(salary > 5000, 'high', 'low') AS salary_level  
FROM employees;
```

5.02 CASE

What it does: Checks multiple conditions one by one and returns value for first matching condition.

Process: Evaluates first WHEN condition → if TRUE returns THEN value → if FALSE moves to next WHEN → if none match returns ELSE value.

```
SELECT name,  
       CASE  
         WHEN age < 13 THEN 'child'  
         WHEN age BETWEEN 13 AND 17 THEN 'teen'  
         WHEN age BETWEEN 18 AND 60 THEN 'adult'  
         ELSE 'senior'  
       END AS age_group  
FROM users;  
//checks each WHEN top to bottom//  
  
//returns first matching THEN value//  
  
//returns ELSE if nothing matches//
```

6. AGGREGATE FUNCTIONS

6.01 COUNT()

What it does: Counts the number of rows – or non-NULL values in a column.

Process: Goes through each row → counts matching rows → returns total count.

```
SELECT COUNT(*) FROM users;
//counts ALL rows//

SELECT COUNT(email) FROM users;
//counts only rows where email is NOT NULL//

SELECT COUNT(DISTINCT country) FROM users;
//counts unique countries//
```

6.02 SUM()

What it does: Adds up all values in a column – ignores NULL values.

Process: Goes through each row → adds value to running total → returns final total.

```
SELECT SUM(salary) FROM employees;
//total of all salaries//

SELECT SUM(price * quantity) AS revenue FROM orders;
//total revenue//
```

6.03 AVG()

What it does: Calculates the average of all values in a column – ignores NULL values.

Process: Adds all values → divides by count of non-NULL values → returns average.

```
SELECT AVG(salary) FROM employees;
//average salary//

SELECT AVG(price) FROM orders;
//average order price//
```


6.04 MIN()

What it does: Returns the SMALLEST value in a column.

Process: Goes through all values → keeps track of smallest → returns smallest found.

```
SELECT MIN(salary) FROM employees;
//lowest salary//

SELECT MIN(price) FROM orders;
//cheapest order//

SELECT MIN(created_at) FROM orders;
//oldest order date//
```

6.05 MAX()

What it does: Returns the LARGEST value in a column.

Process: Goes through all values → keeps track of largest → returns largest found.

```
SELECT MAX(salary) FROM employees;
//highest salary//

SELECT MAX(price) FROM orders;
//most expensive order//

SELECT MAX(created_at) FROM orders;
//most recent order date//
```

6.06 GROUP_CONCAT()

What it does: Joins all values in a group into one comma-separated string.

Process: Goes through grouped rows → collects values → joins them with separator → returns single string.

```
SELECT user_id, GROUP_CONCAT(product_name) AS products
FROM orders
GROUP BY user_id;

//result: user 1 → 'Apple,Banana,Cherry'//

//all products of each user joined in one string//
```

```
SELECT GROUP_CONCAT(name ORDER BY name SEPARATOR ' | ')  
FROM users;
```

```
//result: Alice | Bob | Charlie//
```

7. TYPE CONVERSION FUNCTIONS

7.01 CAST()

What it does: Converts a value from one data type to another.

Process: Takes given value → converts to specified type → returns converted value.

```
SELECT CAST('123' AS UNSIGNED);  
//string to number: result 123//  
  
SELECT CAST(123 AS CHAR);  
//number to string: result '123'//  
  
SELECT CAST('2024-01-15' AS DATE);  
//string to date//  
  
SELECT CAST(9.99 AS UNSIGNED);  
//decimal to integer: result 9//
```

7.02 CONVERT()

What it does: Converts a value to a different data type or character set – similar to CAST.

Process: Takes given value → converts to specified type or charset → returns converted value.

```
SELECT CONVERT('123', UNSIGNED);  
//string to number: result 123//  
  
SELECT CONVERT(123, CHAR);  
//number to string: result '123'//  
  
SELECT CONVERT(name USING utf8mb4) FROM users;  
//convert charset//
```

7.03 BINARY

What it does: Converts a string to binary format – makes string comparisons CASE SENSITIVE.

Process: Takes string → treats it as binary data → every character comparison is now case sensitive.

```
//Without BINARY – case insensitive//
```

```

SELECT 'hello' = 'HELLO';
//result: 1 (TRUE – same)//

SELECT * FROM users WHERE name = 'john';
//returns john, John, JOHN//

//With BINARY – case sensitive//

SELECT BINARY 'hello' = 'HELLO';
//result: 0 (FALSE – different)//

SELECT * FROM users WHERE BINARY name = 'john';
//returns ONLY 'john'//

SELECT * FROM users WHERE BINARY name = 'John';
//returns ONLY 'John'//

//Real use: case sensitive password check//

SELECT * FROM users
WHERE username = 'john'
AND BINARY password = 'MyPass123';

//password must match exact case//

```

9. WINDOW FUNCTIONS (MYSQL 8.0+)

9.01 ROW_NUMBER()

What it does: Assigns a unique sequential number to each row within a partition – no gaps, no ties.

Process: Divides rows into groups (partitions) → numbers each row starting from 1 → every row gets unique number.

```
SELECT name, salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM employees;
```

```
//assigns 1 to highest salary, 2 to next, etc.//
```

```
//With PARTITION – reset number for each department//
```

```
SELECT name, department, salary,  
       ROW_NUMBER() OVER (PARTITION BY department  
                          ORDER BY salary DESC) AS row_num
```

```
FROM employees;
```

```
//row numbers restart from 1 for each department//
```

9.02 RANK()

What it does: Assigns rank to each row – same values get SAME rank but next rank SKIPS numbers (has gaps).

Process: Orders rows → assigns rank → if tie gives same rank → skips next rank number → continues.

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS rank_num  
FROM employees;
```

```
//Example result: //
```

```
//John    10000    rank: 1//
```

```
//Alice   8000     rank: 2//
```

```
//Bob      8000    rank: 2    (tie with Alice)//
//Mary     6000    rank: 4    (skips 3 because of tie)//
```

9.03 DENSE_RANK()

What it does: Assigns rank to each row – same values get SAME rank but next rank does NOT skip (no gaps).

Process: Orders rows → assigns rank → if tie gives same rank → continues with NEXT consecutive rank number.

```
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_num
FROM employees;
```

```
//Example result: //
//John   10000   dense_rank: 1 //
//Alice   8000   dense_rank: 2 //
//Bob     8000   dense_rank: 2  (tie with Alice) //
//Mary    6000   dense_rank: 3  (NO gap – continues from 3) //
```

```
// Difference from RANK: //
// RANK:           1, 2, 2, 4  (gap after tie) //
// DENSE_RANK: 1, 2, 2, 3  (no gap after tie) //
```

9.04 LEAD()

What it does: Accesses the value from the NEXT row – useful to compare current row with next row.

Process: For each row → looks ahead N rows → returns value from that future row → NULL if no next row.

```
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;

//shows current salary and next person's salary side by side//
```

```
//With offset (look 2 rows ahead)//
SELECT name, salary,
       LEAD(salary, 2) OVER (ORDER BY salary) AS salary_2_ahead
FROM employees;
```

```
//Real use: calculate salary difference with next person//
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary) - salary
       AS diff_to_next
FROM employees;
```

9.05 LAG()

What it does: Accesses the value from the PREVIOUS row – useful to compare current row with previous row.

Process: For each row → looks back N rows → returns value from that previous row → NULL if no previous row.

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary
FROM employees;

//shows current salary and previous person's salary side by side//

//Real use: calculate month over month growth//
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS growth
FROM monthly_sales;
```

9.06 FIRST_VALUE()

What it does: Returns the FIRST value in a partition or ordered set.

Process: Orders rows in partition → looks at first row → returns its value for every row in partition.

```
SELECT name, department, salary,  
       FIRST_VALUE(salary) OVER (  
         PARTITION BY department ORDER BY salary DESC  
       ) AS highest_in_dept  
FROM employees;  
  
//shows highest salary in department next to each employee//
```

9.07 LAST_VALUE()

What it does: Returns the LAST value in a partition or ordered set.

Process: Orders rows in partition → looks at last row → returns its value.

```
SELECT name, department, salary,  
       LAST_VALUE(salary) OVER (  
         PARTITION BY department  
         ORDER BY salary DESC  
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
       ) AS lowest_in_dept  
FROM employees;  
  
//shows lowest salary in department next to each employee//
```

9.08 NTILE()

What it does: Divides rows into N equal groups and assigns group number to each row.

Process: Counts total rows → divides into N groups → assigns group number 1 to N to each row.


```

SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees;

//divides employees into 4 equal groups//

//quartile 1 = top 25% earners//

//quartile 4 = bottom 25% earners//


//Real use: divide customers into segments//

SELECT name, total_spent,
       NTILE(3) OVER (ORDER BY total_spent DESC) AS segment
FROM customers;

//segment 1 = top spenders (gold)//

//segment 2 = medium spenders (silver)//

//segment 3 = low spenders (bronze)//

```

9.09 PERCENT_RANK()

What it does: Returns relative rank of a row as a percentage between 0 and 1.

Process: Calculates $(\text{rank} - 1) / (\text{total rows} - 1)$ → returns value between 0.0 and 1.0.

```

SELECT name, salary,
       PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank
FROM employees;

//result: 0.0 for lowest salary, 1.0 for highest salary//

//0.5 means salary is higher than 50% of employees//

//Real use: find top 10% earners//

SELECT * FROM (
    SELECT name, salary,
           PERCENT_RANK() OVER (ORDER BY salary)
           AS pct_rank FROM employees
    ) ranked
WHERE pct_rank >= 0.90;

//returns top 10% earners//

```

9.10 CUME_DIST()

What it does: Returns the cumulative distribution – what proportion of rows have values less than or equal to current row.

Process: For each row → counts rows with value less than or equal to current → divides by total rows → returns proportion.

```
SELECT name, salary,
       CUME_DIST() OVER (ORDER BY salary) AS cume_dist
FROM employees;

//0.25 means 25% of employees earn same or less//
//1.0 means this is the highest earner//

//Real use: find employees in bottom 30% //
SELECT * FROM (
    SELECT name, salary,
           CUME_DIST() OVER (ORDER BY salary) AS cume_dist
    FROM employees
) dist
WHERE cume_dist <= 0.30;

//returns bottom 30% earners//
```

10. INFORMATION FUNCTIONS

10.01 DATABASE()

What it does: Returns the name of the currently selected database.

```
SELECT DATABASE();  
//result: mydb (current database name)//
```

10.02 USER()

What it does: Returns the current MySQL user and host.

```
SELECT USER();  
//result: root@localhost//
```

10.03 VERSION()

What it does: Returns the current MySQL version.

```
SELECT VERSION();  
//result: 8.0.32//
```

10.04 LAST_INSERT_ID()

What it does: Returns the AUTO_INCREMENT ID generated by the last INSERT statement.

Process: After inserting a row → stores generated ID → returns it when called.

```
INSERT INTO users (name) VALUES ('John');  
SELECT LAST_INSERT_ID();  
//returns ID of John just inserted//  
//useful to get newly created ID immediately after insert//
```

10.05 ROW_COUNT()

What it does: Returns the number of rows affected by the last INSERT, UPDATE or DELETE.

Process: After running a change query → counts affected rows → returns count.

```
UPDATE users SET status = 'inactive' WHERE age < 18;
```

```
SELECT ROW_COUNT();
```

```
//returns how many users were updated//
```

ALIASES

MySQL Aliases

A1. COLUMN ALIAS

What it does: Gives a column a temporary display name in the result set. Makes output more readable.

Syntax:

```
SELECT column AS alias_name  
FROM table_name;
```

Example:

```
SELECT name AS employee_name,  
       salary AS monthly_salary  
FROM employees;
```

Output:

employee_name	monthly_salary
Alice	55000
Bob	72000

Key Points:

- AS is optional → SELECT name employee_name also works
- Use quotes for aliases with spaces → SELECT salary AS "monthly salary"
- Cannot be used in WHERE (runs before SELECT), but can be used in ORDER BY and HAVING

A2. TABLE ALIAS

What it does: Gives a table a short temporary name. Makes queries shorter and easier to read, especially with JOINS.

Syntax:

```
SELECT t.column  
FROM table_name AS t;
```

Example:

```
SELECT e.name, d.department_name  
FROM employees AS e  
INNER JOIN departments AS d ON e.dept_id = d.id;
```

Output:

name	department_name
Alice	HR
Bob	Finance

Key Points:

- AS is optional for table aliases too → FROM employees e
- Once aliased, you must use the alias – you cannot mix alias and original name in the same query
- Essential for SELF JOINS where the same table appears twice

A3. EXPRESSION ALIAS

What it does: Gives a calculated expression a name so the result column has a meaningful label instead of the raw formula.

Syntax:

```
SELECT expression AS alias_name
FROM table_name;
```

Example:

```
SELECT name,
       salary * 12 AS annual_salary,
       salary * 0.10 AS bonus
FROM employees;
```

Output:

name	annual_salary	bonus
Alice	660000	5500
Bob	864000	7200

Key Points:

- Without alias, the column header would show the raw expression `salary * 12`
- You can reuse the alias in ORDER BY → ORDER BY `annual_salary` DESC
- Cannot reuse the alias in WHERE – repeat the expression instead

A4. AGGREGATE ALIAS

What it does: Gives the result of an aggregate function (COUNT, SUM, AVG, MAX, MIN) a readable name in the result.

Syntax:

```
SELECT AGGREGATE(column) AS alias_name
FROM table_name
GROUP BY column;
```

Example:

```
SELECT department,
       COUNT(*)      AS total_employees,
       AVG(salary)    AS avg_salary,
       MAX(salary)    AS highest_salary
FROM employees
GROUP BY department;
```

Output:

department	total_employees	avg_salary	highest_salary
HR	4	52000	62000
IT	5	67000	85000
Finance	3	71000	80000

Key Points:

- Without alias, column header shows COUNT(*) or AVG(salary) which is hard to read
- The alias can be used in HAVING → HAVING total_employees > 3
- MySQL is more permissive than standard SQL – it allows SELECT aliases in HAVING

A5. SUBQUERY ALIAS

What it does: Gives a subquery used in FROM a name so it can be referenced in the outer query. In MySQL, this alias is **mandatory**.

Syntax:

```
SELECT columns
FROM (SELECT ...) AS alias_name;
```

Example:

```
SELECT dept, avg_sal
FROM (
    SELECT department AS dept,
           AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department
) AS dept_summary
WHERE avg_sal > 60000;
```

Output:

dept	avg_sal
Finance	71000
IT	67000

Key Points:

- MySQL throws an error if the subquery alias is missing – Every derived table must have its own alias
- The alias name becomes the "table name" for the outer query
- Also called a "derived table alias"

A6. CTE ALIAS

What it does: The name you give a WITH block (CTE). This name is used to reference the CTE in the main query just like a regular table name.

Syntax:

```
WITH alias_name AS (  
    SELECT ...  
)  
SELECT * FROM alias_name;
```

Example:

```
WITH high_earners AS (  
    SELECT name, salary  
    FROM employees  
    WHERE salary > 70000  
)  
dept_counts AS (  
    SELECT department, COUNT(*) AS total  
    FROM employees  
    GROUP BY department  
)  
SELECT h.name, h.salary, d.total  
FROM high_earners h  
JOIN dept_counts d ON /* condition */;
```

Key Points:

- Available in MySQL 8.0+
- Multiple CTEs can be defined, separated by commas
- Each CTE name acts as a temporary table name valid only for that query
- Makes deeply nested subqueries much more readable and maintainable



ICET Certified Developer

COURSEWORK

Module	Data Base Management System
Coursework Title	Final Coursework
Coursework Type	Practical Exercise

CityMed Hospital Management System (HMS)

Scenario – CityMed Hospital Management System

CityMed is a large private hospital located in a busy urban area. The hospital provides medical services through several specialized departments such as Cardiology, Neurology, Orthopedics, Pediatrics, and General Surgery. Currently, most hospital records are maintained manually using paper files and spreadsheets, which has created difficulties in managing patient information, appointments, prescriptions, staff details, and billing records efficiently.

The hospital management has decided to implement a fully digital Hospital Management System (HMS) using a relational database. You have been appointed as the database developer responsible for analyzing the requirements and designing the database system.

The system must manage the following information:

Departments and Wards

The hospital contains multiple departments. Each department stores **Department name, Physical location, Phone extension, Head doctor name**.

Each department contains one or more wards.

Each ward stores **Ward name, Maximum patient capacity, Ward type**.

Ward types include **General, ICU, Surgical, Maternity, Paediatric**.

Medical Staff

The hospital employs doctors and nurses.

Each doctor has a **first name, last name, speciality, email, phone number, hire date, and salary**. Each doctor belongs to one department.

Each nurse Has a first name, last name, email address, and work shift. Shift types are morning, evening, night.

Business Rules

- Doctor email addresses must be unique.
- Salaries cannot be negative.
- Nurse email addresses must be unique.

Patients

When a patient visits the hospital for the first time, the following information must be recorded, first name, last name, date of birth, gender, blood group, phone number, email address, address, registration date.

Gender values male, female, other.

Appointments

Patients can book appointments with doctors.

Each appointment stores appointment date, appointment time, reason for visit, appointment status, clinical notes. Appointment status values scheduled, completed, cancelled, No-Show.

A patient may have many appointments, but each appointment is handled by one doctor.

Prescriptions

After completed appointments, doctors may issue prescriptions.

Each prescription records medicine name, dosage, duration in days, special instructions.

Business Rules

- Duration must be between 1 and 365 days
- The same medicine cannot be prescribed twice within the same appointment

Billing

Each appointment may generate one bill.

Each bill stores total amount, paid amount, payment status, billing date. Payment status values should be unpaid, partial, paid.

Business Rule

- Paid amount cannot exceed the total bill amount.

01. Based on the given scenario, design a complete Entity Relationship Diagram (ERD) for the CityMed Hospital Management System.

Your ER Diagram must clearly show:

- Entities
- Attributes
- Primary Keys
- Relationships
- Cardinalities
- Participation

02. Write the SQL statement to create a database named citymed_hms and select it for use.

03. Write the CREATE TABLE statement for the departments table including all required constraints.

Column Name	Requirement
dept_id	Primary Key and Auto Increment
dept_name	NOT NULL, UNIQUE
location	NOT NULL
phone_ext	NOT NULL, exactly 10 characters
head_doctor	Nullable

04. Write the CREATE TABLE statement for the wards table including Foreign Key constraint.

Column name	Requirement
ward_id	Primary Key, Auto Increment
ward_name	NOT NULL
dept_id	Foreign Key referencing departments
capacity	Value between 1 and 100
ward_type	General, ICU, Surgical, Maternity, Paediatric

05. Write the CREATE TABLE statement for the doctors table.

Column name	Requirement
doctor_id	Primary Key
first_name	NOT NULL
last_name	NOT NULL
speciality	NOT NULL
dept_id	Foreign Key referencing departments
email	UNIQUE
phone	NOT NULL
hire_date	DATE
salary	DECIMAL(10,2), positive values only

06. Write the CREATE TABLE statement for the nurses table.

Column name	Requirements
nurse_id	Primary Key, Auto Increment
first_name	NOT NULL
last_name	NOT NULL
ward_id	Foreign Key referencing wards
shift	Morning, Evening, Night
email	UNIQUE

07. Write the CREATE TABLE statement for the patients table.

Column name	Requirements
patient_id	Primary Key, Auto Increment
first_name	NOT NULL
last_name	NOT NULL
dob	DATE
gender	Male, Female, Other
blood_group	Nullable
phone	NOT NULL
email	Nullable
address	Nullable
registered_on	Default current date

08. Write the CREATE TABLE statement for the appointments table.

Column name	Requirements
appointment_id	Primary Key, Auto Increment
patient_id	Foreign Key referencing patient
doctor_id	Foreign Key referencing doctors
appt_date	DATE
appt_time	TIME
reason	VARCHAR(255)
status	Scheduled, Completed, Cancelled, No-Show
notes	Nullable

09. Write the CREATE TABLE statement for the prescriptions table including Foreign Key constraint, CASCADE actions, Composite UNIQUE constraint.

Column name	Requirements
prescription_id	Primary Key, Auto Increment
appointment_id	Foreign Key referencing appointments
medicine_name	VARCHAR(150)
dosage	NOT NULL
duration_days	Value between 1 and 365
instructions	Nullable
issued_date	Default current date

10. Write the CREATE TABLE statement for the bills table including all required CHECK constraints

Column Name	Requirement
bill_id	Primary Key, Auto Increment
appointment_id	UNIQUE Foreign Key
total_amount	Positive values only
paid_amount	Default value 0
payment_status	Unpaid, Partial, Paid
billing_date	Default current date

11. Write an ALTER TABLE statement to add a nullable column named emergency_contact to the patients table after the address column.
12. Write an ALTER TABLE statement to modify the phone column in the doctors table from VARCHAR(20) to VARCHAR(30).
13. Write an ALTER TABLE statement to add a named CHECK constraint to the appointments table to prevent appointments being booked for past dates.
14. Write the DROP TABLE statements required to remove all tables in the correct order without violating foreign key constraints. Include a comment explaining why the order is important.
15. Insert at least FOUR (4) records into the departments table.
16. Insert at least TWO (2) wards for each department.
17. Insert at least FIVE (5) doctors distributed across different departments.
18. Insert at least THREE (3) nurses assigned to different wards.
19. Insert at least SIX (6) patient records with different genders and blood groups.

20. Insert at least EIGHT (8) appointment records including the following statuses

- Scheduled
- Completed
- Cancelled

21. Insert prescription records for at least FOUR (4) completed appointments.

Requirement:

- At least one appointment must contain TWO (2) medicines.

22. Insert bill records for appointments that contain prescriptions.

Requirements:

- At least one bill must have status Partial
- At least one bill must have status Unpaid

23. Write an UPDATE statement to change one appointment status to Cancelled and add a cancellation note to the notes column.

24. *A patient has paid an additional 5,000 toward a bill.*

Write an UPDATE statement that:

- Increases the *paid_amount*
- Automatically changes:
 - Paid if the bill is fully paid
 - Partial otherwise

Use a CASE expression.

25. Write an UPDATE statement using JOIN to increase the salaries of all doctors in the Cardiology department by 10%.

26. Write a DELETE statement to remove all bills linked to cancelled appointments using a subquery.

27. Write a DELETE statement to remove all No-Show appointments older than TWO (2) years.

28. Display the following patient details

- patient_id
- Full name
- Date of birth
- Gender
- Blood group
- Phone number

Sort the results alphabetically by last name.

29. Display all upcoming scheduled appointments showing

- Patient name
- Doctor name
- Appointment date
- Appointment time
- Reason for visit

Sort the results by appointment date and time.

30.Using INNER JOIN, display

- appointment_id
- Patient name
- Doctor name
- Department name
- Appointment date
- Appointment time
- Appointment status

31.Using INNER JOIN, display prescription details for completed appointments showing

- prescription_id
- Patient name
- Doctor name
- Medicine name
- Dosage
- Duration

32.Using LEFT JOIN, display all patients together with the total number of appointments they have
Requirement

- Include patients with zero appointments
- Label the count column as total_appointments

33.Using LEFT JOIN, display all wards and the number of nurses assigned to each ward Include wards with no nurses assigned.

34.Using RIGHT JOIN, display all doctors and the number of appointments they have
Requirements

- Include doctors with zero appointments
- Label the count column as appointment_count
- Sort the results in descending order

35. Write a query to generate a department revenue report showing

- Department name
- Total appointments
- Gross revenue
- Total collected amount
- Outstanding balance

Group the results by department and sort by gross revenue in descending order.

36. Using a subquery, display the name and phone number of all patients who have unpaid or partially paid bills.

37. Using a correlated subquery, display all doctors whose salary is above the average salary of their own department.

Display

- Doctor name
- Department name
- Salary

38. List all PRIMARY KEY constraints used in the database

For each constraint, explain

- Table name
- Column(s) used
- Reason for selecting the column(s) as the PRIMARY KEY

39. List all FOREIGN KEY constraints used in the database

For each constraint, explain

- Referencing table
- Referenced table
- ON DELETE behavior
- ON UPDATE behavior
- Reason for choosing the behavior

40. Display the details of the patient who has the highest total bill amount in the system

The output should include

- Patient name
- Phone number
- Total bill amount

41. Identify all UNIQUE constraints used in the database and explain the business rule enforced by each constraint.

42. Display the details of all departments that currently do not have any wards assigned to them

43. Generate a report showing the number of appointments handled by each doctor for each appointment status.

The report should display

- Doctor name
- Appointment status
- Total appointments

Sort the results by doctor name.

44. List all CHECK constraints created in the database and explain

- The rule enforced
- The type of invalid data prevented

45. Display the names of patients who have visited more than one department through their appointments.

The output should include

- Patient name
- Number of different departments visited

46. Write an ALTER TABLE statement to add a CHECK constraint limiting *duration_days* in the *prescriptions* table to a maximum of 365 days

47. Doctors may transfer between departments.

Explain what happens to doctor records if a department record is deleted based on the ON DELETE rule you selected for the *fk_doctor_dept* foreign key.

Would you change this behavior? Justify your answer.

48. Write an ALTER TABLE statement to add a constraint ensuring that a bill's *billing_date* cannot be earlier than the related appointment date.
49. Explain the difference between a PRIMARY KEY constraint and a UNIQUE constraint.
Provide one example of each from your database schema.
50. Explain the meaning of referential integrity.
Describe one example from the CityMed Hospital Management System where a foreign key constraint protects referential integrity.

Student Guidelines for Coursework Submission: CityMed Hospital Database Management System

Please adhere to the following guidelines when completing and submitting your CityMed Hospital Database Management System assignment.

1. Submission Requirements

- **Document Format:** All answers, SQL code, and required evidence must be compiled into a single digital document, which should be a **Microsoft Word Document (.docx)** or a **Google Document**.
- **Completeness:** You must provide a clear and comprehensive answer to **all questions** listed in the coursework (Q1 through Q45).
- **Question Numbering:** Clearly label each answer with the corresponding question number (e.g., **Q1.**, **Q27.**, **Q44.**).
- **Submission File Naming & Format:** Your final document must be saved as a PDF file with the naming convention: **DBMS_FINAL_CW_<Your_StudentID>**.
 - Example: **DBMS_FINAL_CW_PR26120345**

2. Content Specifics

- **SQL Tasks (Sections 2, 3, 4):** For all DDL, DML, and Query questions, you must include the complete, correct, and final SQL code used to solve the problem directly in your answer document. Adhere strictly to the required formatting: SQL keywords in **UPPERCASE**, and table/column names in **lowercase**.
- **Constraint Tasks (Section 5):** Provide detailed, well-justified, and technically accurate explanations for all constraint-related questions.

3. Mandatory Evidence

A critical component of this submission is proving the correct implementation of your database schema and data integrity rules.

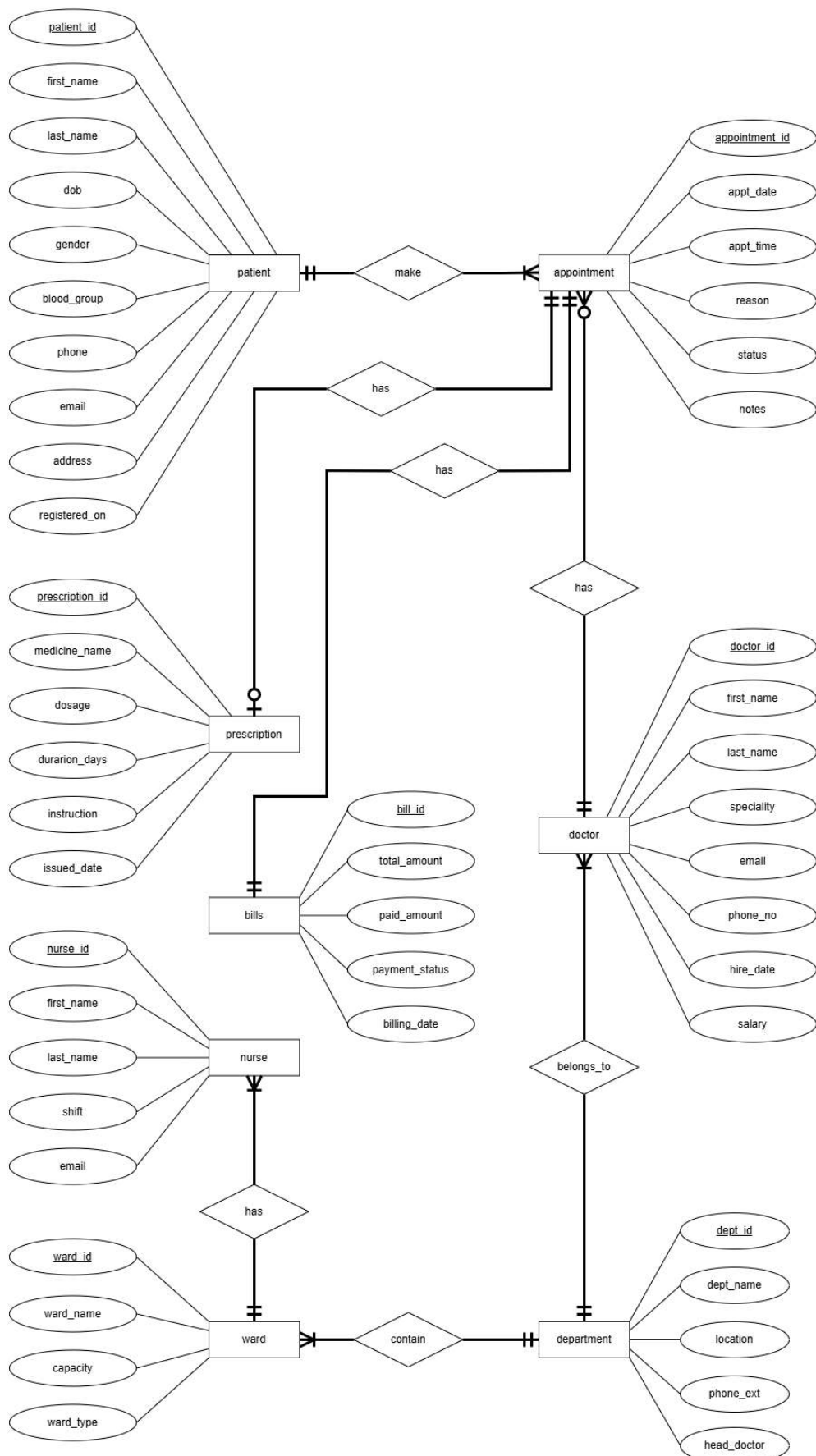
- **Database Structure Screenshot:** After successfully completing the DDL (Section 2) and DML (Section 3) tasks -meaning all eight tables are created and all sample data has been inserted- you must include a **screenshot** clearly showing the structure of the entire citymed_hms database.
 - **Requirement:** This screenshot must be taken from your SQL client (MySQL) and must clearly display the names of all eight tables, their columns, data types, and the definitions of Primary Keys, Foreign Keys, and other constraints. This should represent the database after all data has been loaded.

4. General Advice

- Read the entire scenario (Section 1) and all table specifications carefully before beginning to write your SQL.
- Ensure that all INSERT, UPDATE, and DELETE statements respect the constraints you have defined. Test your code thoroughly before adding it to your final submission document.

This coursework will also be evaluated during the final viva examination.

Q01.



Q02.

```
mysql> CREATE DATABASE citymed_hms;  
Query OK, 1 row affected (0.01 sec)  
  
mysql> USE citymed_hms;  
Database changed
```

Q03.

```
mysql> CREATE TABLE department (  
->     dept_id INT AUTO_INCREMENT PRIMARY KEY,  
->     dept_name VARCHAR(100) NOT NULL UNIQUE,  
->     location VARCHAR(50) NOT NULL,  
->     phone_ext VARCHAR(12) NOT NULL CHECK(LENGTH(phone_ext) = 10),  
->     head_doctor VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.05 sec)
```

Q04.

```
mysql> CREATE TABLE ward (  
->     ward_id INT AUTO_INCREMENT PRIMARY KEY,  
->     ward_name VARCHAR(50) NOT NULL,  
->     dept_id INT,  
->     capacity INT CHECK(capacity BETWEEN 1 AND 100),  
->     ward_type ENUM("General", "ICU", "Surgical", "Maternity", "Paediatric") NOT NULL,  
->     CONSTRAINT fk_ward_dept  
->     FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
->         ON DELETE RESTRICT  
->         ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

Q05.

```
mysql> CREATE TABLE doctor (  
->     doctor_id INT AUTO_INCREMENT PRIMARY KEY,  
->     first_name VARCHAR(50) NOT NULL,  
->     last_name VARCHAR(50) NOT NULL,  
->     speciality VARCHAR(50) NOT NULL,  
->     dept_id INT,  
->     email VARCHAR(100) UNIQUE,  
->     phone VARCHAR(20) NOT NULL,  
->     hire_date DATE,  
->     salary DECIMAL(10,2) CHECK(salary > 0),  
->     CONSTRAINT fk_doctor_dept  
->     FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
-> );  
Query OK, 0 rows affected (0.05 sec)
```


Q06.

```
mysql> CREATE TABLE nurse (  
->     nurse_id INT AUTO_INCREMENT PRIMARY KEY,  
->     first_name VARCHAR(50) NOT NULL,  
->     last_name VARCHAR(50) NOT NULL,  
->     ward_id INT,  
->     shift ENUM("Morning", "Evening", "Night") NOT NULL,  
->     email VARCHAR(100) UNIQUE,  
->     CONSTRAINT fk_nurse_ward  
->     FOREIGN KEY (ward_id) REFERENCES ward(ward_id)  
->         ON DELETE RESTRICT  
->         ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Q07.

```
mysql> CREATE TABLE patient (  
->     patient_id INT AUTO_INCREMENT PRIMARY KEY,  
->     first_name VARCHAR(50) NOT NULL,  
->     last_name VARCHAR(50) NOT NULL,  
->     dob DATE,  
->     gender ENUM("Male", "Female", "Other") NOT NULL,  
->     blood_group VARCHAR(5),  
->     phone VARCHAR(20) NOT NULL,  
->     email VARCHAR(100),  
->     address VARCHAR(150),  
->     registered_on DATE DEFAULT (CURRENT_DATE)  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

Q08.

```
mysql> CREATE TABLE appointment (  
->     appointment_id INT AUTO_INCREMENT PRIMARY KEY,  
->     patient_id INT,  
->     doctor_id INT,  
->     appt_date DATE,  
->     appt_time TIME,  
->     reason VARCHAR(255),  
->     status ENUM("Scheduled", "Completed", "Cancelled", "No-Show") NOT NULL,  
->     notes VARCHAR(100),  
->     CONSTRAINT fk_appt_patient  
->     FOREIGN KEY (patient_id) REFERENCES patient(patient_id)  
->         ON DELETE RESTRICT  
->         ON UPDATE CASCADE,  
->     CONSTRAINT fk_appt_doctor  
->     FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id)  
->         ON DELETE RESTRICT  
->         ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Q09.

```
mysql> CREATE TABLE prescription (  
->     prescription_id INT AUTO_INCREMENT PRIMARY KEY,  
->     appointment_id INT,  
->     medicine_name VARCHAR(150),  
->     dosage VARCHAR(50) NOT NULL,  
->     duration_days INT CHECK(duration_days BETWEEN 1 AND 365),  
->     instruction VARCHAR(150),  
->     issued_date DATE DEFAULT (CURRENT_DATE),  
->     CONSTRAINT fk_prescription_appointment  
->     FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id)  
->     ON DELETE CASCADE ON UPDATE CASCADE,  
->     CONSTRAINT uk_prescription_pres_appt  
->     UNIQUE (appointment_id, medicine_name)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Q10.

```
mysql> CREATE TABLE bill (  
->     bill_id INT AUTO_INCREMENT PRIMARY KEY,  
->     appointment_id INT UNIQUE,  
->     total_amount DECIMAL(10,2) CHECK(total_amount > 0),  
->     paid_amount DECIMAL(10,2) DEFAULT 0,  
->     payment_status VARCHAR(20) CHECK(payment_status IN ("Unpaid", "Partial", "Paid")),  
->     billing_date DATE DEFAULT (CURRENT_DATE),  
->     CONSTRAINT fk_bill_appointment  
->     FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id),  
->     CONSTRAINT chk_bill_paid  
->     CHECK(paid_amount <= total_amount)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

Q11.

```
mysql> ALTER TABLE patient  
-> ADD emergency_contact VARCHAR(20)  
-> AFTER address;  
Query OK, 0 rows affected (0.03 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

Q12.

```
mysql> ALTER TABLE doctor  
-> MODIFY phone VARCHAR(30);  
Query OK, 0 rows affected (0.08 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

Q13.

```
ALTER TABLE appointment
ADD CONSTRAINT chk_appointment_appt_date
CHECK(appt_date >= CURRENT_DATE);
```

I cannot prove it via MySQL. Because MySQL showing error; showing that it doesn't allow the statements like CURRENT_DATE inside a CHECK constraint. For my knowledge I wrote this query.

Q14.

```
DROP TABLE IF EXISTS prescription;
DROP TABLE IF EXISTS bill;
DROP TABLE IF EXISTS appointment;
DROP TABLE IF EXISTS patient;
DROP TABLE IF EXISTS doctor;
DROP TABLE IF EXISTS nurse;
DROP TABLE IF EXISTS ward;
DROP TABLE IF EXISTS department;
```

Q15.

```
mysql> INSERT INTO department (dept_name, location, phone_ext, head_doctor)
-> VALUES
-> ("Cardiology", "A block, 1 st floor", "0112345670", "Dr. Sunil Perera"),
-> ("Neurology", "B block, 2 st floor", "0112345671", "Dr. Kamani Silva"),
-> ("Orthopedics", "C block, 1 st floor", "0112345672", "Dr. Ruwan Fernando"),
-> ("Paediatrics", "D block, 3 st floor", "0112345673", "Dr. Dilani Rajapaksa"),
-> ("General Medicine", "E block, 1st floor", "0112345674", "Dr. Asanka Jayawardena"),
-> ("Radiology", "F block, 2nd floor", "0112345675", "Dr. Priya Mendis");
Query OK, 6 rows affected (0.00 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

Q16.

```
mysql> INSERT INTO ward (ward_name, dept_id, capacity, ward_type)
-> VALUES
-> ("Cardiology General Ward", 1, 30, "General"),
-> ("Cardiology ICU", 1, 10, "ICU"),
-> ("Neurology General Ward", 2, 25, "General"),
-> ("Neurology Surgical Ward", 2, 15, "Surgical"),
-> ("Ortho Surgical Ward", 3, 20, "Surgical"),
-> ("Ortho General Ward", 3, 30, "General"),
-> ("Paediatric Ward", 4, 20, "Paediatric"),
-> ("Paediatric ICU", 4, 8, "ICU"),
-> ("General Recovery Ward", NULL, 20, "General"),
-> ("Emergency ICU", NULL, 10, "ICU");
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

Q17.

```
mysql> INSERT INTO doctor (first_name, last_name, speciality, dept_id, email, phone, hire_date, salary)
-> VALUES
-> ("Sunil", "Perera", "Cardiologist", 1, "sunil.perera@citymed.lk", "+94771234561", '2018-03-15', 150000),
-> ("Kamani", "Silva", "Neurologist", 2, "kamani.silva@citymed.lk", "+94772234562", '2019-06-20', 145000),
-> ("Ruwan", "Fernando", "Orthopaedic", 3, "ruwan.fernando@citymed.lk", "+94773234563", '2020-01-10', 140000),
-> ("Dilani", "Rajapaksa", "Paediatrician", 4, "dilani.rajapaksa@citymed.lk", "+94774234564", '2017-09-05', 135000),
-> ("Chaminda", "Bandara", "Cardiac Surgeon", 1, "chaminda.bandara@citymed.lk", "+94775234565", '2021-04-18', 160000),
-> ("Kasun", "Wijesinghe", "Interventional Cardiologist", 1, "kasun.w@citymed.lk", "+94771234566", '2019-03-10', 155000.00),
-> ("Thilanka", "Perera", "Cardiac Electrophysiologist", 1, "thilanka.p@citymed.lk", "+94771234567", '2020-07-15', 148000.00),
-> ("Nadeesha", "Fernando", "Heart Failure Specialist", 1, "nadeesha.f@citymed.lk", "+94771234568", '2022-01-20', 142000.00),
-> ("Lasith", "Bandara", "Neurophysiologist", 2, "lasith.b@citymed.lk", "+94772234569", '2018-06-12', 147000.00),
-> ("Hiruni", "Dissanayake", "Stroke Specialist", 2, "hiruni.d@citymed.lk", "+94772234570", '2021-09-05', 143000.00),
-> ("Sachini", "Rajapaksa", "Epilepsy Specialist", 2, "sachini.r@citymed.lk", "+94772234571", '2023-02-18', 138000.00),
-> ("Nuwan", "Jayawardena", "Spine Surgeon", 3, "nuwan.j@citymed.lk", "+94773234572", '2017-11-30', 152000.00),
-> ("Chamari", "Silva", "Joint Replacement Surgeon", 3, "chamari.s@citymed.lk", "+94773234573", '2020-04-22', 146000.00),
-> ("Tharaka", "Kumari", "Sports Medicine Specialist", 3, "tharaka.k@citymed.lk", "+94773234574", '2022-08-14', 139000.00),
-> ("Malsha", "Senanayake", "Trauma Surgeon", 3, "malsha.sn@citymed.lk", "+94773234575", '2019-05-08', 150000.00),
-> ("Dinusha", "Wickramasinghe", "Neonatologist", 4, "dinusha.w@citymed.lk", "+94774234576", '2018-12-01', 144000.00),
-> ("Sewwandi", "Abeysekara", "Paediatric Surgeon", 4, "sewwandi.a@citymed.lk", "+94774234577", '2021-03-25', 141000.00),
-> ("Kasuni", "Gunawardena", "Paediatric Neurologist", 4, "kasuni.g@citymed.lk", "+94774234578", '2023-06-10', 137000.00),
-> ("Dilini", "Pathirana", "Paediatric Cardiologist", 4, "dilini.pt@citymed.lk", "+94774234579", '2020-10-17', 145000.00),
-> ("Roshan", "Mendis", "Paediatric Oncologist", 4, "roshan.m@citymed.lk", "+94774234580", '2022-11-28', 136000.00);
Query OK, 20 rows affected (0.01 sec)
Records: 20 Duplicates: 0 Warnings: 0
```

Q18.

```
mysql> INSERT INTO nurse (first_name, last_name, ward_id, shift, email)
-> VALUES
-> ("Madhavi", "Kumari", 1, "Morning", "madhavi.kumari@citymed.lk"),
-> ("Thilanka", "Dissanayake", 3, "Evening", "thilanka.d@citymed.lk"),
-> ("Sachini", "Wijesinghe", 7, "Night", "sachini.w@citymed.lk"),
-> ("Hiruni", "Bandara", 3, "Morning", "hiruni.b@citymed.lk"),
-> ("Dilini", "Rajapaksa", 1, "Night", "dilini.r@citymed.lk"),
-> ("Dinusha", "Kumari", 7, "Morning", "dinusha.k@citymed.lk"),
-> ("Malsha", "Perera", 1, "Evening", "malsha.p@citymed.lk"),
-> ("Thilini", "Dissanayake", 8, "Night", "thilini.d@citymed.lk"),
-> ("Kasuni", "Fernando", 1, "Morning", "kasuni.f@citymed.lk"),
-> ("Sewwandi", "Wickrama", 7, "Evening", "sewwandi.w@citymed.lk"),
-> ("Nadeesha", "Silva", 3, "Night", "nadeesha.s@citymed.lk"),
-> ("Chamari", "Jayawardena", 5, "Morning", "chamari.j@citymed.lk"),
-> ("Amaya", "Perera", 1, "Morning", "amaya.p@citymed.lk"),
-> ("Buddhika", "Fernando", 3, "Evening", "buddhika.f@citymed.lk"),
-> ("Chathurika", "Silva", 5, "Night", "chathurika.s@citymed.lk"),
-> ("Devindi", "Bandara", 7, "Morning", "devindi.b@citymed.lk"),
-> ("Erandhi", "Kumari", 8, "Evening", "erandhi.k@citymed.lk"),
-> ("Fathima", "Rizvi", 1, "Night", "fathima.r@citymed.lk"),
-> ("Gayani", "Wickramasinghe", 3, "Morning", "gayani.w@citymed.lk"),
-> ("Hashini", "Jayawardena", 7, "Night", "hashini.j@citymed.lk");
Query OK, 20 rows affected (0.01 sec)
Records: 20 Duplicates: 0 Warnings: 0
```

Q19.

```
mysql> INSERT INTO patient (first_name, last_name, dob, gender, blood_group, phone, email, address, registered_on)
-> VALUES
-> ("Kamal", "Dias", '1990-05-12', 'Male', 'A+', '+94761234561', 'kamal.p@gmail.com', 'No.12, Kandy Road, Kandy', '2020-05-05'),
-> ("Nimasha", "Silva", '1995-08-23', 'Female', 'B+', '+94762234562', 'nimasha.s@gmail.com', 'No.45, Galle Road, Colombo', '2020-05-05'),
-> ("Ruman", "Abeysekara", '1988-11-30', 'Male', 'O+', '+94763234563', 'ruman.f@gmail.com', 'No.78, Matara Road, Galle', '2020-05-05'),
-> ("Dinithi", "Wickramasinghe", '2000-03-17', 'Female', 'AB+', '+94764234564', 'dilani.w@gmail.com', 'No.22, Kurunegala Road, Matara', '2020-05-06'),
-> ("Sithum", "Jayawardena", '1992-07-08', 'Other', 'O-', '+94765234565', 'sithum.j@gmail.com', 'No.55, Negombo Road, Negombo', '2020-05-06'),
-> ("Anjana", "Bandara", '1985-12-25', 'Male', 'B-', '+94766234566', 'anjana.b@gmail.com', 'No.90, Colombo Road, Kurunegala', '2020-05-06'),
-> ("Tharaka", "Dissanayake", '1993-08-13', 'Male', 'AB-', '+94767234567', 'tharaka.d@gmail.com', 'No.34, Kandy Road, Kurunegala', '2020-05-07'),
-> ("Sachini", "Wijasinghe", '1998-09-22', 'Female', 'A-', '+94768234568', 'sachini.w@gmail.com', 'No.67, Galle Road, Matara', '2020-05-07'),
-> ("Nuwari", "Senanayake", '1987-01-30', 'Male', 'B+', '+94769234569', 'nuwan.s@gmail.com', 'No.89, Colombo Road, Negombo', '2020-05-07'),
-> ("Dilani", "Perera", '1994-06-18', 'Female', 'A+', '+94770234570', 'dilani.p@gmail.com', 'No.11, Kandy Road, Colombo', '2020-05-08'),
-> ("Chamara", "Fernando", '1983-02-27', 'Male', 'O+', '+94771234571', 'chamara.f@gmail.com', 'No.33, Galle Road, Galle', '2020-05-08'),
-> ("Ishara", "Rajapaksa", '2001-11-05', 'Female', 'B-', '+94772234572', 'ishara.r@gmail.com', 'No.56, Matara Road, Matara', '2020-05-08'),
-> ("Kasun", "Kunara", '1990-08-14', 'Male', 'AB+', '+94773234573', 'kasun.k@gmail.com', 'No.78, Negombo Road, Kurunegala', '2020-05-09'),
-> ("Nalisha", "Gunawardena", '1997-03-22', 'Female', 'O-', '+94774234574', 'nalisha.g@gmail.com', 'No.99, Colombo Road, Kandy', '2020-05-09'),
-> ("Dilan", "Madushanka", '1980-07-31', 'Male', 'A-', '+94775234575', 'dilan.m@gmail.com', 'No.15, Kandy Road, Negombo', '2020-05-09'),
-> ("Thilini", "Amarasinghe", '1999-12-10', 'Female', 'B+', '+94776234576', 'thilini.a@gmail.com', 'No.42, Galle Road, Colombo', '2020-05-10'),
-> ("Lasith", "Rathirana", '1991-05-19', 'Male', 'A+', '+94777234577', 'lasith.r@gmail.com', 'No.63, Matara Road, Galle', '2020-05-10'),
-> ("Nadeesha", "Ranasinghe", '1996-09-07', 'Female', 'O+', '+94778234578', 'nadeesha.r@gmail.com', 'No.84, Kurunegala Road, Matara', '2020-05-10'),
-> ("Buddhika", "Weerasinghe", '1984-01-25', 'Male', 'AB-', '+94779234579', 'buddhika.w@gmail.com', 'No.27, Negombo Road, Negombo', '2020-05-11'),
-> ("Sewandi", "Liyanage", '2002-04-13', 'Female', 'B-', '+94780234580', 'sewandi.l@gmail.com', 'No.38, Colombo Road, Kurunegala', '2020-05-11'),
-> ("Ravindu", "Lakshana", '1989-10-02', 'Male', 'A+', '+94781234581', 'ravindu.l@gmail.com', 'No.51, Kandy Road, Kandy', '2020-05-11'),
-> ("Hiruni", "Jayasuriya", '1993-06-28', 'Female', 'O-', '+94782234582', 'hiruni.j@gmail.com', 'No.72, Galle Road, Colombo', '2020-05-12'),
-> ("Asanka", "Rathnayake", '1980-03-15', 'Male', 'AB+', '+94783234583', 'asanka.r@gmail.com', 'No.93, Matara Road, Galle', '2020-05-12'),
-> ("Dinusha", "Morath", '1996-08-01', 'Female', 'B+', '+94784234584', 'dinusha.h@gmail.com', 'No.18, Kurunegala Road, Matara', '2020-05-12'),
-> ("Pradeep", "Sapeth", '1987-12-20', 'Male', 'A+', '+94785234585', 'pradeep.s@gmail.com', 'No.29, Negombo Road, Negombo', '2020-05-13'),
-> ("Chathurika", "Muthukumarana", '1995-07-11', 'Female', 'O+', '+94786234586', 'chathurika.m@gmail.com', 'No.44, Colombo Road, Kurunegala', '2020-05-13'),
-> ("Vimukthi", "Siriwardena", '2000-02-08', 'Male', 'B-', '+94787234587', 'vimukthi.s@gmail.com', 'No.65, Kandy Road, Kandy', '2020-05-13'),
-> ("Erandhi", "Dahanayake", '1992-11-17', 'Female', 'AB-', '+94788234588', 'erandhi.d@gmail.com', 'No.87, Galle Road, Colombo', '2020-05-14'),
-> ("Gayani", "Wickrama", '1985-04-30', 'Male', 'A-', '+94789234589', 'gayani.w@gmail.com', 'No.19, Matara Road, Galle', '2020-05-14'),
-> ("Hashini", "Tennakoon", '1997-01-23', 'Female', 'O-', '+94790234590', 'hashini.t@gmail.com', 'No.32, Kurunegala Road, Matara', '2020-05-14');
Query OK, 30 rows affected (0.81 sec)
Records: 30 Duplicates: 0 Warnings: 0
```

Q20.

```
mysql> INSERT INTO appointment (patient_id, doctor_id, appt_date, appt_time, reason, status, notes)
-> VALUES
-> (4, 4, '2026-05-10', '09:00:00', "Child fever", "Completed", "Paediatric checkup done, fever subsided"),
-> (1, 1, '2026-05-15', '09:00:00', "Chest pain checkup", "Scheduled", NULL),
-> (5, 1, '2026-05-16', '14:00:00', "Heart palpitations", "Completed", "ECG taken, medication prescribed"),
-> (3, 3, '2026-05-18', '08:00:00', "Shoulder pain", "Cancelled", "Patient cancelled due to family emergency"),
-> (2, 2, '2026-05-20', '10:30:00', "Frequent headaches", "Scheduled", NULL),
-> (6, 5, '2026-05-22', '10:00:00', "Post surgery followup", "Completed", "Recovery progressing well, stitches removed"),
-> (3, 3, '2026-05-27', '11:00:00', "Knee pain", "Scheduled", NULL),
-> (1, 2, '2026-06-01', '11:30:00', "Dizziness and nausea", "Completed", "MRI scan recommended, follow up in 2 weeks"),
-> (7, 1, '2026-06-02', '08:30:00', "Shortness of breath", "Scheduled", NULL),
-> (9, 4, '2026-06-03', '10:00:00', "Child vaccination", "Completed", "Vaccination administered successfully"),
-> (10, 2, '2026-06-04', '09:30:00', "Migraine episodes", "Scheduled", NULL),
-> (12, 3, '2026-06-05', '11:00:00', "Back pain", "Completed", "Physiotherapy recommended"),
-> (14, 5, '2026-06-06', '13:00:00', "Appendix followup", "Completed", "Post operative recovery normal"),
-> (15, 1, '2026-06-07', '14:30:00', "Irregular heartbeat", "Cancelled", "Doctor unavailable, rescheduled"),
-> (16, 2, '2026-06-08', '09:00:00', "Severe headache", "Completed", "CT scan done, no abnormalities found"),
-> (18, 3, '2026-06-09', '10:30:00', "Hip joint pain", "Scheduled", NULL),
-> (20, 4, '2026-06-10', '08:00:00', "Fever and rash", "Completed", "Allergic reaction treated, antihistamines prescribed"),
-> (22, 5, '2026-06-11', '11:30:00', "Hernia checkup", "Scheduled", NULL),
-> (23, 1, '2026-06-12', '14:00:00', "Chest tightness", "Completed", "Stress ECG done, medication adjusted"),
-> (24, 2, '2026-06-13', '09:30:00', "Memory loss concern", "Scheduled", NULL),
-> (25, 3, '2026-06-14', '10:00:00', "Wrist fracture followup", "Completed", "X-ray taken, healing well"),
-> (26, 4, '2026-06-15', '08:30:00', "Child growth concern", "Cancelled", "Patient did not show up"),
-> (27, 5, '2026-06-16', '13:30:00', "Gallstone followup", "Completed", "Ultrasound done, surgery may be required"),
-> (28, 1, '2026-06-17', '11:00:00', "Breathlessness on exertion", "Scheduled", NULL),
-> (30, 2, '2026-06-18', '10:00:00', "Chronic dizziness", "Scheduled", NULL);
Query OK, 25 rows affected (0.81 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Q21.

```
mysql> INSERT INTO prescription (appointment_id, medicine_name, dosage, duration_days, instruction, issued_date)
-> VALUES
-> (1, "Paracetamol Syrup", "5ml three times daily", 5, "Give after meals, store in cool place", '2026-05-10'),
-> (1, "Cetirizine Syrup", "2.5ml once daily at night", 5, "Give before bedtime", '2026-05-10'),
-> (1, "Vitamin C Syrup", "5ml once daily", 7, "Give after breakfast", '2026-05-10'),
-> (1, "Oral Rehydration Salts", "200ml after each loose stool", 3, "Dissolve one sachet in 200ml clean water", '2026-05-10'),
-> (3, "Metoprolol", "50mg once daily", 30, "Take in the morning before breakfast", '2026-05-16'),
-> (3, "Aspirin", "75mg once daily", 30, "Take after meals, do not crush or chew", '2026-05-16'),
-> (3, "Atorvastatin", "20mg once daily at night", 60, "Take at bedtime, avoid grapefruit juice", '2026-05-16'),
-> (3, "Pantoprazole", "40mg once daily", 14, "Take 30 minutes before breakfast", '2026-05-16'),
-> (6, "Amoxicillin", "500mg three times daily", 7, "Complete the full course, take with food", '2026-05-22'),
-> (6, "Ibuprofen", "400mg twice daily", 5, "Take after meals, avoid on empty stomach", '2026-05-22'),
-> (6, "Povidone Iodine Ointment", "Apply thin layer twice daily", 7, "Clean wound before applying, keep covered", '2026-05-22'),
-> (8, "Cinnarizine", "25mg three times daily", 14, "Take with food, avoid driving", '2026-06-01'),
-> (8, "Metoclopramide", "10mg three times daily", 7, "Take 30 minutes before meals", '2026-06-01'),
-> (8, "Betahistine", "16mg twice daily", 30, "Take with food at regular intervals", '2026-06-01'),
-> (8, "Diazepam", "2mg once daily at night", 7, "Take before bedtime, avoid alcohol", '2026-06-01'),
-> (10, "Paracetamol Syrup", "5ml every 6 hours if needed", 2, "Give only if fever develops after vaccination", '2026-06-03'),
-> (10, "Cetirizine Syrup", "2.5ml once daily", 3, "Give if skin redness or itching develops", '2026-06-03'),
-> (12, "Diclofenac", "50mg twice daily", 10, "Take after meals, avoid if stomach is sensitive", '2026-06-05'),
-> (12, "Methocarbamol", "750mg three times daily", 7, "May cause drowsiness, avoid operating machinery", '2026-06-05'),
-> (12, "Calcium Carbonate", "500mg twice daily", 30, "Take with meals for better absorption", '2026-06-05'),
-> (12, "Vitamin D3", "1000 IU once daily", 60, "Take with a meal containing fat", '2026-06-05'),
-> (13, "Cefuroxime", "250mg twice daily", 7, "Take at the same time each day, complete full course", '2026-06-06'),
-> (13, "Metronidazole", "400mg three times daily", 5, "Avoid alcohol completely during course", '2026-06-06'),
-> (13, "Tramadol", "50mg every 8 hours as needed", 5, "Take only when pain is present, max 3 doses daily", '2026-06-06'),
-> (13, "Lactulose", "15ml twice daily", 7, "Drink plenty of water, helps prevent constipation", '2026-06-06'),
-> (13, "Omeprazole", "20mg once daily", 14, "Take 30 minutes before breakfast", '2026-06-06'),
-> (15, "Sumatriptan", "50mg at onset of headache", 10, "Take as soon as headache begins, max 2 doses in 24 hours", '2026-06-08'),
-> (15, "Amitriptyline", "10mg once daily at night", 30, "Take at bedtime, do not stop suddenly", '2026-06-08'),
-> (15, "Propranolol", "40mg twice daily", 30, "Take at regular intervals, do not miss doses", '2026-06-08'),
-> (15, "Naproxen", "250mg twice daily", 7, "Take with food or milk, avoid alcohol", '2026-06-08');
Query OK, 30 rows affected (0.02 sec)
Records: 30 Duplicates: 0 Warnings: 0
```

Q22.

```
mysql> INSERT INTO bill (appointment_id, total_amount, paid_amount, payment_status, billing_date)
-> VALUES
-> (1, 5000, 5000, "Paid", '2026-05-10'),
-> (3, 25000, 15000, "Partial", '2026-05-16'),
-> (6, 18000, DEFAULT, "Unpaid", '2026-05-22'),
-> (8, 12000, 6000, "Partial", '2026-06-01'),
-> (10, 8500, 8500, "Paid", '2026-06-03'),
-> (12, 15000, 15000, "Paid", '2026-06-05'),
-> (13, 32000, 20000, "Partial", '2026-06-06'),
-> (15, 14000, DEFAULT, "Unpaid", '2026-06-08'),
-> (17, 9500, 9500, "Paid", '2026-06-10'),
-> (19, 22000, 22000, "Paid", '2026-06-12'),
-> (21, 11000, 5000, "Partial", '2026-06-14'),
-> (23, 28000, DEFAULT, "Unpaid", '2026-06-16'),
-> (25, 7500, 7500, "Paid", '2026-06-14'),
-> (7, 16000, 8000, "Partial", '2026-05-27'),
-> (9, 19500, DEFAULT, "Unpaid", '2026-06-02');
Query OK, 15 rows affected (0.01 sec)
Records: 15 Duplicates: 0 Warnings: 0
```

Q23.

```
mysql> UPDATE appointment
-> SET status = "Cancelled",
-> notes = "Patient cancelled due to sudden recovery"
-> WHERE appointment_id = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

Q24.

```
mysql> UPDATE bill
-> SET paid_amount = paid_amount + 5000,
->     payment_status = CASE
->   WHEN (paid_amount + 5000) = total_amount THEN "Paid"
->   ELSE "Partial"
->   END
-> WHERE bill_id = 3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Q25.

```
mysql> UPDATE
-> doctor AS d JOIN department AS dept
-> ON d.dept_id = dept.dept_id
-> SET d.salary = d.salary * 1.1
-> WHERE dept.dept_name = "Cardiology";
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5  Changed: 5  Warnings: 0
```

Q26.

```
mysql> DELETE FROM bill
-> WHERE appointment_id IN ( SELECT appointment_id
->                           FROM appointment
->                           WHERE status = "Cancelled" );
Query OK, 0 rows affected (0.00 sec)
```

Q27.

```
mysql> DELETE FROM appointment
-> WHERE status = "No-Show"
->     AND CURRENT_DATE < appt_date + INTERVAL 2 YEAR;
Query OK, 0 rows affected (0.03 sec)
```

Q28.

```
mysql> SELECT patient_id, CONCAT(first_name, " ", last_name) AS Full_name,
->      dob AS Date_of_Birth,
->      Gender,
->      Blood_group,
->      phone AS Phone_number
-> FROM patient
-> ORDER BY last_name ASC;
```

patient_id	Full_name	Date_of_Birth	Gender	Blood_group	Phone_number
3	Ruwan Abeysekara	1988-11-30	Male	O+	+94763234563
16	Thilini Amarasinghe	1999-12-10	Female	B+	+94776234576
6	Anjana Bandara	1985-12-25	Male	B-	+94766234566
28	Erandhi Dahanayake	1992-11-17	Female	AB-	+94788234588
1	Kamal Dias	1990-05-12	Male	A+	+94761234561
7	Tharaka Dissanayake	1993-04-15	Male	AB-	+94767234567
11	Chamara Fernando	1983-02-27	Male	O+	+94771234571
14	Malsha Gunawardena	1997-03-22	Female	O-	+94774234574
24	Dinusha Herath	1998-08-04	Female	B+	+94784234584
22	Hiruni Jayasuriya	1993-06-28	Female	O-	+94782234582
5	Sithum Jayawardena	1992-07-08	Other	O-	+94765234565
13	Kasun Kumara	1990-08-14	Male	AB+	+94773234573
21	Ravindu Lakshan	1989-10-02	Male	A-	+94781234581
20	Sewwandi Liyanage	2002-04-13	Female	B-	+94780234580
15	Dilan Madushanka	1986-07-31	Male	A-	+94775234575
26	Chathurika Muthukumarana	1995-07-11	Female	O+	+94786234586
17	Lasith Pathirana	1991-05-19	Male	A+	+94777234577
10	Dilani Perera	1994-06-18	Female	A+	+94770234570
12	Ishara Rajapaksa	2001-11-05	Female	B-	+94772234572
18	Nadeesha Ranasinghe	1996-09-07	Female	O+	+94778234578
23	Asanka Rathnayake	1980-03-15	Male	AB+	+94783234583
25	Pradeep Sampath	1987-12-20	Male	A+	+94785234585
9	Nuwan Senanayake	1987-01-30	Male	B+	+94769234569
2	Nimasha Silva	1995-08-23	Female	B+	+94762234562
27	Vimukthi Siriwardena	2000-02-08	Male	B-	+94787234587
30	Hashini Tennakoon	1997-01-23	Female	O-	+94790234590
19	Buddhika Weerasinghe	1984-01-25	Male	AB-	+94779234579
29	Gayana Wickrama	1985-04-30	Male	A-	+94789234589
4	Dinithi Wickramasinghe	2000-03-17	Female	AB+	+94764234564
8	Sachini Wijesinghe	1998-09-22	Female	A-	+94768234568

30 rows in set (0.00 sec)

Q29.

```
mysql> SELECT CONCAT(p.first_name, " ", p.last_name) AS Patient_name,
->      CONCAT(d.first_name, " ", d.last_name) AS Doctor_name,
->      a.appt_date AS Appointment_date,
->      a.appt_time AS Appointment_time,
->      a.reason AS Reason_for_visit
-> FROM appointment AS a
-> JOIN patient AS p
-> ON a.patient_id = p.patient_id
-> JOIN doctor AS d
-> ON a.doctor_id = d.doctor_id
-> WHERE a.status = "Scheduled"
-> AND CONCAT(a.appt_date, " ", a.appt_time) > NOW()
-> ORDER BY a.appt_date AND a.appt_time;
```

Patient_name	Doctor_name	Appointment_date	Appointment_time	Reason_for_visit
Nimasha Silva	Kamani Silva	2026-05-20	10:30:00	Frequent headaches
Ruwan Abeysekara	Ruwan Fernando	2026-05-27	11:00:00	Knee pain
Tharaka Dissanayake	Sunil Perera	2026-06-02	08:30:00	Shortness of breath
Dilani Perera	Kamani Silva	2026-06-04	09:30:00	Migraine episodes
Nadeesha Ranasinghe	Ruwan Fernando	2026-06-09	10:30:00	Hip joint pain
Hiruni Jayasuriya	Chaminda Bandara	2026-06-11	11:30:00	Hernia checkup
Dinusha Herath	Kamani Silva	2026-06-13	09:30:00	Memory loss concern
Erandhi Dahanayake	Sunil Perera	2026-06-17	11:00:00	Breathlessness on exertion
Hashini Tennakoon	Kamani Silva	2026-06-18	10:00:00	Chronic dizziness

9 rows in set (0.00 sec)

Q30.

```
mysql> SELECT a.appointment_id,
->      CONCAT(p.first_name, " ", p.last_name) AS Patient_name,
->      CONCAT(d.first_name, " ", d.last_name) AS Doctor_id,
->      dept.dept_name AS Department_name,
->      a.appt_date AS Appointment_date,
->      a.appt_time AS Appointment_time,
->      a.status AS Appointment_status
-> FROM appointment AS a
-> INNER JOIN patient AS p
->   ON p.patient_id = a.patient_id
-> INNER JOIN doctor AS d
->   ON d.doctor_id = a.doctor_id
-> INNER JOIN department AS dept
->   ON d.dept_id = dept.dept_id;
```

appointment_id	Patient_name	Doctor_id	Department_name	Appointment_date	Appointment_time	Appointment_status
2	Kamal Dias	Sunil Perera	Cardiology	2026-05-15	09:00:00	Cancelled
3	Sithum Jayawardena	Sunil Perera	Cardiology	2026-05-16	14:00:00	Completed
9	Tharaka Dissanayake	Sunil Perera	Cardiology	2026-06-02	08:30:00	Scheduled
14	Dilan Madushanka	Sunil Perera	Cardiology	2026-06-07	14:30:00	Cancelled
19	Asanka Rathnayake	Sunil Perera	Cardiology	2026-06-12	14:00:00	Completed
24	Erandhi Dahanyake	Sunil Perera	Cardiology	2026-06-17	11:00:00	Scheduled
6	Anjana Bandara	Chaminda Bandara	Cardiology	2026-05-22	10:00:00	Completed
13	Malsha Gunawardena	Chaminda Bandara	Cardiology	2026-06-06	13:00:00	Completed
18	Hiruni Jayasuriya	Chaminda Bandara	Cardiology	2026-06-11	11:30:00	Scheduled
23	Vimukthi Siriwardena	Chaminda Bandara	Cardiology	2026-06-16	13:30:00	Completed
5	Nimasha Silva	Kamani Silva	Neurology	2026-05-28	10:30:00	Scheduled
8	Kamal Dias	Kamani Silva	Neurology	2026-06-01	11:30:00	Completed
11	Dilani Perera	Kamani Silva	Neurology	2026-06-04	09:30:00	Scheduled
15	Thilini Amarasinghe	Kamani Silva	Neurology	2026-06-08	09:00:00	Completed
20	Dinusha Herath	Kamani Silva	Neurology	2026-06-13	09:30:00	Scheduled
25	Hashini Tennakoon	Kamani Silva	Neurology	2026-06-18	10:00:00	Scheduled
4	Ruwan Abeyssekara	Ruwan Fernando	Orthopedics	2026-05-27	08:00:00	Cancelled
7	Ruwan Abeyssekara	Ruwan Fernando	Orthopedics	2026-05-27	11:00:00	Scheduled
12	Ishara Rajapaksa	Ruwan Fernando	Orthopedics	2026-06-05	11:00:00	Completed
16	Nadeesha Ranasinghe	Ruwan Fernando	Orthopedics	2026-06-09	10:30:00	Scheduled
21	Pradeep Sampath	Ruwan Fernando	Orthopedics	2026-06-14	10:00:00	Completed
1	Dinithi Wickramasinghe	Dilani Rajapaksa	Paediatrics	2026-05-10	09:00:00	Completed
10	Nuwan Senanayake	Dilani Rajapaksa	Paediatrics	2026-06-03	10:00:00	Completed
17	Sewandi Liyanage	Dilani Rajapaksa	Paediatrics	2026-06-10	08:00:00	Completed
22	Chathurika Muthukumarana	Dilani Rajapaksa	Paediatrics	2026-06-15	08:30:00	Cancelled

25 rows in set (0.00 sec)

Q31.

```
mysql> SELECT pres.prescription_id,
->      CONCAT(p.first_name, " ", p.last_name) AS Patient_name,
->      CONCAT(d.first_name, " ", d.last_name) AS Doctor_name,
->      pres.medicine_name AS Medicine_name,
->      pres.dosage AS Dosage,
->      pres.duration_days AS Duration
-> FROM appointment AS a
-> INNER JOIN doctor AS d
->   ON d.doctor_id = a.doctor_id
-> INNER JOIN patient AS p
->   ON p.patient_id = a.patient_id
-> INNER JOIN prescription AS pres
->   ON pres.appointment_id = a.appointment_id
-> WHERE a.status = "Completed";
```

prescription_id	Patient_name	Doctor_name	Medicine_name	Dosage	Duration
2	Dinithi Wickramasinghe	Dilani Rajapaksa	Cetirizine Syrup	2.5ml once daily at night	5
4	Dinithi Wickramasinghe	Dilani Rajapaksa	Oral Rehydration Salts	200ml after each loose stool	3
1	Dinithi Wickramasinghe	Dilani Rajapaksa	Paracetamol Syrup	5ml three times daily	5
3	Dinithi Wickramasinghe	Dilani Rajapaksa	Vitamin C Syrup	5ml once daily	7
6	Sithum Jayawardena	Sunil Perera	Aspirin	75mg once daily	30
7	Sithum Jayawardena	Sunil Perera	Atorvastatin	20mg once daily at night	60
5	Sithum Jayawardena	Sunil Perera	Metoprolol	50mg once daily	30
8	Sithum Jayawardena	Sunil Perera	Pantoprazole	40mg once daily	14
9	Anjana Bandara	Chaminda Bandara	Amoxicillin	500mg three times daily	7
10	Anjana Bandara	Chaminda Bandara	Ibuprofen	400mg twice daily	5
11	Anjana Bandara	Chaminda Bandara	Povidone Iodine Ointment	Apply thin layer twice daily	7
14	Kamal Dias	Kamani Silva	Betahistine	16mg twice daily	30
12	Kamal Dias	Kamani Silva	Cinnarizine	25mg three times daily	14
15	Kamal Dias	Kamani Silva	Diazepam	2mg once daily at night	7
13	Kamal Dias	Kamani Silva	Metoclopramide	10mg three times daily	7
17	Nuwan Senanayake	Dilani Rajapaksa	Cetirizine Syrup	2.5ml once daily	3
16	Nuwan Senanayake	Dilani Rajapaksa	Paracetamol Syrup	5ml every 6 hours if needed	2
20	Ishara Rajapaksa	Ruwan Fernando	Calcium Carbonate	500mg twice daily	30
18	Ishara Rajapaksa	Ruwan Fernando	Diclofenac	50mg twice daily	10
19	Ishara Rajapaksa	Ruwan Fernando	Methocarbamol	750mg three times daily	7
21	Ishara Rajapaksa	Ruwan Fernando	Vitamin D3	1000 IU once daily	60
22	Malsha Gunawardena	Chaminda Bandara	Cefuroxime	250mg twice daily	7
25	Malsha Gunawardena	Chaminda Bandara	Lactulose	15ml twice daily	7
23	Malsha Gunawardena	Chaminda Bandara	Metronidazole	400mg three times daily	5
26	Malsha Gunawardena	Chaminda Bandara	Omeprazole	20mg once daily	14
24	Malsha Gunawardena	Chaminda Bandara	Tranadol	50mg every 8 hours as needed	5
28	Thilini Amarasinghe	Kamani Silva	Amisriptyline	10mg once daily at night	30
30	Thilini Amarasinghe	Kamani Silva	Naproxen	250mg twice daily	7
29	Thilini Amarasinghe	Kamani Silva	Propranolol	40mg twice daily	30
27	Thilini Amarasinghe	Kamani Silva	Sunatriptan	50mg at onset of headache	10

30 rows in set (0.00 sec)

Q32.

```
mysql> SELECT CONCAT(p.first_name, " ", p.last_name) as Patient_name,  
-> COUNT(a.appointment_id) AS total_appointments  
-> FROM patient AS p  
-> LEFT JOIN appointment AS a  
-> ON a.patient_id = p.patient_id  
-> GROUP BY patient_name;
```

Patient_name	total_appointments
Kamal Dias	2
Nimasha Silva	1
Ruwan Abeysekara	2
Dinithi Wickramasinghe	1
Sithum Jayawardena	1
Anjana Bandara	1
Tharaka Dissanayake	1
Sachini Wijesinghe	0
Nuwan Senanayake	1
Dilani Perera	1
Chamara Fernando	0
Ishara Rajapaksa	1
Kasun Kumara	0
Malsha Gunawardena	1
Dilan Madushanka	1
Thilini Amarasinghe	1
Lasith Pathirana	0
Nadeesha Ranasinghe	1
Buddhika Weerasinghe	0
Sewwandi Liyanage	1
Ravindu Lakshan	0
Hiruni Jayasuriya	1
Asanka Rathnayake	1
Dinusha Herath	1
Pradeep Sampath	1
Chathurika Muthukumarana	1
Vimukthi Siriwardena	1
Erandhi Dahanayake	1
Gayan Wickrama	0
Hashini Tennakoon	1

30 rows in set (0.00 sec)

Q33.

```
SELECT w.ward_name,  
       COUNT(n.nurse_id) AS total_nurses  
FROM ward AS ' at line 1  
mysql> SELECT w.ward_name,  
->         COUNT(n.nurse_id) AS total_nurses  
-> FROM ward AS w  
->     LEFT JOIN nurse AS n  
->         ON w.ward_id = n.ward_id  
-> GROUP BY w.ward_name;
```

ward_name	total_nurses
Cardiology General Ward	6
Cardiology ICU	0
Neurology General Ward	5
Neurology Surgical Ward	0
Ortho Surgical Ward	2
Ortho General Ward	0
Paediatric Ward	5
Paediatric ICU	2

8 rows in set (0.00 sec)

Q34.

```
mysql> SELECT CONCAT(d.first_name, " ", d.last_name) AS Doctor_name,
->          COUNT(a.appointment_id) AS appointment_count
-> FROM appointment AS a
->      RIGHT JOIN doctor AS d
->          ON d.doctor_id = a.doctor_id
-> GROUP BY CONCAT(d.first_name, " ", d.last_name)
-> ORDER BY appointment_count DESC;
```

Doctor_name	appointment_count
Sunil Perera	6
Kamani Silva	6
Ruwan Fernando	5
Dilani Rajapaksa	4
Chaminda Bandara	4
Kasun Wijesinghe	0
Thilanka Perera	0
Nadeesha Fernando	0
Lasith Bandara	0
Hiruni Dissanayake	0
Sachini Rajapaksa	0
Nuwan Jayawardena	0
Chamari Silva	0
Tharaka Kumari	0
Malsha Senanayake	0
Dinusha Wickramasinghe	0
Sewwandi Abeysekara	0
Kasuni Gunawardena	0
Dilini Pathirana	0
Roshan Mendis	0

20 rows in set (0.00 sec)

Q35.

```
mysql> SELECT dept.dept_name AS Department_name,
->          COUNT(a.appointment_id) AS Total_appointments,
->          SUM(b.total_amount) AS Gross_revenue,
->          SUM(b.paid_amount) AS Total_collected_amount,
->          SUM(b.total_amount) - SUM(b.paid_amount) AS Outstanding_balance
-> FROM department AS dept
->      INNER JOIN doctor AS d
->          ON d.dept_id = dept.dept_id
->      INNER JOIN appointment AS a
->          ON d.doctor_id = a.doctor_id
->      INNER JOIN bill AS b
->          ON b.appointment_id = a.appointment_id
-> GROUP BY dept.dept_name
-> ORDER BY Gross_revenue;
```

Department_name	Total_appointments	Gross_revenue	Total_collected_amount	Outstanding_balance
Paediatrics	3	23000.00	23000.00	0.00
Neurology	3	33500.00	13500.00	20000.00
Orthopedics	3	42000.00	28000.00	14000.00
Cardiology	6	144500.00	62000.00	82500.00

4 rows in set (0.00 sec)

Q36.

```
mysql> SELECT CONCAT(first_name, " ", last_name) AS Patient_name,  
-> phone AS Phone_no  
-> FROM patient  
-> WHERE patient_id IN (SELECT DISTINCT p.patient_id  
-> FROM patient AS p  
-> INNER JOIN appointment AS a  
-> ON a.patient_id = p.patient_id  
-> INNER JOIN bill AS b  
-> ON a.appointment_id = b.appointment_id  
-> WHERE b.payment_status IN ("Unpaid", "Partial"));
```

Patient_name	Phone_no
Sithum Jayawardena	+94765234565
Anjana Bandara	+94766234566
Kamal Dias	+94761234561
Malsha Gunawardena	+94774234574
Thilini Amarasinghe	+94776234576
Pradeep Sampath	+94785234585
Vimukthi Siriwardena	+94787234587
Ruwan Abeysekara	+94763234563
Tharaka Dissanayake	+94767234567

9 rows in set (0.01 sec)

Q37.

```
mysql> SELECT CONCAT(d1.first_name, " ", d1.last_name) AS Doctor_name,  
-> dept.dept_name AS Department_name,  
-> d1.salary AS Salary  
-> FROM doctor AS d1  
-> INNER JOIN department AS dept  
-> ON dept.dept_id = d1.dept_id  
-> WHERE d1.salary > (SELECT AVG(d2.salary)  
-> FROM doctor AS d2  
-> WHERE d2.dept_id = d1.dept_id);
```

Doctor_name	Department_name	Salary
Chaminda Bandara	Cardiology	176000.00
Kasun Wijesinghe	Cardiology	170500.00
Kamani Silva	Neurology	145000.00
Lasith Bandara	Neurology	147000.00
Nuwan Jayawardena	Orthopedics	152000.00
Chamari Silva	Orthopedics	146000.00
Malsha Senanayake	Orthopedics	150000.00
Dinusha Wickramasinghe	Paediatrics	144000.00
Sewwandi Abeysekara	Paediatrics	141000.00
Dilini Pathirana	Paediatrics	145000.00

10 rows in set (0.00 sec)

Q38.

PRIMARY KEYS

- dept_id
- ward_id
- doctor_id
- nurse_id
- patient_id
- appointment_id
- prescription_id
- bill_id

TABLE NAME	COLUMN(S) USED	REASON
department	dept_id	dept_id uniquely identify each department record , it cannot be duplicated and cannot be null.
ward	ward_id	ward_id uniquely identify each ward record , it cannot be duplicated and cannot be null.
doctor	doctor_id	doctor_id uniquely identify each doctor record , it cannot be duplicated and cannot be null.
nurse	nurse_id	nurse_id uniquely identify each nurse record , it cannot be duplicated and cannot be null.
patient	patient_id	patient_id uniquely identify each patient record , it cannot be duplicated and cannot be null.
appointment	appointment_id	appointment_id uniquely identify each appointment record , it cannot be duplicated and cannot be null.
prescription	prescription_id	prescription_id uniquely identify each prescription record , it cannot be duplicated and cannot be null.
bill	bill_id	bill_id uniquely identify each bill record , it cannot be duplicated and cannot be null.

Q39.

FOREIGN KEY CONSTRAINTS

- fk_ward_dept
- fk_doctor_dept
- fk_nurse_ward
- fk_appt_patient
- fk_appt_doctor
- fk_prescription_appointment
- fk_bill_appointment

REFERENCING TABLE	REFERENCED TABLE	ON DELETE BEHAVIOR AND REASON	ON UPDATE BEHAVIOR AND REASON
ward	department	RESTRICT <i>a ward cannot exist without a department. So, this will prevent from deleting the dept_id if related child rows exists in that dept_id</i>	CASCADE <i>a change in the dept_id need to automatically make that same change to the dept_id in the ward table. Because wards must be identified through departments.</i>
doctor	department	RESTRICT <i>a doctor must have a department. So deleting department_id is prohibited if it is referenced to any doctor_ids</i>	CASCADE <i>an update or change in a department_id should also reflect in the doctor table</i>
nurse	ward	RESTRICT <i>every nurse must work in a ward. So deleting a ward_id will ruin this condition. So we need to block deleting a ward_id s which are referenced the nurses</i>	CASCADE <i>with the change of the ward_id in the ward table , the referenced ward_id in nurse table should be updated. Because every time nurse_id point the correct ward_ids in that exact time. So every time the ward_id in nurse table should be updated with its parent table key</i>
appointment	patient	RESTRICT <i>the referenced patient_ids cannot be deleted. Because all appointments should remain for the legal and business purposes)</i>	CASCADE <i>update in the patient_ids in the patient table should also be reflected in the appointment table. Because every time the appointments should point the correct patient_ids on that exact time</i>

appointment	doctor	RESTRICT <i>restrict the deletion of the doctor_ids if those referenced the appointment_ids. Because appointment_id should always referenced by the doctor_id because of the business and legal purposes</i>	CASCADE <i>doctor_id in the appointment table should also be updated with the updation of the doctor_id in the doctor table. Because doctor_ids in both table for a single record should be same</i>
prescription	appointment	CASCADE <i>if we delete an appointment_id this will automatically delete the prescription records associated with that appointment_id</i>	CASCADE <i>if we update an appointment_id that should also be reflected in the prescription table especially in the column which referenced the appointment_id</i>
bill	appointment	RESTRICT <i>bill record maintainance is must in a clinic system. So, we need to restrict the deletion of a bill records when we delete the related appointment_ids</i>	CASCADE <i>the bill records every time should show the correct existing appointment_id. So, we should make the bill table to automatically update the referenced appointment_ids whenever those are being updated</i>

Q40.

```
mysql> SELECT CONCAT(p.first_name, " ", p.last_name) AS Patient_name,
->      p.phone AS Phone_no,
->      SUM(b.total_amount) AS Total_bill_amount
-> FROM appointment AS a
->      INNER JOIN bill AS b
->      ON b.appointment_id = a.appointment_id
->      INNER JOIN patient AS p
->      ON p.patient_id = a.patient_id
-> GROUP BY Patient_name, Phone_no
-> ORDER BY Total_bill_amount DESC
-> LIMIT 1;
```

Patient_name	Phone_no	Total_bill_amount
Malsha Gunawardena	+94774234574	32000.00

Q41.

TABLE NAME	COLUMN NAME	APPLIED BUSINESS RULE
department	dept_name	department names cannot be duplicated.
doctor	email	one email can only be assigned for one doctor. And it is not mandatory for a doctor to provide an email to the clinic administration.
nurse	email	one email can only be assigned for one nurse. And it is not mandatory for a nurse to provide an email to the clinic administration.
bill	appointment_id	one appointment can have only one bill and it is not mandatory for an appointment to have a bill.
prescription	prescription_id + appointment_id (composite unique constraint)	one appointment_id and prescription_id can participate in only one row and the combination is not mandatory for all records. Prescription_id is must , because it has defined as the primary key of that table. but appointment_id is not mandatory to participate in a record.

Q42.

```
mysql> SELECT d.*, w.ward_name
-> FROM department AS d
-> LEFT JOIN ward AS w
-> ON w.dept_id = d.dept_id
-> WHERE ward_name IS NULL;
```

dept_id	dept_name	location	phone_ext	head_doctor	ward_name
5	General Medicine	E block, 1st floor	0112345674	Dr. Asanka Jayawardena	NULL
6	Radiology	F block, 2nd floor	0112345675	Dr. Priya Mendis	NULL

2 rows in set (0.00 sec)

Q43.

```
mysql> SELECT CONCAT(d.first_name, " ", d.last_name) AS Doctor_name,
-> a.status AS Appointment_status,
-> COUNT(a.appointment_id) AS Total_appointments
-> FROM doctor AS d
-> INNER JOIN appointment AS a
-> ON d.doctor_id = a.doctor_id
-> GROUP BY d.doctor_id, Doctor_name, Appointment_status
-> ORDER BY Doctor_name;
```

Doctor_name	Appointment_status	Total_appointments
Chaminda Bandara	Completed	3
Chaminda Bandara	Scheduled	1
Dilani Rajapaksa	Completed	3
Dilani Rajapaksa	Cancelled	1
Kamani Silva	Scheduled	4
Kamani Silva	Completed	2
Ruwan Fernando	Cancelled	1
Ruwan Fernando	Scheduled	2
Ruwan Fernando	Completed	2
Sunil Perera	Cancelled	2
Sunil Perera	Completed	2
Sunil Perera	Scheduled	2

12 rows in set (0.00 sec)

Q44.

TABLE	COLUMN	CHECK SYNTAX	RULE ENFORCED	PREVENTED INVALID TYPES OF DATA
department	phone_ext	CHECK (LENGTH(phone_ext) = 10)	<i>the length of the data in the column must be exact 10 characters.</i>	<i>more or less length values than 10 are prevented</i>
ward	capacity	CHECK (capacity BETWEEN 1 AND 100)	<i>the column must have the value between 1 and 100 (inclusive)</i>	<i>the values not between 1 and 100 are prohibited</i>
doctor	salary	CHECK(salary > 0)	<i>the values of the column must be positive values.</i>	<i>the minus values are prohibited</i>
prescription	duration_days	CHECK (duration_days BETWEEN 1 AND 365)	<i>the value in the column must be between 1 and 365 (inclusive)</i>	<i>the values not between 1 and 365 are prohibited</i>
bill	paid_amount , total_amount	CHECK (paid_amount <= total_amount)	<i>the total_amount always want to be greater than the paid_amount in the same row.</i>	<i>paid amount being bigger than total amount is prohibited</i>

Q45.

```
mysql> SELECT CONCAT(p.first_name, " ", p.last_name) AS Patient_name,  
-> COUNT(DISTINCT dept.dept_id) AS Number_of_different_department_visited  
-> FROM patient AS p  
-> INNER JOIN appointment AS a  
-> ON a.patient_id = p.patient_id  
-> INNER JOIN doctor AS d  
-> ON d.doctor_id = a.doctor_id  
-> INNER JOIN department AS dept  
-> ON dept.dept_id = d.dept_id  
-> GROUP BY p.patient_id , Patient_name  
-> HAVING Number_of_different_department_visited > 1;  
+-----+-----+  
| Patient_name | Number_of_different_department_visited |  
+-----+-----+  
| Kamal Dias | 2 |  
+-----+-----+  
1 row in set (0.00 sec)
```

Q46.

```
mysql> ALTER TABLE prescription  
-> ADD CONSTRAINT chk_pres_duration  
-> CHECK(duration_days BETWEEN 1 AND 365);  
Query OK, 30 rows affected (0.18 sec)  
Records: 30 Duplicates: 0 Warnings: 0
```

Q47.

Since I have chosen "ON DELETE RESTRICT" I cannot delete a department record which is referenced with doctor records.

YES, I will change the "ON DELETE RESTRICT" to "ON DELETE SET NULL" .

Because,

ON DELETE CASCADE	<i>automatically delete the child records if the parent table's primary key is deleted</i>
ON DELETE RESTRICT	<i>preventing the deletion of a parent table's primary key if it referenced with even one child row.</i>
ON DELETE SET NULL	<i>automatically set the child table referenced column record to null if the parent table referenced primary key is deleted.</i>

Q48.

```
ALTER TABLE
bill AS b JOIN appointment AS a
ON b.appointment_id = a.appointment_id
ADD CONSTRAINT chk_bill_appt
CHECK(b.billing_date>=a.appt_date);
```

According to MySQL cannot use JOIN inside an ALTER TABLE statement. According to the logic I write this .

Q49.

PRIMARY KEY	UNIQUE
• cannot be duplicated • cannot be null eg – doctor_id in doctor table	• cannot be duplicated • can be null eg – email in the doctor and nurse table

Q50.

Referential integrity is a concept in a database which ensure the connections between the tables and the includes the conditions applied to those connections.

Ex –

CONNECTION	Referencing table	ward
	Referenced table	department
CONDITION	ON DELETE behavior	ON DELETE + RESTRICT
	ON UPDATE behavior	ON UPDATE + CASCADE

The above table tells an example for where a referential integrity getting protected. Conditions command the database to not delete/restrict deletion the referenced primary key of the payment table by "ON DELETE RESTRICT" statement. And allow updating the primary key of the parent table and also update the referenced child column data in the child table using "ON UPDATE CASCADE" .